REMOTE & CLOUD

# 远程之门

## 远程操作的技术细节

THE REMOTE GATEWAY

**Dr. Dezeming**

2026

从终端登录到对象存储

# 远程之门

远程操作的技术细节

Dr. Dezeming

2026

# 远程之门

## 版权

本书作者为 Dr. Dezeming。

书中出现的所有主机名、IP 地址、用户名、密钥文件名、桶名都是示例，用来演示配置的写法，不指向任何真实的服务器、账号或存储资源。请勿把示例直接用于生产环境。

书中涉及的命令与配置以 OpenSSH 9.x、Visual Studio Code 1.9x、JumpServer 3.x、腾讯云 COS 与 AWS S3 的公开接口为准。软件的版本更新可能导致界面与参数变化，遇到差异请以你所使用版本的官方文档为准。

本书的文字、插图与排版由作者完成。插图由配套代码生成，代码随书发布。未经许可，不得用于商业用途的再分发。

第一版 2026 年

# 前 言

远程操作在今天是一件太普通的事。写代码的人把程序放到云上编译，做模型的人把任务交给机房里的一排显卡，整理数据的人通过一个网址管理几百 TB 文件，运维的人要登录到几十台机器上检查状态。这些动作看起来各不相同，底下却是同一组问题：怎么把指令安全地送到远端，怎么确认对面确实是你以为的那台机器，怎么让做过的事留下痕迹，怎么在别人的磁盘上存取自己的数据。

这组问题不是一天形成的。最早的计算只有一台机器，谁要用就走到它面前。分时系统出现后，一台主机接上许多终端，终端只负责显示，真正干活的是远处那台大机器，远程操作的第一层含义在这里定下了。终端从串口线换成网络之后，数据要穿过许多不属于你的设备，明文传输变得不可接受，SSH 就是在这个背景下被发明出来的。再往后，机器归云厂商所有，数据存在对象存储里，运维被收拢到专门的系统上，技术问题开始和管理问题纠缠在一起。

这三步留下的约定叠在一起，就是今天这套工具箱。它们分属不同的年代，回答的是不同层面的问题，所以放在一起看时显得零散。理解了它们各自的来历，零散感就会消失。

## 这本书讲什么

书里讲四层东西。

第一层是链路。一条命令从键盘出发，经过网卡、路由器、运营商的骨干网，落到远端的一个进程上。这一层要回答的是“为什么连不上”，以及怎么把这句话拆成可以逐段验证的判断题。

第二层是信任。链路通了只说明字节能过去，不说明对面是谁，也不说明中间有没有人偷看。密码学在这里不是装饰，而是 SSH 每一个设计决定的来源。这一层要回答的是“凭什么相信”，以及这些信任在什么条件下会失效。它顺带解决了另一个问题：当你不在那张网里的时候，怎么把整张网搬过来，这就是 VPN。

第三层是控制。一台机器一个人的时候，信任可以靠熟人关系维持。几百台机器几十个人的时候，就必须靠制度和技术。堡垒机把分散的凭据收拢，把操作记录下来。远程开发把开发环境整个搬到远端，只把界面留在手边。这一层要回答的是“怎么管住”，以及管住之后付出了什么代价。

第四层是数据。代码跑起来会产生产出，产出要存在别人的磁盘上，还要能被下一台机器读到。对象存储的接口简单到只有几个 HTTP 方法，可用起来处处是坑。这一层要回答的是“怎么存取”，以及为什么一个字节的差异就会得到 403。

四层之间是依赖关系。不懂链路就看不懂信任里的握手，不懂信任就看不懂堡垒机为什么要托管密钥，不懂控制就看不懂远程开发为什么需要端口转发。所以这本书按这个顺序写，每一章都建立在前面几章之上。

## 怎么读

每一章的结构是一样的。先说清这一章要解决的问题，再讲解决它的机制，最后落到你在终端、配置文件、日志里实际会看到的东西。机制部分是重点，操作部分是它的自然结果。如果只想要命令，附录 A 有速查表，但那份表单独看没有多大价值。

书里的图和表都服务于理解，不追求覆盖所有细节。图注写在图的下方，正文会在需要时引用编号。代码块只放能直接运行或者能直接粘贴的内容，注释用英文，避免等宽字体缺字。

阅读时建议手边有一台可以折腾的机器。很多概念看起来抽象，实际敲一遍就清楚了。比如把 `ssh -v` 的输出从头读到尾，比读三页协议描述更有效。书里会指出哪些输出值得逐行看。

## 这本书假设你会什么

假设你用过 Windows 或 macOS 的电脑，会在终端里敲几条命令，知道文件和目录的区别，写过程序，哪怕只是 Python 的入门水平。不假设你学过计算机网络，不假设你懂密码学，也不假设你管过 Linux 服务器。需要的背景知识，书里会从头讲起。

占位符写成 〈你的用户名〉 这样的形式，需要替换成你自己的信息。书里出现的地址都是文档专用地址段，不对应任何真实资产。

## 关于版本

书里的命令以 OpenSSH 9.x、VS Code 1.9x、JumpServer 3.x、腾讯云 COS 与 AWS S3 的公开接口为准。这些软件都在持续演进，算法默认值、界面文字、命令行参数都可能变化。遇到不一致时，先看 `--help` 和你所用版本的官方文档，再用书里的原理去判断差异。

原理的寿命比参数长。十五年之后，具体的加密算法可能换了一轮，配置文件里可能多出几个新选项，但这本书里讲的分层、认证、授权、审计、签名，仍然会是理解新工具的那把钥匙。

Dr. Dezeming

2026 年于实验室

# 目 录

**前 言** . . . . . 5

这本书讲什么 . . . . . 5

怎么读 . . . . . 6

这本书假设你会什么 . . . . . 6

关于版本 . . . . . 6

**第 1 章 计算机之间怎么通信** . . . . . 15

1.1 先要有线 . . . . . 15

1.2 局域网与地址 . . . . . 15

1.3 出了局域网要找网关 . . . . . 16

1.4 数据是怎么走的 . . . . . 16

1.5 协议：先把话怎么说定下来 . . . . . 17

1.6 明文与密文 . . . . . 17

1.7 早期的远程登录协议 telnet . . . . . 18

1.8 客户端与服务端 . . . . . 18

1.9 一次访问经过的路 . . . . . 18

1.10 这一章的位置 . . . . . 19

**第 2 章 SSH 是什么** . . . . . 20

2.1 从明文到加密 . . . . . 20

2.2 SSH 的定义，以及它保证的四件事 . . . . . 20

2.3 SSH 不是什么 . . . . . 21

2.4 远程访问一定要经过堡垒机吗 . . . . . 22

2.5 一次连接里的四个角色 . . . . . 24

2.6 最小的一次使用 . . . . . 24

2.7 一次会话的轮廓 . . . . . 25

2.8 全书的知识地图 . . . . . 27

**第 3 章 地址、端口与那条看不见的路 . . . . . 28**

3.1 一台机器在网上叫什么 . . . . . 28

3.2 端口是同一台机器上的不同门 . . . . . 28

3.3 NAT 与地址改写 . . . . . 29

3.4 域名只是给人看的 . . . . . 30

3.5 TCP 保证了什么 . . . . . 30

3.6 UDP 少做了什么 . . . . . 32

3.7 数据是怎么被装进包里的 . . . . . 33

3.8 自己动手看连接 . . . . . 34

3.9 常见端口一览 . . . . . 34

**第 4 章 服务器上发生了什么 . . . . . 36**

4.1 登录成功那一刻 . . . . . 36

4.2 shell 是什么 . . . . . 36

4.3 进程与 fork . . . . . 37

4.4 三个标准文件描述符 . . . . . 37

4.5 终端与伪终端 . . . . . 38

4.6 把这条链拼起来 . . . . . 39

4.7 用户、组与权限 . . . . . 40

4.8 该记住的几个路径 . . . . . 41

4.9 环境变量与 PATH . . . . . 42

4.10 常驻的服务 . . . . . 44

4.11 日志里能看到什么 . . . . . 44

**第 5 章 加密、哈希与密钥交换 . . . . . 46**

5.1 三种敌人 . . . . . 46

5.2 一把钥匙的对称加密 . . . . . 46

5.3 哈希与消息认证码 . . . . . 47

5.4 非对称加密的两个用途 . . . . . 47

5.5 密钥交换 . . . . . 48

5.6 为什么需要证书 . . . . . 50

5.7 前向保密 . . . . . 50

5.8 这些工具在 SSH 里怎么分工 . . . . . 50

**第 6 章 SSH 协议的三层结构 . . . . . 52**

6.1 一行明文的版本号 . . . . . 52

6.2 三个子协议 . . . . . 52

6.3 一个数据包长什么样 . . . . . 53

6.4 算法协商 . . . . . 54

6.5 密钥交换与密钥派生 . . . . . 55

6.6 重密钥 . . . . . 57

6.7 认证层 . . . . . 57

6.8 连接层 . . . . . 58

6.9 看看自己的机器支持什么 . . . . . 58

6.10 抓包能看到什么 . . . . . 58

**第 7 章 密钥、指纹与信任 . . . . . 60**

7.1 生成一对钥匙 . . . . . 60

7.2 私钥文件里装的是什么 . . . . . 60

7.3 公钥认证到底怎么验证 . . . . . 61
7.4 authorized_keys 的写法 . . . . . 62
7.5 主机密钥与首次信任 . . . . . 63
7.6 ssh-agent 与密钥转发 . . . . . 65
7.7 证书 . . . . . 65
7.8 一份能用的客户端配置 . . . . . 66

**第 8 章 端口转发，把远方的服务搬到本地 . . . . . 68**

8.1 只开一个端口的世界 . . . . . 68
8.2 本地转发 . . . . . 68
8.3 远程转发 . . . . . 69
8.4 动态转发 . . . . . 70
8.5 跳板与 ProxyJump . . . . . 71
8.6 转发与堡垒机的关系 . . . . . 72
8.7 让隧道稳定下来 . . . . . 72
8.8 一条通道能装多少东西 . . . . . 73

**第 9 章 VPN：把一整张内网搬到身边 . . . . . 74**

9.1 一条走不通的地址 . . . . . 74
9.2 三个动作：隧道、虚拟网卡、路由 . . . . . 74
9.3 封装：包外面再套一个包 . . . . . 75
9.4 隧道两端的互相信任 . . . . . 76
9.5 全隧道与拆分隧道 . . . . . 77
9.6 连不上时按这个顺序查 . . . . . 77
9.7 校园网里的那个客户端 . . . . . 78
9.8 它和端口转发、堡垒机各管什么 . . . . . 80

**第 10 章 为什么需要堡垒机 . . . . . 82**

10.1 一个常见的事故 . . . . . 82

10.2 网状连接的代价 . . . . . 82

10.3 4A . . . . . 83

10.4 流量怎么走 . . . . . 85

10.5 合规要求 . . . . . 86

10.6 堡垒机保护不了什么 . . . . . 86

**第 11 章 JumpServer 的内部构造 . . . . . 87**

11.1 组件地图 . . . . . 87

11.2 资产模型的四层 . . . . . 88

11.3 三合一用户名 . . . . . 89

11.4 一次 SSH 网关会话 . . . . . 90

11.5 命令过滤与会话录像 . . . . . 90

11.6 凭据是怎么存的 . . . . . 91

11.7 部署时的几个关键项 . . . . . 91

**第 12 章 VS Code 远程开发的工作原理 . . . . . 93**

12.1 一个编辑器，四个进程 . . . . . 93

12.2 连接时发生了什么 . . . . . 94

12.3 那条隧道的两端 . . . . . 95

12.4 ssh_config 的解析规则 . . . . . 95

12.5 逐行拆解那份配置 . . . . . 96

12.6 出问题时看哪里 . . . . . 98

**第 13 章 对象存储的模型 . . . . . 100**

13.1 为什么不用文件夹 . . . . . 100

13.2 三种存储形态 . . . . . 101
13.3 接口就是 HTTP . . . . . 102
13.4 大文件怎么传 . . . . . 102
13.5 元数据与存储类型 . . . . . 103
13.6 一致性与持久性 . . . . . 104
13.7 工具与挂载 . . . . . 104
13.8 和堡垒机的关系 . . . . . 104

**第 14 章 签名、密钥与 403 . . . . . 106**

14.1 每个请求都要证明自己 . . . . . 106
14.2 访问密钥 . . . . . 106
14.3 签名的计算过程 . . . . . 106
14.4 预签名 URL . . . . . 109
14.5 临时密钥 . . . . . 110
14.6 权限是怎么判的 . . . . . 110
14.7 403 家族 . . . . . 111
14.8 几条实践建议 . . . . . 112

**第 15 章 从登录到数据存取 . . . . . 113**

15.1 一个真实的工作流 . . . . . 113
15.2 第一步，把登录打通 . . . . . 114
15.3 第二步，让编辑器连上 . . . . . 115
15.4 第三步，让训练机读写桶 . . . . . 116
15.5 第四步，把结果取回来 . . . . . 117
15.6 排错的时候按顺序来 . . . . . 117
15.7 一份可以照着做的检查清单 . . . . . 117

15.8 最后 . . . . . 118

**附录 A 命令速查** . . . . . **119**

A.1 客户端连接 . . . . . 119

A.2 密钥管理 . . . . . 119

A.3 agent . . . . . 120

A.4 诊断 . . . . . 120

A.5 服务端 . . . . . 120

A.6 VPN 与接入 . . . . . 121

A.7 对象存储 . . . . . 121

A.8 常用配置项 . . . . . 122

**附录 B 术语表** . . . . . **123**

**后 记** . . . . . **127**

第 一 篇

# 从一台机器到一条链路

远程操作最容易被忽略的一点是，它其实由两段完全不同的知识拼成。一段在机器外面，讲的是设备怎么连起来、地址怎么分配、数据怎么穿过一层层网络。另一段在机器里面，讲的是登录之后谁在等你、命令怎么变成进程、权限怎么生效。

两段知识都不难，难在它们平时被工具挡着。`ssh` 一条命令就把两段都替你走完了，顺利的时候你不需要知道任何细节，不顺利的时候又不知道该往哪一段去找原因。

这一篇分四章。第一章从最基础的网络讲起，网卡、交换机、路由器、网关、地址、网段，这些词先立住。第二章回到这本书的主角，说清 SSH 到底是什么、保证什么、不保证什么，以及它和堡垒机的关系。第三章往下走一层，把地址、端口、数据包的结构和排错方法讲透。第四章换到服务器内部，从 sshd 接受连接开始，一直走到你看到的那个提示符。

走完这四章，“连不上”这句话应该能拆成几个可以分别验证的小问题。

第 1 章

# 计算机之间怎么通信

## 1.1 先要有线

两台计算机要交换数据，中间必须有某种连接。最常见的是网线。网线两端插在机器背后的网口上，网口后面是一块叫网卡的硬件，它负责把计算机内部的 0 和 1 变成线路上可以传输的电信号，也负责把收到的电信号还原成 0 和 1。

一台机器通常只有一两个网口。要让五台机器互相通信，不可能两两拉线，于是有了交换机。交换机是一个带很多网口的小盒子，所有机器都连到它上面，它负责把一台机器发出的数据转给另一台。交换机不关心数据的内容，只关心“这条数据要送给谁”。

无线网络把这根线换成了无线电波。机器里装一块无线网卡，通过天线收发信号，房间里再放一个接入点，所有无线设备都连到它上面。接入点的作用和交换机类似，只是连接方式从线变成了波。

把上面这些设备连在一起，就形成了一个网络。网络这个词没有更神秘的含义，它就是一组能够互相发送数据的设备。

## 1.2 局域网与地址

一个房间、一层楼、一个机房里连起来的网络，叫局域网。局域网的范围通常在几十米到几百米，家里的 Wi-Fi、办公室的网线、机房里机柜之间的连线，都是局域网。

局域网里的设备多了，就需要地址。交换机把数据转给谁，靠的就是地址。最常用的地址格式叫 IP 地址，写成四组数字，中间用点隔开，比如 `192.168.1.23`。这个地址在同一个局域网里必须唯一，否则数据会送错。

地址还有一层含义，叫网段。网段指的是前几组数字相同的一组地址。`192.168.1.23`、`192.168.1.24`、`192.168.1.25` 属于同一个网段，因为前三组都是 `192.168.1`。同一个网段里的设备可以直接把数据发给对方，交换机就能完成转发，不需要别的设备帮忙。

图 1-1 画的是一个典型的局域网。几台电脑连到交换机上，交换机再连到一台路由器，路由器通向外部。

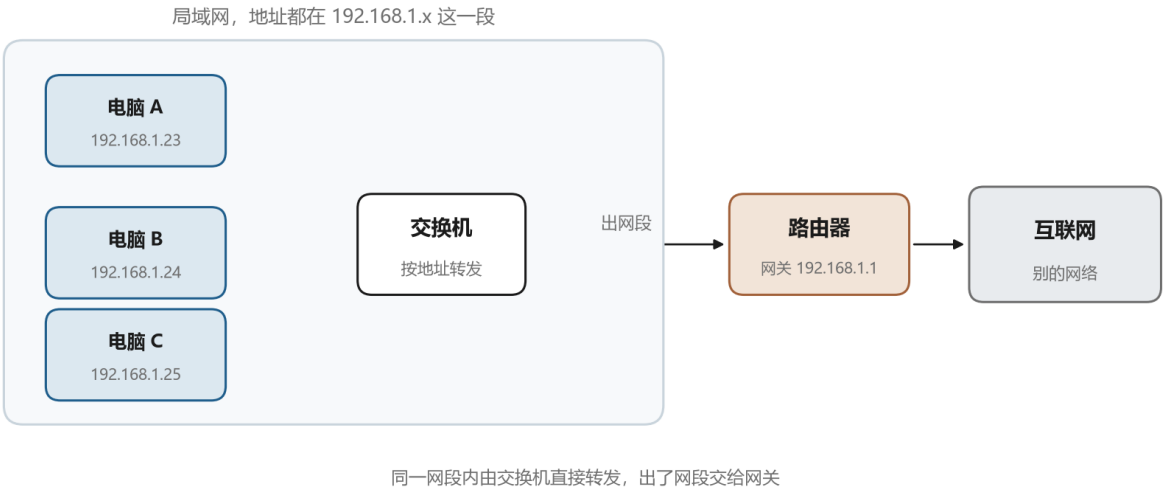



图 1-1 一个局域网里的设备与网段

## 1.3 出了局域网要找网关

局域网再大也有边界。数据要送到另一个网络里的机器，交换机就无能为力了，它只认识自己这一片。承担跨网络转发任务的是路由器。

路由器的工作是连接两个或多个网络。它有好几个网口，每个网口接一个网络，收到数据之后看目的地属于哪个网络，再从对应的网口送出去。家里的那台“路由器”其实是个合体设备，里面同时有路由器、交换机、无线接入点和一层简单的防护功能，所以它既能发 Wi-Fi，又能让手机和电脑互相访问，还能连上运营商。

数据离开本网段时，要交给一台特定的设备，这台设备叫网关。网关通常是路由器在本网段里的那个地址，比如 `192.168.1.1`。可以把它理解成小区的大门，所有出门的包裹都要先送到门卫那里，由门卫决定往哪条路送。

把无数个网络用路由器连起来，就是互联网。互联网没有中心，它是一张由运营商、数据中心、海底光缆拼成的网，任何一个网络都只是其中一个节点。

## 1.4 数据是怎么走的

网络传输的不是一个完整的文件，而是一个个小块。发送方把数据切成小块，每块前面加上一段说明，写明从哪来、到哪去、属于哪个文件的第几块。这种带说明的小块叫数据包，通常简称包。

包交给网卡，网卡把里面的 0 和 1 变成电信号或者无线电波送出去。沿途的交换机、路由器只看包头里的地址，不看内容，把包一段段往下传。对端的网卡收到信号，还原成 0 和 1，再交给操作系统，操作系统按包头里的序号把小块拼回原来的数据。图 1-2 是这个过程的示意。

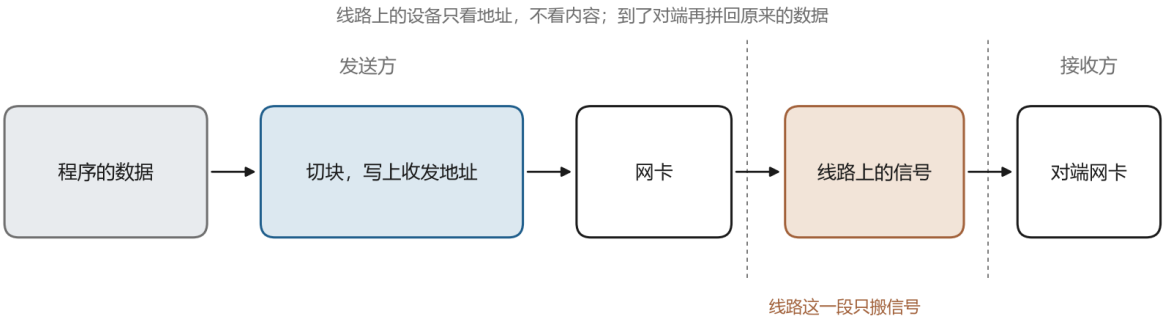



图 1-2 数据从程序到线路，再在对端还原

这个设计和寄快递很像。包裹上写清收件地址，分拣中心只扫地址，不拆包裹。包裹可能经过好几个中转站，每一站只负责往下一站送。

## 1.5 协议：先把话怎么说定下来

不同厂商生产的网卡、交换机、路由器要能互通，就必须事先约定数据长什么样、先发什么后发什么、收到之后怎么回应。这种约定叫协议。

协议在生活里到处都是。寄信要按格式写邮编、地址、收件人，否则分拣不了。打电话要先说“喂”，确认对方在听，再讲正事。网络协议也是这个意思，只不过约定得非常精确，精确到每一个比特的位置。

网络里的协议不止一个，而是一组，各管一段。有的负责把 0 和 1 送到隔壁机器，有的负责把包从北京送到广州，有的负责在丢包时重发，有的负责把数据加密。它们叠在一起工作，形成分层结构，第三章会把这套分层拆开讲。现在只需要记住一件事，网络不是一块铁板，它由许多各管一段的约定拼成。

## 1.6 明文与密文

数据在线路上传输时，本质上是一串 0 和 1。如果这串 0 和 1 就是原始内容，没有做任何伪装，那么任何人拿到它都能直接读出来。这种状态叫明文。

明文的问题在于，线路上不只有你和对方。数据要经过交换机、路由器、运营商的设备，还要经过无数中间网络。这些设备的所有者并不是你，理论上他们都能把经过的数据复制一份。共享 Wi-Fi 的场景更直接，同一个无线网络里的其他设备，只要愿意，就有机会收到不属于自己的信号。

解决办法是加密。发送方用一套规则把明文变换成看起来毫无意义的内容，这叫密文。只有掌握对应规则的人才能把它还原，这个还原过程叫解密。加密的规则由一把密钥控制，密钥不同，同样的明文会变成完全不同的密文。

打个比方。明文是明信片，一路上谁都能读。密文是装进保险箱的纸条，路上的人只能看到箱子，打不开。密钥就是那把钥匙。

加密能解决“被看到”的问题，但还有两件事要处理。一是内容可能被改动，二是对面可能不是你以为的那台机器。这两件事在后面的章节里会分别展开，第五章讲加密与校验，第七章讲身份与指纹。

## 1.7 早期的远程登录协议 telnet

有了网络之后，人们很自然地想到用它来操作远处的机器。1969 年出现的 telnet 就是做这件事的。它让本地的一台机器变成远端机器的终端（终端指负责显示和输入的那一端，本身不做计算），你在本地敲键盘，字符被送到远端，远端的输出被送回来显示。用起来和坐在那台机器面前几乎一样。

telnet 的设计非常简单，它把按键原样变成字节发出去，把回传的字节原样显示出来。简单带来了两个后果。一是兼容性好，几乎任何系统都能实现。二是没有加密，也没有可靠的身份验证，登录时输入的用户名和密码以明文形式经过整条链路。在所有人都待在同一个可信机房的年代，这个代价可以接受。网络扩展到校园、城市、跨洲之后，它就成了一个明显的漏洞。

1995 年出现的 SSH（Secure Shell，一种加密的远程登录协议）针对的正是这一点。它保留了 telnet 的使用方式，你依然是在本地敲命令、看远端输出，但整条通道是加密的，双方还要互相证明身份。今天 telnet 基本只用于调试和教学，实际登录一律用 SSH。

## 1.8 客户端与服务端

网络上的通信总是由一方发起，另一方等待。发起连接的程序叫客户端，等待连接的程序叫服务端。

浏览器是客户端，网站是服务端。你在终端里敲的 `ssh` 是客户端，远端运行的 `sshd` 是服务端。服务端要先在某个位置“守着”，等别人来连，这个守候的动作叫监听。客户端则主动向那个位置发起连接。

同一台机器可以同时扮演两个角色。你的笔记本既是浏览器客户端，也运行着各种服务端程序，随时准备接受局域网里其他设备的请求。角色取决于这一次通信里谁发起、谁响应，和机器本身无关。

## 1.9 一次访问经过的路

把上面这些串起来，一次访问的路径大致如图 1-3。

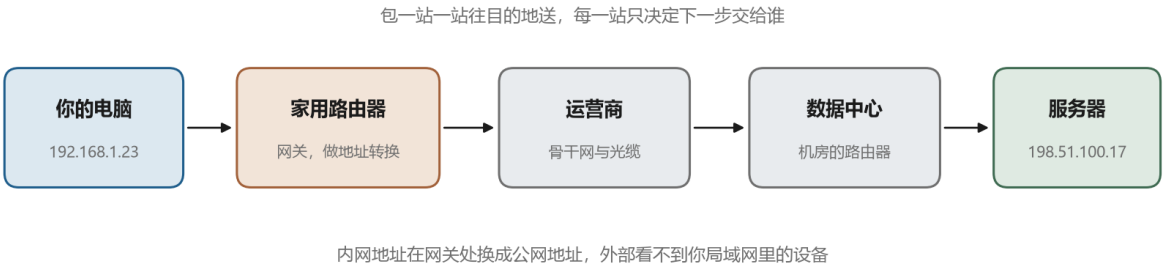



图 1-3 一次访问从本机到服务器的路径

你的电脑先判断目标地址在不在本网段。不在，就把包交给网关。网关看目的地址，决定往哪条线路送，包经过运营商的网络，经过若干台路由器，最终到达服务器所在的数据中心，再由那里的路由器送到服务器。服务器处理完，把结果按原路送回。

这条路上有两个细节值得先知道。第一，家里的网关通常会把你的内网地址换成它的公网地址再发出去，这样外部只看到网关的地址，看不到你内网的具体设备，这个机制叫地址转换，第三章会讲。第二，路上任何一跳都可能出问题，所以“连不上”从来不是一个单一的原因，而是一串可能。

## 1.10 这一章的位置

这一章只做了两件事。把网络拆成看得见的设备（网卡、交换机、路由器、网关），把通信拆成看得见的动作（切包、加地址、转发、还原）。有了这两层直觉，后面讲具体协议时就不会悬空。

第二章回到这本书的主角，说明 SSH 到底是什么、它保证什么、不保证什么。第三章再往下走一层，把地址、端口、数据包的结构和排错方法讲清楚。如果你在读后面的章节时遇到没解释的名词，多半是在这一章或第三章出现过，可以回头查。

第 2 章

# SSH 是什么

## 2.1 从明文到加密

服务器通常没有显示器和键盘。它被放在机房的机架上，或者以虚拟机的形式跑在云里，通电、联网、启动，然后就静静地等着。要让这样的机器干活，只有一条路，把指令送过去，把结果取回来。

上一章结尾提到过 telnet。它是 1969 年出现的远程登录协议，工作方式很朴素，把本地敲下的字符原样变成字节发出去，把远端回传的字节原样显示出来。问题在于它不加密，登录时输入的用户名和密码以明文形式经过整条链路。网络还局限在同一个机房时，这个代价可以接受；网络扩展到校园、城市、跨洲之后，任何一段链路上的设备都能读到这些内容。

1995 年，芬兰赫尔辛基理工大学的研究者 Tatu Ylönen 在校园网出现窃听之后写了 SSH，本意是给同事一个能安全登录的工具。它保留了 telnet 的使用方式，你依然是在本地敲命令、看远端输出，但整条通道是加密的，双方还要互相证明身份。这个工具传得很快，一年之内就从校园扩散到全世界，后来由 IETF（互联网工程任务组，负责制定互联网标准的组织）标准化为 SSH-2 协议，文档编号是 RFC 4250 到 4254（RFC 是互联网标准的文档编号体系）。1999 年，OpenBSD 项目基于当时最后一版自由发布的代码整理出了 OpenSSH（一套开源的 SSH 实现），今天绝大多数 Linux 服务器上跑的 `sshd` 都是它的后代，Windows 10 之后的系统也自带了这个客户端。

## 2.2 SSH 的定义，以及它保证的四件事

SSH 的全称是 Secure Shell，安全外壳。这个名字容易让人以为它是一个加密版的终端，实际上它是一族协议的集合，规定了两个程序在一条不安全的网络连接上，如何建立一条可信的通道，如何确认彼此的身份，如何在这条通道里开出一个终端、执行一条命令、传一个文件，或者转发一段别的流量。

把它的承诺拆开，有四条。

第一条是身份认证。客户端要能证明“我是某个账号的主人”，服务端也要能证明“我确实是你以为的那台机器”。这两件事分开做，各用各的凭据：客户端用密码、密钥或证书，服务端用它自己的一对主机密钥。

密钥这个词后面会出现很多次，先给它一个最朴素的解释：一段能用来加密、解密或签名的数据，本质上是一长串随机数。非对称加密里的密钥成对出现，一把叫私钥，只留在自己手里；另一把叫公钥，可以发出去。服务器的那一对叫主机密钥，作用一样，只是用来证明“我是这台机器”。证书是第三方出具的证明，说明某把公钥确实属于某个身份。

这些名词第五章会逐个拆开，第七章再讲服务器怎么证明自己。这里先记住一句话：认证是双向的，双方各自证明自己的身份，谁也不替谁担保。很多工具只做前者，SSH 两件都做。

第二条是机密性。连接建立之后，链路上传输的所有字节都是加密的。中间经过的路由器、交换机、运营商的接入设备能看到包的来源、去向和大小，看不到内容。

第三条是完整性。每个数据包都带有校验值，接收方先验证再使用。任何一个字节被改动，连接立刻断开，不会出现“内容被悄悄改了一半”的情况。

第四条是复用。一条 SSH 连接里可以同时开多个逻辑通道，一个跑交互式 shell（读命令、解释命令、再交给系统执行的程序，第四章细讲），一个传文件，一个转发数据库端口，互不干扰。这也是它能顶替掉一大堆专用工具的原因。

需要说明的是，授权不在这四条里。你能登录之后能做什么，由服务端的策略决定，比如 `authorized_keys`（服务端保存允许登录的公钥的文件）里的限制选项、`sshd_config` 里的规则、操作系统本身的权限。SSH 协议只负责把你安全带进门，进门之后能走到哪个房间，是另一套机制的事。第三篇讲堡垒机（一种集中管控运维访问、记录操作的系统）时，这一点会变得很重要。

## 2.3 SSH 不是什么

初学者常把几个相邻的东西混在一起。下面每一条都先说明那样东西是干什么的，再说它和 SSH 差在哪。

SSH 不是一个软件，而是一套协议。协议是双方事先约定好的通信规则，软件是这套规则的具体实现。OpenSSH 是最流行的一个实现，Linux 和 macOS 自带的 `ssh` 与 `sshd` 都属于它；Windows 上的 PuTTY（一款老牌的图形化 SSH 客户端）是另一个实现。所以“装一个 SSH”这个说法严格讲不成立，装进去的总是某个实现。

SSH 不是 VPN。VPN 的全称是虚拟专用网（Virtual Private Network），它做的事情是把两个相隔很远的网络在逻辑上接成一个内网。你在家里装一个客户端，它连到公司出口，建起一条加密隧道，再在本机加一块虚拟网卡（一块只存在于软件里的网卡），改掉路由表（决定一个数据包从哪块网卡出去的表格）。从此凡是发往公司网段的包，都先被塞进隧道送到公司，再由公司那边替你转发出去。效果是你可以用 `10.0.0.5` 这种内网地址直接访问公司的机器，浏览器和数据库客户端都不用改设置，因为它们根本不知道流量被改了道。

SSH 的端口转发能做类似的事，但机制完全不同。它不改路由，只在你要用的那个端口上单独开一条通道，用它的程序还得把目标地址改成 `127.0.0.1`（本机自己的地址）。一句话概括，VPN 是整台机器的路都改了道，SSH 转发是给某一个端口修了一条小路，修一条才通一条。

所谓修一条小路，具体就是写一段参数，说清本机的哪个端口，对着哪台机器的哪个端口。

想看内网的网页面板，写一段；想连内网的数据库，再写一段；想连缓存，还得写一段。少写一段，对应的那个服务就不通。

这跟“规则有没有配好”是两件事。服务器那边的配置只回答一个问题，允许不允许转发；它不回答另一个问题，转发到哪里。要连哪台机器、哪个端口，协议里没有地方预先记着，必须

每次连接时由你说一遍。服务是后来加的，参数就得跟着加。

代价因此也不一样：VPN 要装客户端、要配账号，配好之后什么都通；SSH 转发不用装别的东西，随手就能开，代价是要用的服务一多，命令行就越来越长。第八章讲端口转发、第九章讲 VPN 时都会回到这个对比。

SSH 不是文件同步工具。`scp` 和 `sftp` 是随 OpenSSH 一起装上的两个命令，做的是搬运：把一个文件从这边拷到那边，拷完就结束。它们不比较两边的文件是否已经一致，不判断哪一份更新，断了也不会接着刚才的位置继续。要这种“让两边保持一致”的语义，得用专门的同步工具，比如 rsync，它会先比对差异，只传变化的部分；而 rsync 自己不管加密，通常就跑在 SSH 之上。

SSH 不是配置管理工具。Ansible 这类工具用来批量配置几十上百台机器：它读一份清单（里面写着要管哪些机器），按预先写好的任务逐台执行，并且保证同一套任务跑一次和跑十次结果相同，这个性质叫幂等。它本身不实现远程连接，脚下的那条路就是 SSH。

SSH 也不等于堡垒机。SSH 是两台机器之间的协议，堡垒机是夹在中间的一个架构角色，前者是协议，后者是一个放在中间的位置。这一条值得单独说，因为它是很多人第一次接触企业环境时最容易误解的地方。

## 2.4 远程访问一定要经过堡垒机吗

不需要。

SSH 是点对点的协议。客户端直接连到服务器的 22 端口，双方认证，建立通道，这就是它最原始也最常见的用法。个人租的云主机、家里的树莓派、实验室里自己管的几台机器，都是这样用的。整个过程里没有任何“中间设备”的角色，也不需要。

堡垒机是组织规模变大之后才出现的架构角色。它不改变 SSH 协议，也不在协议里占一个位置。它的作用是替一个团队做三件事。把散落在每个人手里的密码和私钥收拢到一处，由它统一保管和代填。把“谁能连哪台机器”写成一张可以查询、可以回收的授权表。把每次会话录下来，事后能查、能回放。这三件事在只有三五台机器、三五个人的时候，靠约定和信任就能维持，人一多、机器一多、再加上合规要求，就必须有东西来承担。

所以判断标准不是“远程访问该不该有堡垒机”，而是“这个环境的规模和约束需不需要”。表 2-1 是一个粗略的对照。

表 2-1 直连与经过堡垒机的适用场景

| 场景 | 常见做法 | 原因 |
| --- | --- | --- |
| 个人服务器、家庭实验室 | 直连 | 只有一个人用，密码和密钥本来就在自己手里 |
| 小团队，机器少于十台 | 直连加统一密钥管理 | 人员流动少，口头约定够用 |

| 场景 | 常见做法 | 原因 |
| --- | --- | --- |
| 几十人、上百台机器 | 堡垒机 | 密码与密钥的分发回收成本随人数和机器数平方增长 |
| 有等保、审计要求 | 堡垒机 | 等保是国内对信息系统安全的强制分级要求，需要可追溯的会话记录 |
| 需要临时给外部人员开权限 | 堡垒机 | 可以限时、限资产、限操作，到期自动失效 |

代价也要说清楚。堡垒机是所有运维流量的必经之路，它挂了，所有人都进不去。它集中了所有资产的凭据，一旦被攻破，损失比没有堡垒机更大。它还会带来额外的网络跳数和带宽压力。这些都是真实的成本，第十章会展开讲。

理解了这一层，还有一个细节会跟着冒出来。很多人第一次连企业环境，看到的登录名是这么一个字符串。

```
alice@ubuntu@10.0.0.5
```

一个用户名里有两个 at 符号。放进一条完整的命令里，是这样。

```
ssh -p 58222 -l alice@ubuntu@10.0.0.5 203.0.113.24
```

这条命令连的是 `203.0.113.24`，也就是堡垒机，`-p 58222` 是它在监听的端口，`-l` 后面的那一整串才是用户名。三段被两个 at 符号隔开，各管一件事：`alice` 是你在堡垒机上的账号，用来证明你是你；`ubuntu` 是堡垒机替你连目标机时使用的系统账号，也就是它在自己的库里存着的那套凭据的名字；`10.0.0.5` 才是最终要去的机器。

对比一下就清楚了。平时登录只写一个 at 符号，`ssh alice@198.51.100.17`，左边是人、右边是机器。堡垒机却要在一次连接里同时知道三件事，而 SSH 的用户名字段只给了它一个字符串，于是它把三件事拼进同一个字符串，用第二个 at 符号再分一次。

所以那串字符不是 SSH 的语法，是堡垒机的约定。协议对此一无所知，它只是把用户名当成一个字符串原样传过去。把同样一串用户名发给一台普通服务器，它只会当成一个不存在的账号，直接拒绝。

这里有一个容易踩的坑。命令里不写 `-l` 时，客户端得自己从字符串里找出“用户名到哪为止、主机从哪开始”，它认的是最后一个 at 符号：最后一个 at 之前整段算用户名，之后算主机名。于是 `ssh alice@ubuntu@10.0.0.5` 会被读成“用 `alice@ubuntu` 这个账号，去连 `10.0.0.5` 这台机器”。`10.0.0.5` 只在公司内网里有意义：它在公司里是一台真实的机器，从公司的网络里访问得到；可一旦离开那张内网，这个地址就不指向任何东西——你家的路由器不认识它，包发出去没有路可走，命令只能一直等到超时。你本来想连的是堡垒机 `203.0.113.24`，是客户端把目标认错了。写成 `-l alice@ubuntu@10.0.0.5` 就没有歧义：`-l` 后面整串都算用户名，命令最后那个参数才是要连的机器。第十一章会讲网关怎么把这三段拆开，以及这种写法为什么会给一些

工具带来麻烦。

## 2.5 一次连接里的四个角色

抛开堡垒机，一次最普通的 SSH 连接涉及四样东西。图 2-1 画的是这个结构。

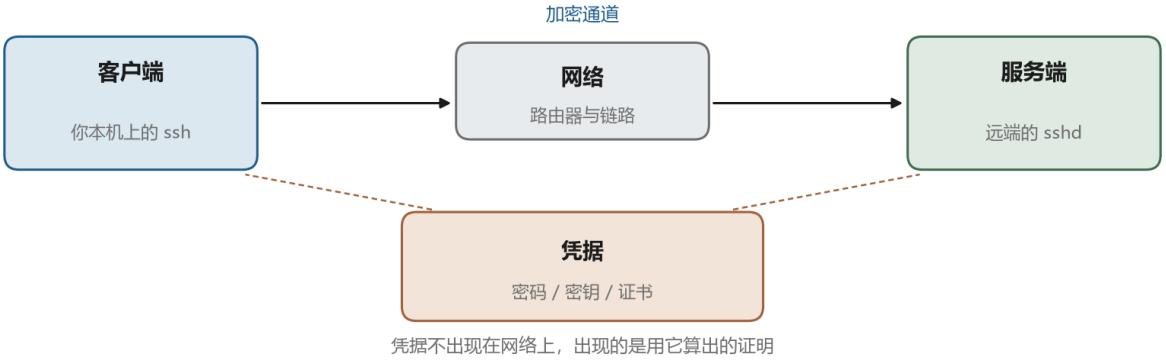



图 2-1 一次直连的三段路径

第一样是客户端。它是你本机上的一个程序，Windows 上是 `ssh.exe`，Linux 和 macOS 上通常就是 `/usr/bin/ssh`。它负责读配置、发起连接、协商算法、完成认证，然后把你的按键打包发出去。

第二样是服务端。它是远端运行的 `sshd`，默认守在一个叫 22 的端口上，等别人来连。端口是同一台机器上用来区分不同服务的编号，第三章会细讲。它负责接受连接、验证你的身份、分配会话、执行命令。

第三样是网络。两端之间可能隔着家用路由器、运营商的骨干网、云厂商的入口设备。这些设备只负责转发字节，不关心内容，但它们决定了连接能不能建立、稳不稳定、会不会中途被重置。

第四样是凭据。它是你用来证明身份的东西，可能是一个密码，一对公私钥，或者一张由 CA（证书颁发机构，专门替别人签发证书的机构）签发的证书。凭据不出现在网络上，出现在网络上的是用它算出的证明。

这四样东西里，只有凭据是“你自己的”。客户端和服务端都是现成的软件，网络更是你控制不了的。所以当连接出问题时，能改的通常只有凭据和配置，能查的则是另外三样各自留下的痕迹。

## 2.6 最小的一次使用

在终端里敲下面这一行，就完成了一次登录。

```
ssh alice@198.51.100.17
```

`alice` 是目标机上的账号，`198.51.100.17` 是它的地址。第一次连接时，会出现类似这样的提示。

```
The authenticity of host '198.51.100.17' can't be established.
ED25519 key fingerprint is SHA256:9f2c1d...（一长串字符）.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

这句话的意思是，客户端从来没有见过这台机器，无法确认对面是不是真的。输入 `yes` 之后，那把主机公钥会被记到 `~/.ssh/known_hosts`（一个专门记录服务器公钥的文件）里，下次再连就自动比对。这个机制叫首次使用即信任，它的作用、局限和风险在第七章讲。

登录成功后会看到一个提示符，和本地终端没什么区别。敲 `exit` 退出。如果只想执行一条命令，不必进交互界面。

```
ssh alice@198.51.100.17 "uptime"
```

这条命令会在远端执行 `uptime`，把输出送回本地，然后断开。批量巡检、临时查状态时常用这种写法。

两个最常用的选项也在这里交代一下。`-p` 指定端口，服务端不监听 22 时用。`-i` 指定私钥文件，一个账号有多把钥匙时用。

```
ssh -p 2222 -i ~/.ssh/id_ed25519_lab alice@198.51.100.17
```

想看到连接建立的过程，在命令后面加 `-v`，加两个 `v` 会更详细。输出里会依次出现连接、版本、算法、密钥、认证、通道这些词，现在还不必理解它们，只要知道它们排成一个固定的顺序。第六章会把每一段拆开讲，那时再回头读这份输出，会清楚得多。

到这里，日常使用需要的命令其实已经够了。剩下的是理解它为什么这样工作，以及出问题时该看哪里。

## 2.7 一次会话的轮廓

把一次连接按时间顺序摊开，会看到图 2-2 里的几个阶段。这张图先给出轮廓，细节在后面的章节逐一展开。




图 2-2 从建立连接到拿到 shell 的时间线

最先是 TCP 连接。SSH 跑在 TCP 之上，所以要先有一条可靠的字节流。TCP 是一种保证数据按顺序、不重不漏送达的协议，第三章会细讲。这一步和 SSH 无关，任何基于 TCP 的服务都要走，第三章讲。

接着双方各发一行版本字符串，形如 `SSH-2.0-OpenSSH_9.6`。这一行是明文，也是整条连接里唯一一定是明文的部分。

然后是算法协商。双方各自列出自己支持的密钥交换、加密、校验算法，从中挑出一套都能接受的组合。

再往后是密钥交换。双方通过数学运算协商出一份只有彼此知道的密钥材料，并由它派生出后面要用的所有密钥。

密钥就绪之后各自发送一个 NEWKEYS 消息，从这一刻起，后面所有内容都加密。第五章讲这套数学，第六章讲协议怎么组织这些步骤。

加密通道建立之后才开始认证。这个顺序常被误解，很多人以为先验证身份再加密，实际上是先有加密通道，再在通道里验证身份。好处是密码和签名（用私钥算出的一段数据，别人可以用对应的公钥验证它确实出自你手）都不会以明文出现在网络上。

认证成功后，客户端请求打开一个会话通道，服务端分配伪终端（一对虚拟设备，让远端程序以为自己接在一块真实终端上）并启动 shell，你才看到那个提示符。第四章讲服务端这一侧的进程与终端，第六章讲通道与流控。

## 2.8 全书的知识地图

图 2-3 把这条链路上涉及的东西按层次排了一遍。它是全书的结构图，也是后面十三章的路线。

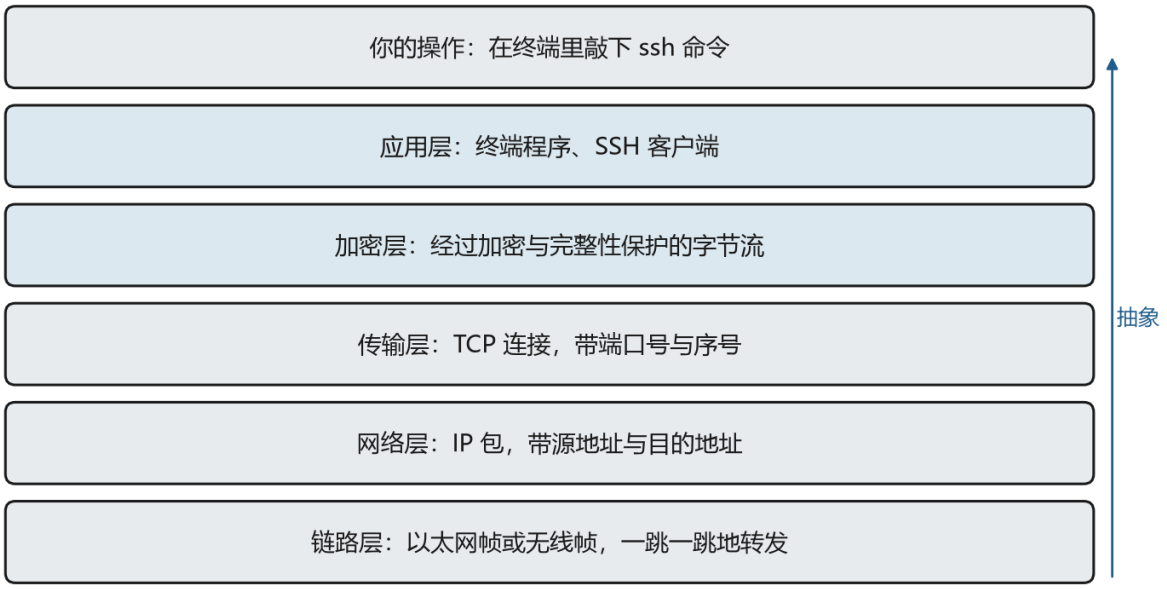



图 2-3 从人的操作到物理线路之间的层次

最上面是你的操作，接着是应用程序，再往下是加密层、传输层、网络层、链路层。越往下越靠近物理线路，越往上越接近人的意图。

这本书按四层展开。

第一篇讲链路，对应图里的传输层、网络层、链路层，以及服务器上的进程与终端。读完这一篇，你能把“连不上”拆成可以逐段验证的问题。

第二篇讲信任，对应加密层和 SSH 协议本身。密码学的基本工具、协议的三个子协议、密钥与指纹、端口转发，以及把整张内网搬过来的 VPN，都在这一篇。

第三篇讲控制，也就是当连接从一个人扩展到一群人时发生的事。堡垒机、JumpServer、VS Code 的远程开发，都是在这个层面上做文章。

第四篇讲数据，离开终端，去看代码运行时读写的那一端。对象存储的模型、签名、权限，以及把四层串起来的完整链路。

每一章都按同一个顺序写。先说清它要解决什么问题，再说它用什么办法解决，最后说这些办法在你的终端、配置文件、日志里长什么样。这样读下来，学到的不是几条命令，而是一张能自己往里填细节的地图。

第 3 章

# 地址、端口与那条看不见的路

## 3.1 一台机器在网上叫什么

第一章里的局域网用的是 `192.168.1.x` 这样的地址。这种写法叫 IPv4 地址，四组数字，每组 0 到 255，合起来 32 位，理论上可以提供约 43 亿个地址。上世纪八十年代设计这套体系的时候，没人认为 43 亿会不够用，可后来的事情大家都知道了，手机、电脑、摄像头、汽车都要联网，地址很快就不够分了。

解决办法有两个。一个是新版本的地址格式 IPv6，长 128 位，写成带冒号的一串十六进制，多到几乎用不完，但你日常接触的服务器大多仍然用 IPv4。另一个是复用，也就是让很多设备共用一个公网地址，私底下各自用一套只在内部有效的地址。这套内部地址就是私有地址，`192.168.x.x`、`10.x.x.x` 和 `172.16.x.x` 到 `172.31.x.x` 三段属于此列。

私有地址有一个重要性质，它在公网上不可路由。也就是说，你家里那台 `192.168.1.23` 的电脑，从外面的服务器是没法直接连过去的，因为全世界有几千万台机器都在用这个地址，路由器不知道该送到哪一家。这个性质看起来是个限制，实际上是一层保护。多数人的电脑不需要被外部主动连接。

而你租用的云服务器不一样。它有一个公网地址，任何人都能向它发起连接。这也是为什么云主机一开通就要配好访问规则，把不需要对外开放的服务关掉。

## 3.2 端口是同一台机器上的不同门

一台机器上可能同时跑着网页服务、数据库、SSH 服务。数据包到了这台机器，操作系统要知道该交给哪个程序。这个区分工作由端口号完成。端口是一个 16 位整数，范围 0 到 65535。IP 地址定位到机器，端口定位到机器上的程序，两者合起来叫套接字地址，写成 `203.0.113.24:22` 这种形式。

端口号被分成几段使用。0 到 1023 是知名端口，只有特权程序才能监听，比如 SSH 的 22、HTTP 的 80、HTTPS 的 443。1024 到 49151 是注册端口，各种服务按惯例占用，比如 MySQL（一种数据库）的 3306、Redis（一种内存里的键值存储）的 6379。再往上是临时端口，操作系统在发起连接时随机分配，用完之后回收。你家里的电脑连出去时用的就是临时端口，它被填在连接的源端口位置上，所以同一个网站会看到你每次连接用的源端口都不一样。

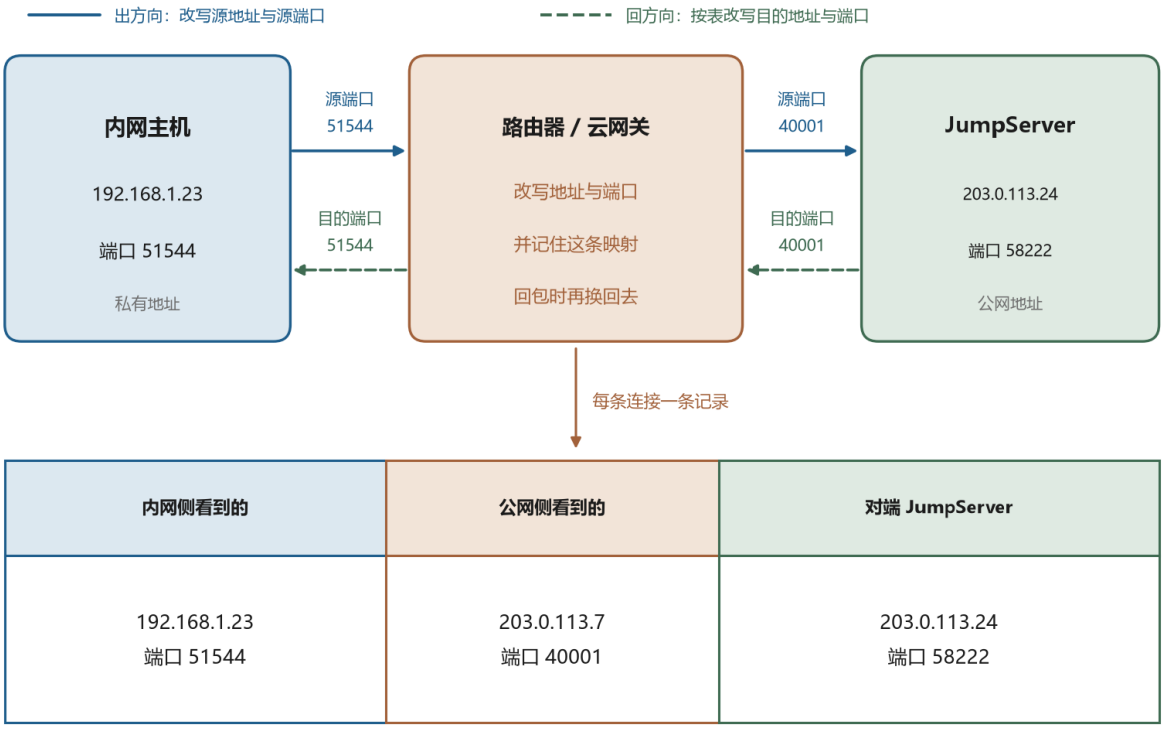
表里还会反复出现 HTTP 和 HTTPS。它们是浏览器访问网站时用的协议，后者在传输前会把内容加密。对象存储的接口也走 HTTPS，第十三章会用到这一点。

这里有一个必须分清的概念，监听和连接是两件事。服务器上的 sshd 在 22 端口监听，等待别人来连，22 是它那一端的端口，也是你要填在命令里的目标端口。你自己这一端同样需要

一个端口：操作系统会从临时端口里随机挑一个，比如 51544，当作这条连接的源端口。所以一条连接的两头各有一个端口号，一头固定、一头随机。四要素合起来唯一确定一条 TCP 连接：源地址、源端口、目的地址、目的端口。任何一项不同，就是另一条连接。

## 3.3 NAT 与地址改写

私有地址的设备要访问公网，需要中间有一台设备帮它做地址转换，这就是 NAT，网络地址转换。图 3-1 画的是这个过程的简化版本。




网关为这条连接分配了公网端口 40001：出方向改源、回方向改目的

对外只有 203.0.113.7 这一个地址，内网主机并不直接暴露

图 3-1 NAT 把内网地址改写成公网地址并记住对应关系

内网主机 `192.168.1.23` 用源端口 51544 访问 `203.0.113.24` 的 58222 端口。数据包经过网关时，网关把源地址换成自己的公网地址 `203.0.113.7`，同时把源端口换成一个它自己管理的端口，比如 40001，并在内存里记下一条映射。回包到达网关时，目的地址是 `203.0.113.7:40001`，网关查表把它改回 `192.168.1.23:51544`，再送进内网。

这个机制解释了两件常见的事。第一，从服务器端看到的客户端地址是网关的地址，不是你的真实地址。很多服务靠来源地址决定放不放行，比如“只允许 `203.0.113.7` 访问这个端口”，这类规则叫基于 IP 的访问控制。NAT 一来，这种规则就不好使了：一家公司几百人共用一个出口地址，服务器没法区分你和你的同事，放行就等于放行整个公司，拒绝就等于拒绝整个公司。规则本身没坏，坏的是粒度。第二，外部无法主动向内网发起连接，因为网关的表里没有对应的条目。要让外部连进来，必须在网关上手工配置端口映射，或者用第八章讲的隧道技术（把一条

连接套在另一条连接里），从内部主动把通道建立出去。

## 3.4 域名只是给人看的

IP 地址难记，于是有了域名。`jumpserver.example.com` 这样的名字人看得懂，机器看不懂，数据包发出去之前必须先把它换成 `203.0.113.24`。干这件事的系统叫 DNS（Domain Name System，域名系统），可以把它想成一本全球共用的电话簿：输入名字，查出号码。

查的过程并不是直接翻那本总簿。你的系统先看自己的缓存，最近查过的名字都记在这里；没有，就去问网络设置里填的那台 DNS 服务器，它通常由路由器或运营商提供。这台服务器手里也没有，就替你去问：先问根服务器 `.com` 归谁管，再问管 `.com` 的服务器 `example.com` 归谁管，最后问到管 `example.com` 的那台权威服务器，它给出 `jumpserver` 对应的地址。答案沿原路返回，沿途每一层都顺手留下一份。

缓存不能永久有效，否则域名换了地址，全世界还会按旧地址连过去。所以每条记录都带一个 TTL（Time To Live，有效期，单位是秒），比如 600，意思是“这个答案你可以信十分钟”。这就是为什么改完域名解析之后，有些地方立刻生效，有些地方要等一会儿——那些地方手里的旧答案还没过期。用 `nslookup` 或 `dig` 查一个域名，能看到它还剩多少秒。

对远程登录来说，域名带来一个额外的好处。你可以把同一个域名指向不同的机器，机器换了地址，配置文件不用改。云厂商的负载均衡和故障转移也依赖这一点。缺点是引入了新的失败点，DNS 解析不出来的时候，看起来就像网络不通。用 `ssh -vv` 看输出时，会有一行 `debug1: Connecting to ...`，那里显示的是解析之后的地址，如果域名有问题，这一步之前就会报 `Could not resolve hostname`。

## 3.5 TCP 保证了什么

IP 层只负责把包送到，不保证顺序，不保证不丢，也不保证不重复。SSH 需要的是一个可靠的字节流，也就是“我发出去的字节，按顺序、不重不漏地到达对端”。这个保证由 TCP 提供。

TCP 的做法是给每个字节编号。发送方发出数据时带上序号，接收方收到后回一个确认，告诉对方“我已经收到第 N 个字节之前的全部内容”。发送方如果在一段时间内没收到确认，就重发。接收方如果收到乱序的包，先缓存起来，等前面的缺口补上再交给应用。这套机制里还有流量控制和拥塞控制，前者防止发送太快把接收方撑爆，后者防止把网络压垮。

建立连接要三次握手，断开要四次挥手，如图 3-2。

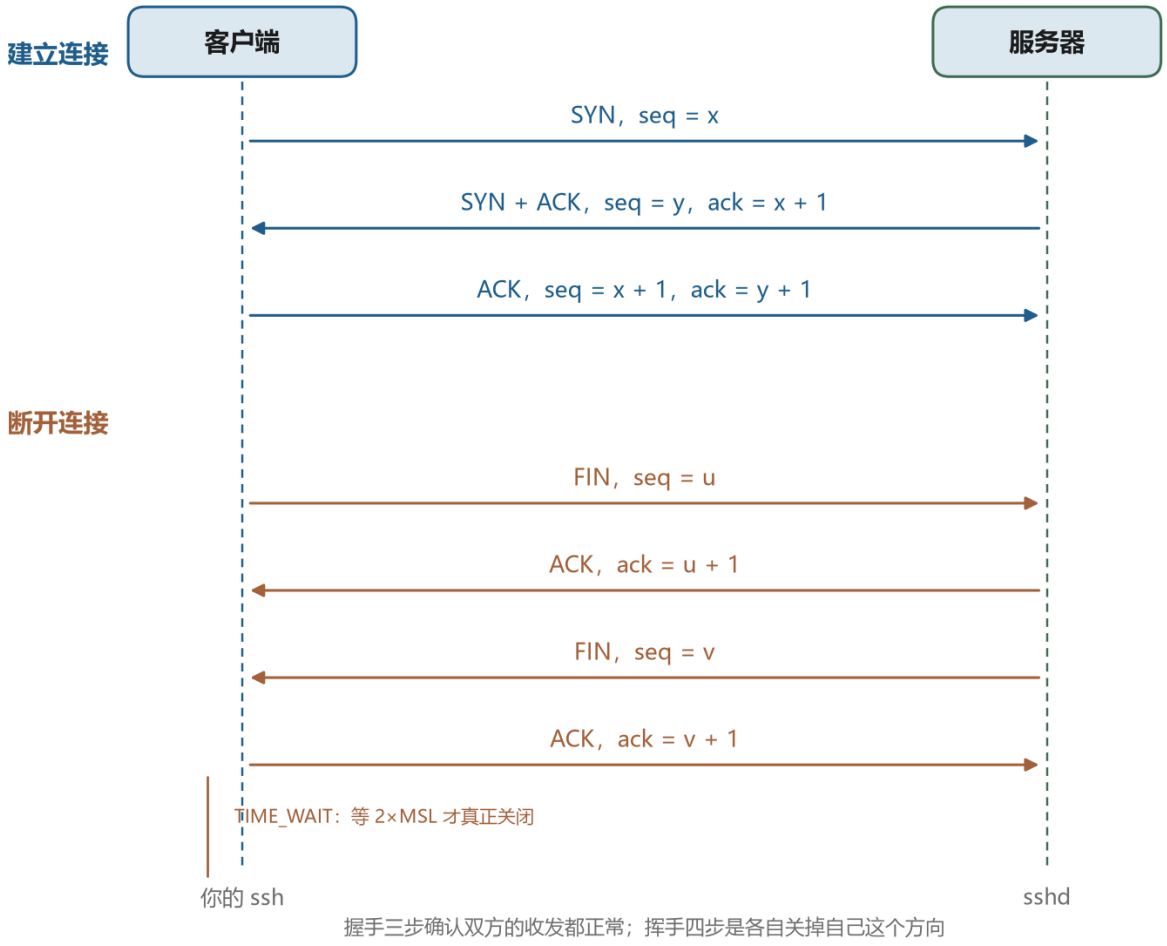



图 3-2 TCP 的连接建立与断开

三次握手的实质是双方各自确认两件事，我能发出去，你能收到。第一次客户端发 SYN（请求建立连接），第二次服务端回 SYN 加 ACK（同意，并确认收到），第三次客户端再回 ACK（确认收到对方的同意）。为什么不是两次，因为两次只能让客户端确认自己的发送能力，服务端无法确认客户端是否收到了自己的包。断开时的四次挥手是因为 TCP 连接是全双工的（两个方向可以同时传数据），两个方向要各自关闭。主动关闭的一方最后会进入 TIME_WAIT 状态，等一段时间确保最后的确认不会丢失。

这些细节和日常使用有什么关系。当你看到 `Connection timed out`，说明 SYN 发出去没有回应，可能是地址不可达，也可能被网络边界的过滤设备静默丢弃。看到 `Connection refused`，说明 SYN 到了，但对方回了一个拒绝包，通常是那个端口上没有程序在监听。两种错误的含义完全不同，处理方式也不同，前者要去查网络路径和边界上的过滤规则，后者要去查服务是否启动。

这些保证都不是白来的。下一节看另一种走向反面的协议，它什么也不保证，却撑起了半个互联网。

## 3.6 UDP 少做了什么

上一节列了 TCP 的四项保证。UDP（User Datagram Protocol，用户数据报协议）一项也不提供：不握手、不编号、不确认、不重传、不排序，也不管网络是否拥堵。它只做三件事，用端口号找到程序，算一个可选的校验和，把一段数据原样装进 IP 包发出去。

区别从头部就能看出来。TCP 的头部至少 20 字节，装着序号、确认号、窗口大小和一堆状态标志；UDP 的头部只有 8 字节，四个字段分别是源端口、目的端口、长度、校验和，没有地方再装别的。

没有连接状态，服务端就不必为每个客户端留一份上下文。一台机器扛几十万个短请求，对 TCP 是压力，对 UDP 只是收发。代价被转移到了应用层：包丢了没人替你重传，包乱序了没人替你排好，重复的包也没人替你丢掉。

“UDP 连接”这个说法其实不严谨。UDP 没有连接这个概念，只有一对地址加端口在收发数据。平时说 UDP 连接，指的是同一对端点在一段时间里持续通信，NAT 设备也把它当成一条记录存着，存的就是第三节里那种映射。

表 3-1 是两者的对照。

表 3-1 TCP 与 UDP 的对照

| 项目 | TCP | UDP |
| --- | --- | --- |
| 建立连接 | 三次握手，双方各自保存状态 | 不握手，无状态 |
| 可靠性 | 确认、重传、排序、去重 | 一概不管 |
| 头部开销 | 20 字节起 | 8 字节 |
| 拥塞控制 | 有，网络拥堵时自动降速 | 没有，发多快由应用决定 |
| 典型用途 | SSH、网页、文件传输 | DNS、音视频、游戏、VPN |

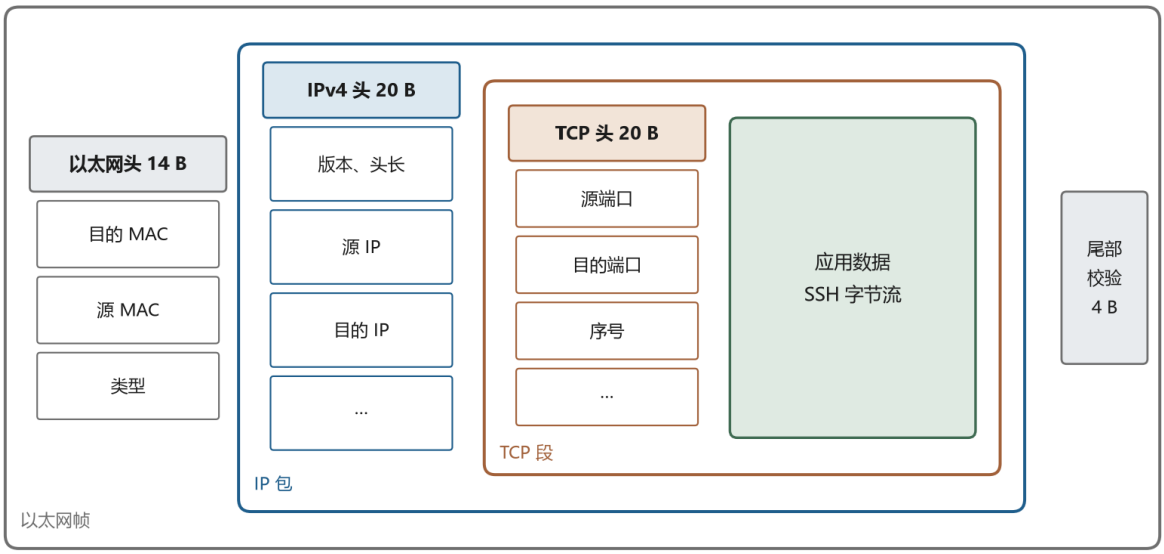
那为什么还要用 UDP。因为有些场景里，等一个迟到的包比丢掉它更糟。视频通话中，每一幅画面只显示几十毫秒，重传回来的那一幅已经没有意义，宁可让画面花一下；游戏里，位置信息晚到两百毫秒就彻底错了，不如直接丢；DNS 查询只有一个来回，重发一次比维护一条连接便宜得多。

第九章讲的 VPN 也用 UDP：隧道里跑的就是加密后的包，丢一个包只损失一点数据，不值得为它在中途等一次重传。HTTP/3 更进一步，它在 UDP 之上自己实现了一套可靠传输，名字叫 QUIC（读作 quick），原因是 TCP 那种“缺一个就全等”的重传方式会把后面的数据一起堵住，业内管这个叫队头阻塞。

判断标准可以压成一句话：要求每个字节都到、顺序不乱，用 TCP；要求快、能容忍丢，用 UDP，可靠性自己补。

## 3.7 数据是怎么被装进包里的

应用层的数据要一层层加上头部才能上路。图 3-3 画的是这个过程。




从外到内依次套住：以太网帧、IP 包、TCP 段、应用数据；每层只在前方加一个头部，帧尾再加 4 字节校验

图 3-3 从应用数据到以太网帧的四层封装

SSH 客户端产生的字节先交给 TCP，TCP 加上源端口、目的端口、序号、校验和等等，形成 TCP 段。TCP 段交给 IP，IP 加上源地址、目的地址、生存时间等，形成 IP 包。IP 包交给链路层，加上源 MAC、目的 MAC（MAC 是网卡的硬件地址，出厂时写死）和帧校验，形成以太网帧，最后变成线路上的电信号。

每一层加的头部都有固定长度。TCP 头部一般是 20 字节，选项会让它变长。IPv4 头部也是 20 字节。以太网帧的头部 14 字节，尾部校验 4 字节。帧是链路层传输的单元，可以理解为一段线路上跑的一整块数据。链路层用 MAC 地址标识网卡，MAC 地址是网卡出厂时写死的硬件地址，和 IP 地址不是一回事。

链路层能承载的单个帧有大小上限，这个上限叫 MTU，以太网通常是 1500 字节，减去各层头部，留给应用的有效载荷大约 1460 字节。如果某一层的数据超过了上限，就要拆成多片，这个过程叫分片，分片会让丢包的概率上升，也会让网络边界的过滤设备处理起来更费劲。

SSH 在建立连接时会自己控制包的大小，通常不会主动制造分片。但如果经过隧道或者 VPN，外层又套了一层头部，可用空间就变小了，可能出现“能连上但传大文件卡住”的怪现象。这种情况在第八章讲隧道时会再提一次。

## 3.8 自己动手看连接

理论讲完，需要一些工具把状态看清楚。Linux 上最常用的是 `ss`，老一些的系统用 `netstat`。

```
ss -lntp           # 列出所有 TCP 监听端口和对应进程
ss -tnp            # 列出当前已建立的 TCP 连接
ss -tnp state established '( dport = :22 )'   # 只看连到 22 端口的连接
```

`-l` 表示只看监听，`-n` 表示不解析服务名，`-t` 表示 TCP，`-p` 显示进程。第一行输出能告诉你这台机器上有哪些服务在等别人连接，第二行输出能告诉你此刻谁连着谁。排障时这两个视角缺一不可。

Windows 上的对应命令是 `netstat -ano`，加上 `| findstr :22` 过滤。想知道某个端口通不通，可以用 `Test-NetConnection`，Linux 上可以用 `nc -vz host port` 或者 `curl -v telnet://host:port`。

```
Test-NetConnection 203.0.113.24 -Port 58222
```

还有一个常被忽略的工具是 `traceroute`（Windows 上叫 `tracert`）。它逐跳显示数据包经过的路由器，能看出是在哪一段断掉的。如果是公司网络内部就不通，那和服务器没关系；如果出了公网才断，就要找运营商，或者检查云平台上那组放行规则。

## 3.9 常见端口一览

表 3-2 列出这本书里会反复出现的端口。记住其中几个，读配置文件时会轻松很多。

表 3-2 书中涉及的常见端口

| 端口 | 服务 | 说明 |
|---|---|---|
| 22 | SSH | OpenSSH 默认端口，公网上被扫描得最频繁的端口之一 |
| 2222 | SSH 备用端口 | 常用来避开针对 22 的暴力破解 |
| 58222 | JumpServer 网关 | 部署时自定义的端口，本书示例中使用 |
| 80 | HTTP | 明文网页，多数站点会重定向到 443 |
| 443 | HTTPS | 加密的网页传输，对象存储的接口也走这里 |
| 3306 | MySQL | 数据库，绝不应该直接暴露在公网 |
| 6379 | Redis | 缓存，默认无认证，暴露在公网等于把数据送人 |
| 1080 | SOCKS | 第八章动态转发时本地监听的端口 |

这张表里第 22 和第 58222 两行值得多看两眼。它们说明了一件事，端口号本身没有任何

魔力，它只是双方约定好的一个数字。把 SSH 挪到别的端口，安全性并不会因此提高多少，因为端口扫描很容易发现它，但能挡掉大量无差别攻击的噪音，让日志干净一些。真正决定安全的是密钥长度、认证方式和访问控制，这些是第二篇的主题。

第 4 章

# 服务器上发生了什么

## 4.1 登录成功那一刻

认证通过之后，sshd 并不会把当前的进程直接交给你。它做三件事：创建一个伪终端设备，把你的输入输出接到这个设备上；fork（复制出一个新进程）出一个子进程，在子进程里切换成你的用户身份、加载你的登录环境；最后在子进程里执行你的登录 shell，也就是写在 `/etc/passwd` 最后一段的那个程序，比如 `/bin/bash`。

所以你看到提示符之前，机器上已经排好了一条链：sshd 在 22 端口监听，收到连接后 fork 出一个子进程，子进程认证通过后再 fork 出 shell，shell 每收到一条命令再 fork 一次。

这条链上用到四个零件：shell、进程、文件描述符、终端。接下来四节逐个讲，第六节再把它们拼成图 4-1，那时你就能一眼看懂这张图。

## 4.2 shell 是什么

上一节的最后一步是执行登录 shell。这个词在书里出现过好几次，值得停下来讲清楚。

操作系统大致分两层。靠近硬件的那一层叫内核（kernel），进程调度、内存、文件系统、网络这些真正干活的活都在里面。内核听不懂人话，它只认系统调用（程序请求内核替它做事的固定接口）。夹在人和内核之间、负责把人话翻译成系统调用的那个程序，就是 shell，中文叫“外壳”。名字的意思很直白：壳包在核外面，你隔着它跟核打交道。

shell 是个普通程序，循环做四件事：打印提示符，读你敲的一行，把这一行解释成一个动作，执行，再把结果送回来。所谓解释，具体是几件活：

- 拆词和处理引号。`cp "a b.txt" c.txt` 里第一个参数是一个词，不是两个
- 展开。`*.log` 变成一串文件名，`$HOME` 换成变量的值
- 连接。`|` 把前一条命令的输出接给后一条，`>` 把输出写进文件，`&&` 决定后一条跑不跑
- 执行。一部分命令由 shell 自己完成，比如`cd`、`export`、`exit`，这类叫内建命令；其余的交给系统去找对应的程序，起一个进程来跑

最后一条解释了 `cd` 为什么特别。它必须由 shell 自己执行，因为“当前目录”是 shell 自己的状态，交给子进程去改，改的是子进程，回头 shell 还站在原地。

同一个 shell 程序在不同场合有不同叫法，这是最容易糊涂的地方。登录时启动的那个叫登录 shell，它会去读 `/etc/profile`、`~/.bash_profile` 这类文件。你在提示符下敲命令的那个叫交互式 shell，有提示符、有行编辑、有 Ctrl+C。`ssh host "uptime"` 这种一次性命令用的是

非交互 shell，它不读那些只给登录准备的文件，所以同一个变量在两种场合可能表现不一样。把一整套命令写进文件批量执行，就是 shell 脚本，还是同一个程序，只是输入来自文件而不是键盘。

还有两个常见的混淆。第一，shell 不是终端。终端是设备或界面，真实的串口终端、伪终端、你眼前的终端窗口都算；shell 是跑在终端里的解释器，同一个终端里可以把它换掉，比如敲一句 `exec zsh`。第二，shell 不止一个。`sh`、`bash`、`zsh`、`fish`、`dash` 都是 shell 的不同实现，语法大体相通，细节有差别。你用哪一个，写在 `/etc/passwd` 的最后一段，登录时 sshd 照着启动。

顺带说一句，SSH 的全称 Secure Shell 里的 shell 指的就是这个东西。它最早只是一个“安全的远程 shell”，后来才长成能传文件、能转发端口的协议族。

## 4.3 进程与 fork

链上的第二次 fork 值得停下来看，因为“敲一条命令之后到底发生了什么”，答案就在这里。

程序是磁盘上的一个文件，进程是它跑起来之后的样子。一个进程有自己的编号（PID）、自己的内存、自己的身份（哪个用户在跑）、自己打开着的文件。同一个程序跑两份，就是两个进程，内存互不相通。

Linux 起一个新进程的常规做法是两步。第一步叫 fork，把当前进程整个复制一份，得到的子进程和父进程几乎一模一样，区别在返回值：父进程拿到子进程的编号，子进程拿到 0。第二步叫 exec，把当前进程的内存换成另一个程序，进程号不变，跑的却已经是另一个程序了。

新复制出来的进程没有第二段可执行代码，所以 fork 出来的子进程立刻 exec，换成你要跑的那个程序。两步合起来，就是 shell 执行一条外部命令的全过程：fork 一个自己，子进程 exec 成 `ls`，父进程用 wait 等它结束、取回退出码，然后重新显示提示符。

因此一条命令一个进程，父子关系层层嵌套。用 `ps -ef` 或 `pstree -p` 能看到这棵树：`sshd` 下面挂着一个 `sshd`，再下面是 `bash`，再下面是你正在跑的 `vim` 或 `python`。关掉一个终端窗口会连带杀掉一串进程，原因就是它们在这棵树上互为父子。

内建命令是例外。`cd`、`export`、`exit` 改的是 shell 自己的状态，如果 fork 出去改，改的是子进程，父进程原地不动。所以这几个命令不 fork，由 shell 自己执行。

## 4.4 三个标准文件描述符

进程读写文件、设备、管道，靠的不是文件名，而是一个编号，叫文件描述符（file descriptor，常缩写为 fd）。打开一次文件，系统分一个编号；关掉，编号回收。

有三个编号是约定好的，任何进程一启动就有：0 是标准输入，1 是标准输出，2 是标准错误。默认情况下，0 接键盘，1 和 2 接屏幕。所谓重定向，就是 shell 在 fork 之后、exec 之前，把这三个编号里的某一个换成别的文件。

```
ls -l > /tmp/list.txt       # 1 指向文件，标准输出进了文件
ls -l 2> /tmp/err.txt       # 2 指向文件，错误信息单独收走
make 2>&1 | tee build.log   # 2 指向 1 当前指向的地方，也就是那根管道
```

`2>&1` 写在 `>` 前面是不行的，因为它复制的是“1 此刻指向哪里”，顺序错了就复制了一个旧值。

还有一件事要记住：这三个编号后面接的可以是一个文件、一根管道，也可以是一台终端。接的到底是什么，程序自己会去问，问完再决定怎么输出。这正是下一节的主题。

## 4.5 终端与伪终端

早期的计算机，终端是一台真实的设备，通过串口线连到主机。操作系统里有一套终端驱动负责和它打交道。后来终端变成了软件，为了让那些只会和终端打交道的程序继续工作，系统提供了伪终端，用一对虚拟设备模拟串口的两端。一端叫 master，由 sshd 持有，另一端叫 slave，交给 shell。

这两端像一根管子的两个口。你在客户端按下的键，由 sshd 写进 master，shell 从 slave 那头读到；shell 和它启动的程序往 slave 写的东西，sshd 从 master 读完，加密后发回你的屏幕。方向是两个方向，动作却只有两个：写进去，读出来。

启动 shell 之前，sshd 的子进程已经把 slave 设成新进程的 0、1、2 号描述符。所以 shell 和它启动的每个程序，打开的都是同一台“终端”，只不过它不知道这台终端是虚拟的。

这件事带来三个具体的能力。

第一是行编辑。按下回车之前，退格能删字、方向键能移动光标、Ctrl+R 能搜历史。这一层由内核里的行规程（终端驱动中负责缓冲和解释输入的那部分）和 shell 自己的行编辑库共同完成，前提都是“输入来自终端”。输入若是管道，就没有这一层，读到什么就是什么。

第二是颜色和对齐。程序启动时会先问一句“我写出去的 1 号描述符是不是终端”，这个检查叫 `isatty`。回答是，它才输出颜色转义序列（`\033[31m` 这类控制字符），才按列对齐；回答不是，它就退化成一列纯文本，免得把控制字符写进文件或管道。

第三是信号。Ctrl+C 并不是把 0x03 这个字节交给程序，而是由行规程解释成中断信号，发给正在前台运行的那组进程。Ctrl+Z 是挂起，Ctrl+\ 是退出。程序自己不用解析按键，它只管收到信号然后决定怎么收拾。

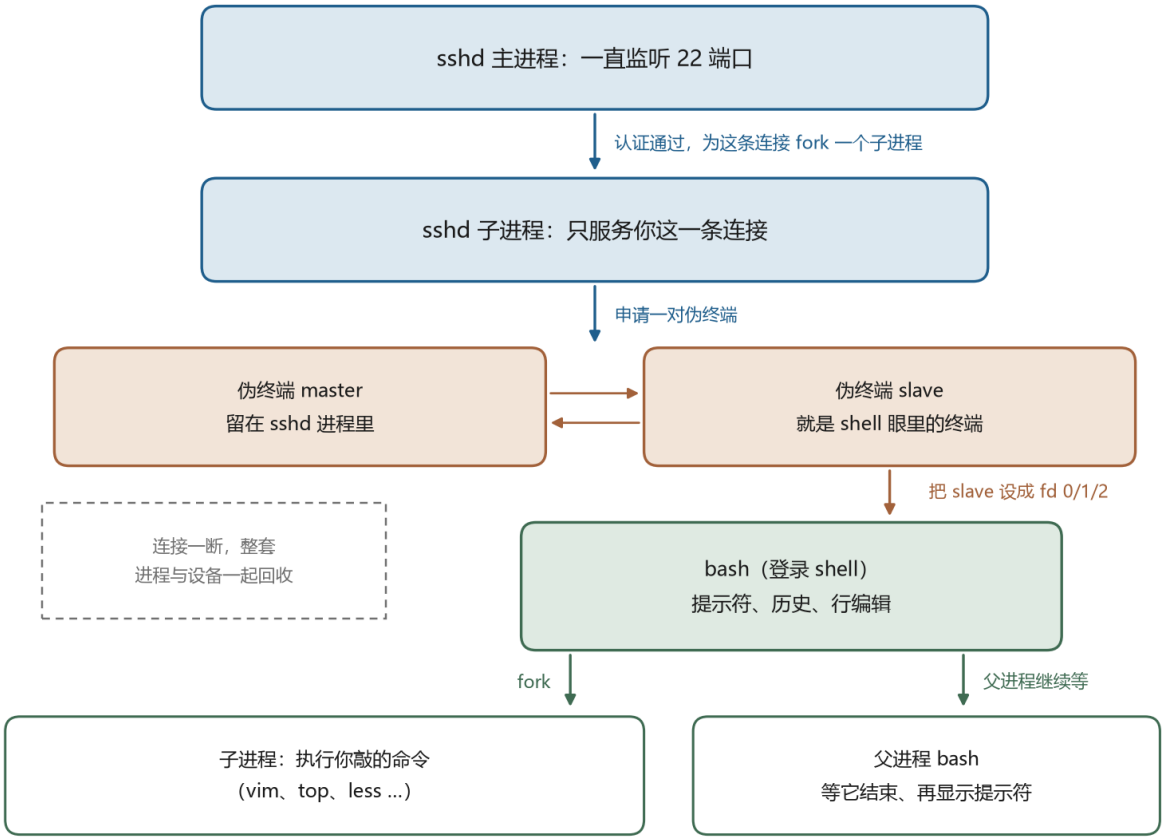
窗口大小也走同一套机制。你在客户端拖动窗口，客户端把新的行列数告诉 sshd，sshd 设置伪终端的尺寸，内核再给前台进程发一个“窗口变了”的信号（SIGWINCH）。`top`、`vim` 收到后重画，远端的布局跟着变。整个链路里没有任何一块屏幕，服务器那头只有这一对设备。

反过来就好理解 `ssh host "ls -l"` 为什么没有颜色了。这条命令不分配伪终端，`ls` 的 1 号描述符接的是一根管道，`isatty` 回答“不是终端”，于是它关掉颜色、不做列对齐。准确地说，不是“输出被重定向到了文件”，而是“输出不是终端”——在程序眼里，这两件事是一回事。

伪终端的分配可以控制。客户端用 `-t` 强制分配，`-T` 禁止分配，配置文件里用 `RequestTTY`。什么时候需要关心它，通常是脚本里跑交互式程序的时候。比如用 `ssh` 执行 `sudo`（临时以管理员身份执行一条命令的工具），如果不分配伪终端，`sudo` 会因为拿不到终端而拒绝读取密码。

## 4.6 把这条链拼起来

四个零件都认识了，现在把图 4-1 完整看一遍。




你看到的“终端”其实是内核里的一对设备：写 master，slave 那头就能读到

标准输入、标准输出、标准错误（fd 0/1/2）全部指向 slave

图 4-1 一次登录在服务器上生成的进程与伪终端

从上往下读。最上面是 sshd 主进程，它在 22 端口监听，永不退出。你连上来之后，它 fork 出一个子进程，这个子进程只服务你这一条连接，认证、分配资源都在它手里。

接着它向内核申请一对伪终端设备。master 留在 sshd 手里，slave 交给即将启动的 shell；启动 shell 之前，子进程把 slave 设成新进程的 0、1、2 号描述符（上一节讲过这一

步）。从这一刻起，shell 的输入输出都通向 slave 那一端，而它自己并不知道对面是 sshd，而不是一块真实的屏幕。

再往下是 bash。它打印提示符，等你敲命令。你敲一条外部命令，它 fork 一个子进程去 exec，自己在原地 wait，等你按下的这条命令结束再显示下一个提示符。整棵树的形状就是图里那几层。

一个字节的旅程可以照着图走一遍。你按下回车，客户端把这一行加密送到 sshd；sshd 写进 master；内核把它从 slave 那头交出来，bash 从 0 号描述符读到；bash 判断这是外部命令，fork 加 exec；子进程把结果写到 1 号描述符，也就是 slave；sshd 从 master 读到，加密发回你的屏幕。

最后是收尾。你关掉窗口或者网络断开，sshd 关掉 master，内核在 slave 上产生一个挂断信号（SIGHUP），bash 收到后退出；退出前它会给自己启动的后台任务也发一个挂断信号。想让后台任务活下来，有两种办法：用 `nohup` 让它忽略这个信号，或用 `setsid` 把它挪到一个新会话里，与这个终端脱钩。

## 4.7 用户、组与权限

Linux 用数字标识用户，这个数字叫 UID。用户名只是给人看的别名，系统内部认的是数字。每个用户至少属于一个组，组也有数字标识，叫 GID。`/etc/passwd` 记录用户名、UID、GID、家目录和登录 shell，`/etc/group` 记录组的成员。

```
id                    # 我是谁，属于哪些组
whoami                # 只输出用户名
getent passwd alice   # 查某个用户的记录
```

文件权限用九个二进制位表示，分成三组，分别是属主、属组和其他人的读、写、执行权限。`ls -l` 输出的第一列就是这九个位，前面还有一个字符表示文件类型，`-` 是普通文件，`d` 是目录，`l` 是符号链接。

```
-rw-r--r--  1 alice dev  1204 Nov  4 10:12 notes.txt
drwxr-x---  2 alice dev  4096 Nov  4 10:12 project
```

第一行表示属主可读写，属组可读，其他人可读。换成八进制是 644。第二行表示属主可读写执行，属组可读执行，其他人什么都没有，八进制是 750。目录上的执行位含义不同，它表示“能不能进入这个目录”。少了这一位，哪怕目录里的文件本身是 755，你也打不开它们：文件权限管的是“这个文件让不让读”，目录的执行位管的是“你能不能走到文件跟前”。

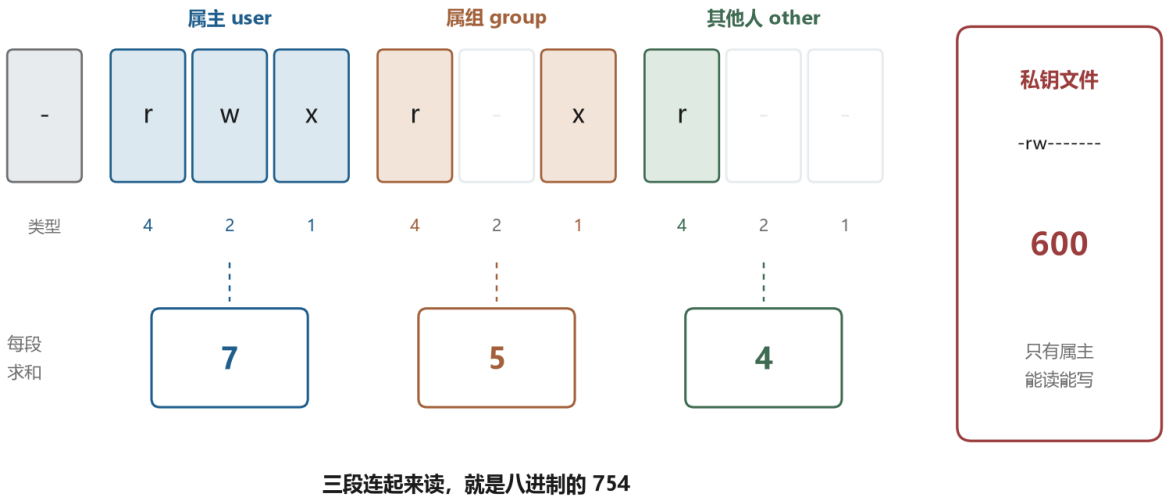



图 4-2 九个权限位与八进制的对应关系

这套权限体系和 SSH 有一个硬性交集，私钥文件的权限必须是 600。第七章会讲原因，这里先说结论，SSH 客户端在读取私钥之前会检查文件权限，如果发现同组或其他用户也能读，它会直接拒绝使用这把钥匙并给出警告。这条检查看起来吹毛求疵，实际很有必要，因为私钥一旦泄露，等于把身份送人。

```
chmod 600 ~/.ssh/id_ed25519
chmod 700 ~/.ssh
```

目录 `.ssh` 用 700，只有自己能进。公钥文件 `id_ed25519.pub` 可以 644，因为公钥本来就是给别人看的。`authorized_keys` 建议 600，虽然 sshd 对这个文件的要求因配置而异，但收紧一点没有坏处。

## 4.8 该记住的几个路径

服务器上的文件散落在各处，但和 SSH 相关的只有几个固定位置。表 4-1 是这本书里会反复用到的。

表 4-1 与 SSH 相关的常用路径

| 路径 | 所在机器 | 作用 |
|---|---|---|
| `~/.ssh/config` | 客户端 | 主机别名与连接参数 |
| `~/.ssh/id_ed25519` | 客户端 | 私钥，权限 600 |
| `~/.ssh/id_ed25519.pub` | 客户端 | 公钥，可以公开 |
| `~/.ssh/known_hosts` | 客户端 | 记下见过的服务器主机密钥 |
| `~/.ssh/authorized_keys` | 服务端 | 允许登录的公钥清单 |

| 路径 | 所在机器 | 作用 |
| --- | --- | --- |
| /etc/ssh/sshd_config | 服务端 | sshd 的全局配置 |
| /etc/ssh/ssh_config | 客户端 | 全系统的客户端默认配置 |
| /var/log/auth.log | 服务端 | Debian 系的认证日志 |
| /var/log/secure | 服务端 | Red Hat 系的认证日志 |

注意 ssh_config 和 sshd_config 只差一个字母，含义完全不同。前者是客户端读的，后者是服务端读的。搜索配置项时一定要看清楚，把服务端的选项写进客户端的文件里，通常不会报错，只是静默失效，这种问题最难查。

## 4.9 环境变量与 PATH

环境变量是一组名值对，跟着进程走。父进程设置好，fork 出来的子进程默认继承一份，所以登录时设好的变量，后面敲的每条命令都能看到。

这里面最重要的一个是 PATH，一串用冒号分隔的目录。你敲 python3，shell 并不全盘搜索，而是按 PATH 里的顺序逐个目录去找同名文件，找到第一个就执行。图 4-3 画的是这个过程。

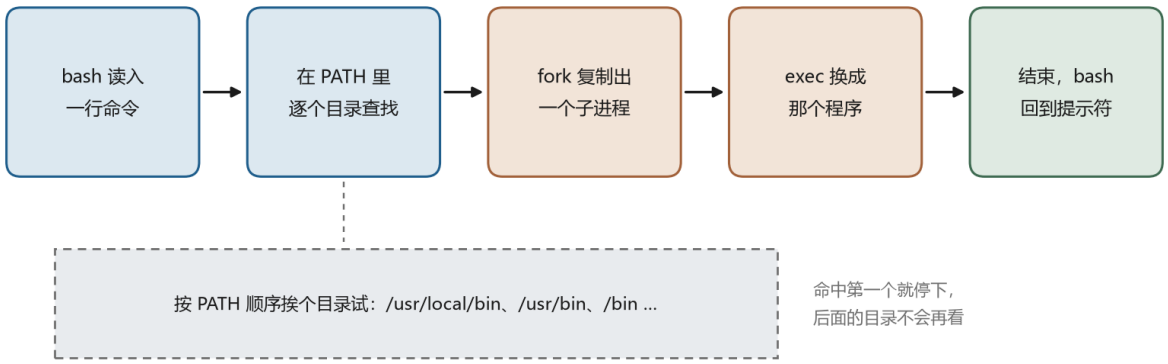



图 4-3 shell 查找命令、创建进程、回收子进程的过程

```
echo $PATH
which python3
export PATH="$HOME/.local/bin:$PATH"
```

export PATH="$HOME/.local/bin:$PATH" 的意思是，把 ~/.local/bin 加到现有 PATH 的最前面。放在前面，是让新目录优先——自己的程序覆盖系统同名命令时，顺序就是这个意思。

这些变量由 shell 在启动时从一个或几个文件里读出来。麻烦在于“读哪几个文件”取决于 shell 是怎么启动的，4.2 讲的四种叫法在这里落到了实处。表 4-2 是 bash 的规则。

表 4-2 bash 在不同启动方式下读的文件

| 怎么启动的 | 登录 shell | 交互式 | 读哪些文件 |
| --- | --- | --- | --- |
| `ssh host` 登录 | 是 | 是 | `/etc/profile`，然后 `~/.bash_profile`（没有就试 `~/.bash_login`、`~/.profile`） |
| 登录后敲 `bash` | 否 | 是 | `~/.bashrc` |
| `ssh host "cmd"` | 否 | 否 | 都不读，只看环境变量 `BASH_ENV` 指的文件 |
| 执行一个脚本 | 否 | 否 | 同上 |

所以你在 `~/.bashrc` 里加的 `export PATH=...`，用 `ssh host` 登录后可能没生效：那次启动的是登录 shell，它根本不读 `~/.bashrc`。

那为什么很多人这样写也没事？因为多数发行版在 `~/.bash_profile` 里放了一句话：

```
source ~/.bashrc
```

`source` 的作用是让当前 shell 直接执行另一个文件里的命令，而不是另起一个进程去跑。所以这句相当于“登录时顺手把 .bashrc 也读一遍”，写在里面的变量就留在了当前 shell。但这句话不是 bash 的规则，只是发行版的习惯：有的系统用 `~/.profile` 代替 `~/.bash_profile`，有的容器镜像里干脆没有这些文件。这就是为什么同一个写法在 A 机器上生效、B 机器上不生效。

遇到“命令找不到”，按顺序问三句：

1. `shopt -q login_shell && echo login || echo non-login` —— 我现在是不是登录 shell
2. `echo $PATH` —— 实际生效的 PATH 是什么
3. `grep -n bashrc ~/.bash_profile ~/.profile 2>/dev/null` —— 登录 shell 有没有把 `.bashrc` 拉进来

想少踩这个坑，有两个稳妥做法。一是把需要全局生效的 `export` 写进 `/etc/profile.d/` 下自己的 `.sh` 文件，它对所有用户的登录 shell 都生效；二是如果只影响自己，就把 `export` 直接写进 `~/.profile` 或 `~/.bash_profile`，别放在 `.bashrc` 里再指望它被读到。

另一个坑和 sudo 有关。`sudo` 默认会重置环境变量，所以你在自己账户下能用的 `PATH`，切到 root 之后可能就变了。要用 `sudo -E` 保留环境，或者在 `sudoers` 里配置 `secure_path`。

## 4.10 常驻的服务

sshd 是一个常驻后台的进程，由 systemd（Linux 上负责启动和看管后台服务的程序）管理。开机时 systemd 读 `/etc/systemd/system/` 和 `/lib/systemd/system/` 下的单元文件，按依赖关系启动服务。sshd 的单元文件叫 `ssh.service` 或者 `sshd.service`，取决于发行版。

```
systemctl status ssh
systemctl restart ssh
journalctl -u ssh -n 50 --no-pager
```

这三条命令分别用于查看状态、重启服务和看最近的日志。改完 `sshd_config` 之后一定要重启或者 `systemctl reload ssh`，否则新配置不生效。重启前建议先保留一个已经登录的会话，因为配置写错可能导致再也连不上，那时候只能通过云厂商的救援控制台进去改。

systemd 还有一个能力值得知道，叫 socket 激活。有些发行版把 sshd 配置成由 systemd 监听 22 端口，收到连接才启动 sshd 进程。这种配置下，用 `ss -lntp` 看到的监听进程是 `systemd` 而不是 `sshd`，第一次见会以为装错了。判断方法很简单，看 `systemctl status ssh.socket` 是否存在。

## 4.11 日志里能看到什么

认证相关的日志包含的信息比多数人以为的多。一次成功的公钥登录会留下这样几行。

```
sshd[2417]: Accepted publickey for alice from 203.0.113.7 port 51544 ssh2: ED25519 SHA25\
    6:xxxx
sshd[2417]: pam_unix(sshd:session): session opened for user alice by (uid=0)
sshd[2417]: Received disconnect from 203.0.113.7 port 51544:11: disconnected by user
```

第一行告诉你用的哪种认证方式、从哪个地址来、用的哪把钥匙的指纹。第二行说明会话已经建立。第三行是断开。失败时格式类似，只是把 `Accepted` 换成 `Failed`，后面会跟上原因，比如 `invalid user`、`authentication failure`、`Connection closed by authenticating user`。

如果日志里出现大量 `Failed password for invalid user admin`，说明有人在扫描。这类噪音在公网上是常态，处理方法在后面的章节会讲。真正需要警惕的是针对你自己账户名的连续失败，那说明有人在有针对性地尝试。

到这里，服务器这一侧的基础就差不多了。你现在应该能回答几个问题。登录之后是谁在跟你说话，你的命令是怎么变成一个进程的，为什么私钥的权限必须是 600，日志里的那些字段各是什么意思。下一篇要进入这本书最硬的部分，也就是把这些连接加密起来的那套数学。

第 二 篇

# 信任是怎么建立的

连接建起来之后，问题才真正开始。

你连过去的那台机器，凭什么相信你是你。你凭什么相信它不是别人伪装的。中间经过的那些设备，凭什么看不到你在传什么。这三个问题不解决，远程登录只是把风险从线下搬到了线上。

这一篇讲的是答案。答案由数学给出，具体说是密码学。但这一篇不打算从公式开始，而是从需求开始。先说清楚要防什么，再说用什么工具防，最后看 SSH 怎么把这些工具组装成一套协议。中间会用到一些数学，都控制在理解原理所必需的范围，不需要你会推导。

信任建立之后，这一篇还要回答一个更实际的问题：怎么把远方的资源搬到手边。答案从一条通道开始，一条通道能开一个端口；把这条通道放大，就是 VPN，它搬的不是一个端口，而是一整张内网。

读完之后，你应该能看懂 `ssh -Q kex` 列出的那些名字，知道每个名字代表什么取舍，也能判断一份配置里的算法选择是否合理。

第 5 章

# 加密、哈希与密钥交换

## 5.1 三种敌人

远程登录要防的不是抽象的“黑客”，而是三种具体的攻击。

第一种是窃听。数据包经过的每一台路由器、每一个交换机、每一个运营商的接入设备，理论上都能把经过的字节抄一份。早年间 telnet 和 FTP 的用户名密码就是这样泄露的，因为它们是明文。第二种是篡改。攻击者不一定需要读懂内容，只要能在传输过程中改动几个字节，就可能让远端执行另一条命令。第三种是冒充。你连过去的地址可能是攻击者伪造的，或者攻击者在中间同时骗过两端，这叫中间人攻击。

三种攻击对应三种技术。加密解决窃听，完整性校验解决篡改，身份认证解决冒充。SSH 把这三件事都做了，而且做得很彻底。理解它的设计，需要先理解它用的三样工具。

## 5.2 一把钥匙的对称加密

对称加密是最古老也最快的加密方式。发送方和接收方用同一把密钥，用同一个算法把明文变成密文，再用同一个算法反过来算回去。图 5-1 是这个过程的示意。

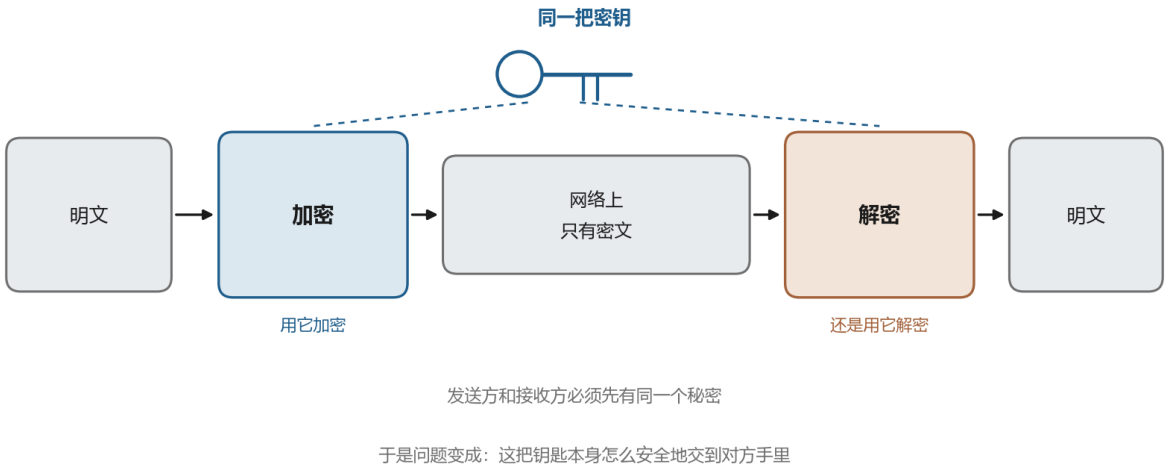



图 5-1 对称加密，加密与解密用同一把钥匙

今天的对称加密算法主要有两类。一类是分组密码，把数据切成固定大小的块逐块处理，AES 属于这一类，块长 128 位，密钥长度可以是 128、192 或 256 位。另一类是流密码，逐字节生成密钥流再和明文异或，ChaCha20 属于这一类。AES 在有大面积硬件加速的服务器上速度极快，ChaCha20 在没有 AES 硬件指令的手机和嵌入式设备上表现更好。SSH 两者都支持，连接时会协商使用哪个。

分组密码有个问题，如果对每个块都独立加密，相同的明文块会产生相同的密文块，攻击者

能从重复模式里推断内容。所以实际使用时要配合工作模式。CTR 模式把一个计数器加密后作为密钥流，等价于把分组密码变成流密码。GCM 模式在 CTR 的基础上加了认证，一次运算同时完成加密和完整性校验。

对称加密的优点是快，缺点是密钥分发。双方必须事先共享同一把钥匙，而在一个几万台机器的机房里，让每两台机器都共享一把不同的钥匙，管理成本是平方级的。这个问题由非对称加密解决。

## 5.3 哈希与消息认证码

哈希函数把任意长度的输入压成固定长度的输出，而且满足几个苛刻的性质。给定输入算输出很容易，给定输出反推输入几乎不可能。输入改一个比特，输出会面目全非，这叫雪崩效应。找到两个不同输入产生同一输出，在计算上也做不到。

SHA-256 是最常用的哈希函数，输出 256 位，写成 64 个十六进制字符。你在下载软件时看到的校验值，在证书里看到的指纹，都是这种形式。哈希不是加密，因为它不可逆，没有“解密”这一步。它的用途是比对，只要两段数据的哈希相同，就可以认为数据相同。

单独用哈希还不够。如果攻击者能同时改数据和哈希值，比对就失去意义。所以需要一把密钥参与，这就是消息认证码（Message Authentication Code，缩写 MAC）。HMAC 是最常见的构造方式，把密钥和数据一起送进哈希函数，只有知道密钥的人才能算出正确的值。SSH 用它来保证每个数据包没有被改动。

```
HMAC-SHA256(key, message) = SHA256((key ^ opad) || SHA256((key ^ ipad) || message))
```

这行公式不必记住，但要理解它的效果。没有密钥的人算不出正确的 HMAC，也就无法伪造一个能通过校验的包。SSH 的每个数据包后面都跟着一段 MAC，接收方先验 MAC 再解密，验不过就直接断开。这种“先认证再解密”的顺序可以避免把恶意构造的密文送进解密器。

这里要提防一个撞名。第三章讲封装时说的 MAC 是 MAC 地址，网卡的硬件编号；这里的 MAC 是消息认证码，用密钥算出来的校验值。两个东西毫无关系，只是英文缩写恰好相同，读的时候看清上下文。

## 5.4 非对称加密的两个用途

非对称加密用一对密钥，公钥和私钥。密钥是统称，凡是可以拿来做加密、解密、签名的那段数据都叫密钥；一对里的两把才分公私。公钥可以公开，私钥必须自己保管。这一对密钥在数学上相关，但从公钥算不出私钥。它的用途有两个，方向刚好相反。

第一个用途是加密。用对方的公钥加密，只有对方的私钥能解开。这适合传递秘密，因为不需要事先共享密钥。第二个用途是签名。用自己的私钥对数据算一个签名，任何人用你的公钥都能验证这个签名确实出自你手，而且数据没被改过。这适合证明身份。

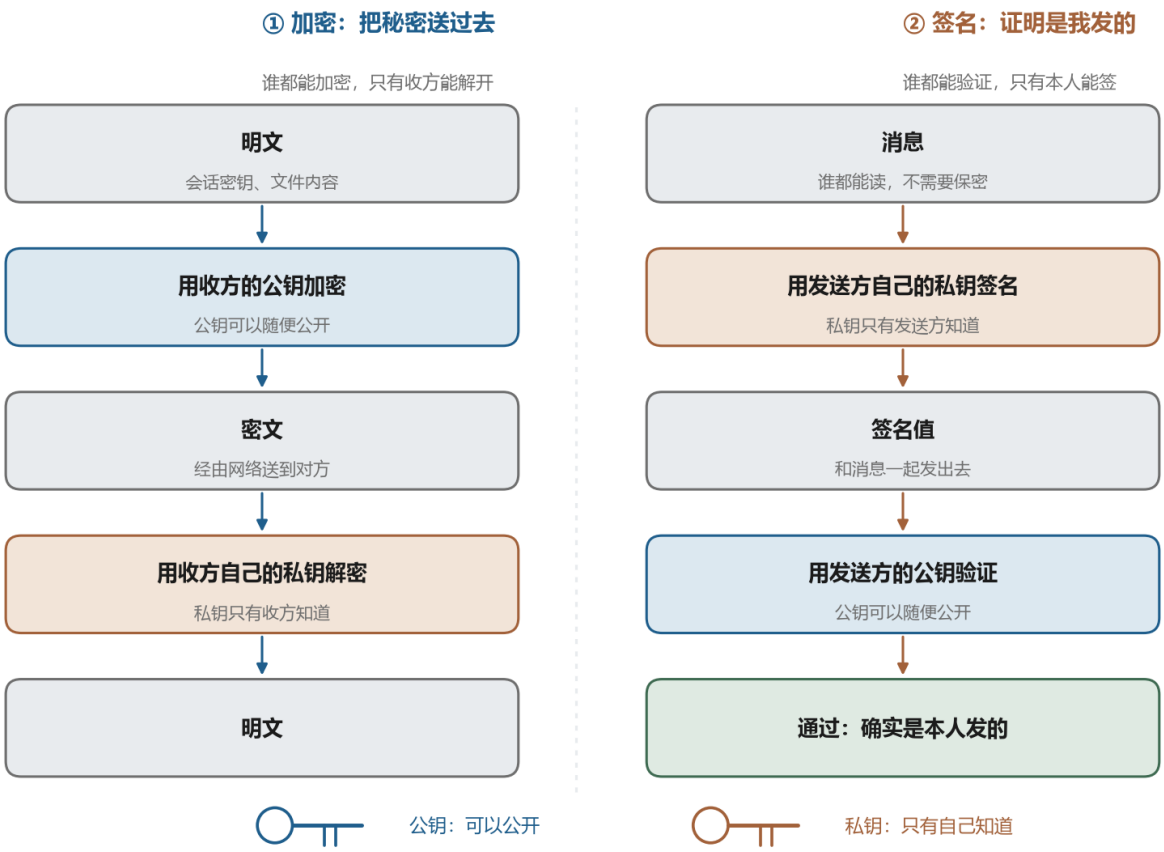



图 5-2 非对称密钥的两种用法，左边加密，右边签名

两种用途容易混淆，记住一句话就够了，加密是“给别人看”，签名是“证明是我”。SSH 的公钥认证用的是签名，不是加密。很多人以为登录时服务器用公钥加密一段数据发给客户端，客户端用私钥解开，这个理解是错的，SSH 的流程在第七章会详细展开。

常见算法有两族。RSA 依靠大整数分解的困难性，密钥长度通常 2048 位以上，签名和验证都比较慢。椭圆曲线算法 ECC 依靠离散对数的困难性，密钥短得多，256 位的 ECC 提供的安全强度大致相当于 3072 位的 RSA。Ed25519 是基于椭圆曲线的签名算法，密钥只有 32 字节，签名 64 字节，速度快，实现也简单，是今天 SSH 的推荐选择。

## 5.5 密钥交换

非对称加密虽然解决了密钥分发，但它的运算速度比对称加密慢几个数量级，用它加密大量数据不划算。所以实际做法是，用非对称的方法协商出一把临时的对称密钥，然后全部数据都用这把密钥加密。协商的过程叫密钥交换。

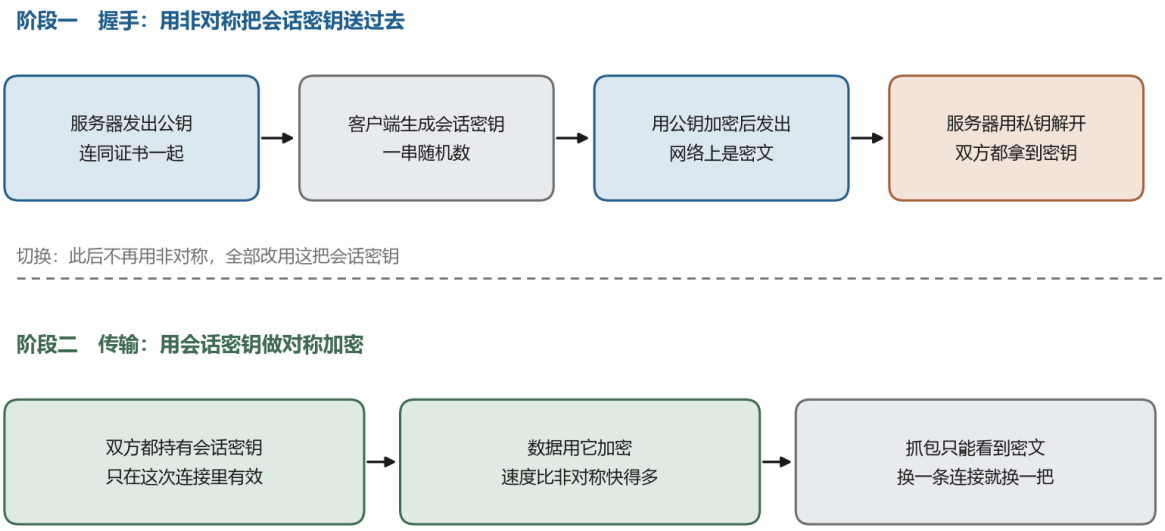



图 5-3 先协商会话密钥，再切换到对称加密

最经典的密钥交换协议是 Diffie 和 Hellman 在 1976 年提出的。双方各自选一个私密数字，用它算出对应的公开值，把公开值发给对方，再用对方的公开值和自己的私密数字算出同一个结果。图 5-4 是这个过程的示意。




图 5-4 双方各自算出相同的共享密钥，窃听者只能看到公开值

这个协议的巧妙之处在于，窃听者能看到两个公开值，但要从公开值反推出私密数字，需要解离散对数问题，在计算上不可行。于是双方得到了一个只有他们知道的共享秘密，而这个秘密从未在网络上传输过。

SSH 里最常用的密钥交换是 `curve25519-sha256`，用 Curve25519 这条椭圆曲线做 Diffie-Hellman，再用 SHA-256 做哈希。它的优点是速度快、密钥短、实现不容易出错。此外还有基于 NIST 曲线的 `ecdh-sha2-nistp256` 等，以及传统的 `diffie-hellman-group14-sha256`。

后者的参数是固定的，安全性依赖那个固定的素数，前向保密性略差一些。

## 5.6 为什么需要证书

非对称加密留下最后一个问题。你怎么知道手里这把公钥真的属于对方。如果攻击者在中间，把“服务器的公钥”换成自己的，你照样能建立加密通道，只是通道的另一端是攻击者。

浏览器访问网站时用的 HTTPS 协议就是靠证书解决这个问题的，它建立在 TLS（一种给网络连接加密的协议）之上。服务器把公钥和身份信息交给一个受信任的第三方，也就是证书颁发机构，由它用自己的私钥签名，形成证书。浏览器内置了一批根证书，只要能沿着签名链一路验到某个根证书，就认为这把公钥是可信的。这套体系叫 PKI，公钥基础设施。

SSH 走了另一条路。它默认不使用证书，而是采用一种叫“首次使用即信任”的策略。第一次连接某台服务器时，客户端把服务器的主机公钥记到 `known_hosts` 里，以后每次连接都核对。如果哪天指纹变了，客户端会大声警告。这套办法的成本低，不需要第三方，缺点是第一次连接时你无法判断对面是不是真的。第七章会详细讨论这个取舍。

值得说明的是，SSH 也支持证书。企业里可以自建一个 CA，给几百台机器签发主机证书，客户端只需要信任这个 CA 的公钥，就能验证所有机器，省掉了每次手工确认指纹的麻烦。JumpServer 这类系统在托管资产时用的就是类似思路。

## 5.7 前向保密

假设某天你的服务器私钥泄露了。攻击者拿到私钥，能不能解开他过去录下来的加密流量。

如果密钥交换用的是 RSA 那种“用服务器公钥加密会话密钥”的方式，答案是能。因为过去每一段流量的会话密钥，都是用服务器公钥加密后传输的，攻击者只要保存了这些密文，拿到私钥就能一一解开。这种设计没有前向保密性。

如果密钥交换用的是 Diffie-Hellman 一族，答案是能也不能。攻击者拿到的是身份私钥，可以用来冒充服务器骗未来的连接，但过去的会话密钥是双方临时算出来的，从来没有用身份私钥加密过，也没有存储在磁盘上，所以解不开。这就是前向保密，也叫完美前向保密。

这也是为什么今天的 SSH 默认使用临时密钥交换算法，而早期的 `diffie-hellman-group1-sha1` 之类的算法被逐步淘汰。判断一个算法有没有前向保密，看它的名字里有没有 `dhe` 或者 `ecdh`。`ssh-rsa` 是签名算法，不涉及密钥交换，但用 RSA 做密钥交换的老方案确实没有前向保密。

## 5.8 这些工具在 SSH 里怎么分工

把上面几样工具放到一起，SSH 的一次连接是这样用它们的。

密钥交换阶段用 Diffie-Hellman 变体，双方算出共享秘密，由此派生出会话密钥。这一步保证了前向保密。服务器在密钥交换过程中用自己的主机私钥对交换内容签名，客户端用对应的

公钥验证，这一步解决了冒充问题，前提是客户端知道正确的公钥。之后的所有数据用对称加密算法处理，每个包附带 MAC，这一步解决了窃听和篡改。

三者缺一不可。没有签名，中间人可以替换公钥。没有对称加密，速度慢到不可用。没有 MAC，攻击者可以改包。理解了这套分工，第六章的协议细节就不难读了。

顺带说一句密钥长度的现实。表 5-1 是几种常见算法组合的密钥与签名长度，这些数字来自算法规范，不涉及任何性能测量。

表 5-1 常见签名算法的密钥与签名长度

| 算法 | 公钥长度 | 私钥长度 | 签名长度 |
| --- | --- | --- | --- |
| Ed25519 | 32 字节 | 32 字节 | 64 字节 |
| ECDSA P-256 | 64 字节（未压缩点） | 32 字节 | 约 70 到 72 字节 |
| RSA 2048 | 约 270 字节（编码后） | 约 1190 字节（PEM，一种把密钥写成文本的编码） | 256 字节 |
| RSA 4096 | 约 530 字节（编码后） | 约 3270 字节（PEM） | 512 字节 |

这组数字解释了一个现象，用 RSA 密钥登录会比 Ed25519 慢一点，因为每次认证都要传更长的签名，做更慢的运算。差异在日常使用中不明显，但在自动化脚本里批量连接几千台机器时，能感觉出来。更重要的是，Ed25519 的密钥短，抄写、备份、贴进配置文件都不容易出错。

第 6 章

# SSH 协议的三层结构

## 6.1 一行明文的版本号

连接建立之后，双方各自先发一行文本，形如 `SSH-2.0-OpenSSH_9.6`。这一行是明文，以回车换行结束，最长 255 字节。它包含协议版本、软件名和可选的注释。之所以允许明文，是因为此时还没有协商出任何密钥，双方需要先说清自己讲的是哪种语言。

服务器通常先发，客户端后发，但协议允许任意顺序，双方都要能容忍对方先开口。这一行的第二个字段是协议版本，`2.0` 表示 SSH-2。历史上存在过 SSH-1，它的设计有严重缺陷，早被废弃，OpenSSH 从 7.6 版起默认不再支持。

这里有个有意思的细节。有些网络设备会往连接里注入自己的横幅，干扰版本交换。OpenSSH 的做法是，在收到以 `SSH-` 开头的行之前，把收到的其他行都当作噪音丢掉，最多容忍 1024 行。如果你在调试输出里看到 `banner exchange` 之后跟着一堆奇怪的内容，多半是中间设备的锅。

## 6.2 三个子协议

SSH 协议被拆成三个子协议，一层叠一层，各自负责一件事。图 6-1 是这个结构。

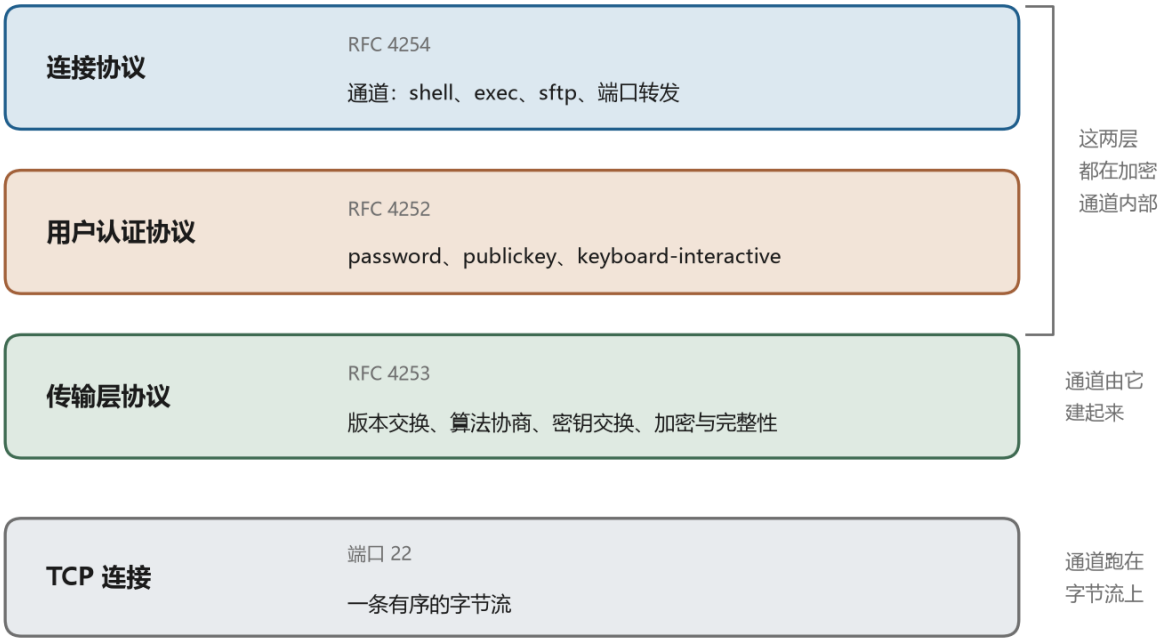



图 6-1 SSH 的三个子协议与它们承载的东西

最底下是传输层协议，也就是 RFC 4253 描述的部分。它负责在 TCP 之上建立一个加密的、

经过完整性保护的通道，完成版本交换、算法协商、密钥交换、密钥派生、重密钥和断开。它不关心你是谁，也不关心你要干什么。

中间是用户认证协议，RFC 4252。它跑在加密通道里面，负责让服务器确认“对面的这个人确实是某个账户的主人”。它支持多种认证方法，客户端可以按顺序尝试，服务器决定接受哪一种。认证成功之后，这个连接就绑定了某个用户身份。

最上面是连接协议，RFC 4254。它也跑在加密通道里，负责把一条连接切成多个逻辑通道，每个通道可以承载一种用途，交互式 shell、执行单条命令、文件传输、端口转发。一条 SSH 连接里可以同时开着很多个通道，它们互不干扰。

这个分层解释了为什么一条 SSH 连接能干那么多事。你开一个终端窗口跑命令，同时用 `scp` 传文件，又用 VS Code 在远端开发，如果都走同一条连接，只需要一次认证、一次密钥交换，后面全是通道复用。第八章讲端口转发时会用到这个性质。

## 6.3 一个数据包长什么样

加密通道建立之后，SSH 传输的是二进制包。每个包的格式固定，如图 6-2。




图 6-2 SSH 二进制包的字段结构

开头 4 个字节是包长度，表示后面还有多少字节。接下来 1 个字节是填充长度，然后是载荷（payload，包里真正要传的内容），最后是随机填充（padding，凑长度用的无用字节）。填充的作用是让包的长度是加密算法分组大小的整数倍，同时增加一点随机性。协议要求填充至少 4 字节，整个包（不含长度字段和 MAC）至少 16 字节。载荷最长 32768 字节，整包不超过 35000 字节，这是 RFC 4253 给出的下限要求，所有实现都必须能处理。

包尾是 MAC（消息认证码，第五章讲过），长度由协商的算法决定，HMAC-SHA256 是 32 字节。接收方收到包之后，先算 MAC 比对，通过之后再解密载荷。这种顺序叫“先认证后解密”，可以防止攻击者把构造的密文喂给解密器做旁路分析。

后来又出现了一种改进叫 ETM，全称是 encrypt-then-MAC。传统的顺序是先加密整个包（含长度字段），再对密文算 MAC。ETM 的做法是不加密长度字段，但把长度字段也纳入 MAC 的

计算范围。这样做的好处是接收方可以先看长度、先验 MAC，再决定要不要解密，既省了算力，也避免了一些边界攻击。OpenSSH 的默认 MAC 算法都带 `-etm` 后缀。

载荷的第一个字节是消息类型编号，后面跟着这个消息自己的字段。表 6-1 是书里会用到的几个编号。

表 6-1 SSH 常用消息编号

| 编号 | 消息 | 出现在哪个阶段 |
|---|---|---|
| 1 | SSH_MSG_DISCONNECT | 任意时刻，表示断开及原因 |
| 20 | SSH_MSG_KEXINIT | 算法协商，双方各发一次 |
| 21 | SSH_MSG_NEWKEYS | 密钥交换结束，通知切换密钥 |
| 30 | SSH_MSG_KEX_ECDH_INIT | 客户端发送临时公钥 |
| 31 | SSH_MSG_KEX_ECDH_REPLY | 服务端回复临时公钥、主机公钥与签名 |
| 50 | SSH_MSG_USERAUTH_REQUEST | 认证请求 |
| 51 | SSH_MSG_USERAUTH_FAILURE | 认证失败，并列出可用的方法 |
| 52 | SSH_MSG_USERAUTH_SUCCESS | 认证成功 |
| 60 | SSH_MSG_USERAUTH_PK_OK | 公钥被接受，可以开始签名 |
| 80 | SSH_MSG_GLOBAL_REQUEST | 全局请求，比如请求转发端口 |
| 90 | SSH_MSG_CHANNEL_OPEN | 打开一个通道 |
| 98 | SSH_MSG_CHANNEL_REQUEST | 通道内的请求，比如分配伪终端 |

这些编号不需要背，但看到抓包或者日志里出现 `SSH_MSG_KEXINIT` 这样的字符串时，能立刻知道现在处于哪个阶段，排查会快很多。

## 6.4 算法协商

版本交换完成后，双方各发一个 `SSH_MSG_KEXINIT`。这个消息里有一个 16 字节的随机 cookie，然后是一串名字列表，分别列出自己支持的密钥交换算法、主机密钥算法、加密算法、MAC 算法、压缩算法、语言，每个方向各一份。

```
kex_algorithms          curve25519-sha256,ecdh-sha2-nistp256,diffie-hellman-group14-sha2\
    56
server_host_key_algs    ssh-ed25519,rsa-sha2-512,rsa-sha2-256
encryption_c2s          chacha20-poly1305@openssh.com,aes256-gcm@openssh.com,aes256-ctr
mac_c2s                 hmac-sha2-256-etm@openssh.com,hmac-sha2-512-etm@openssh.com
compression_c2s         none,zlib@openssh.com
```

协商规则很朴素，从客户端的列表里按顺序取，取第一个服务端也支持的。这个规则叫“客户端优先”，意味着最终选择主要由客户端决定。图 6-3 是这个比较过程的示意。

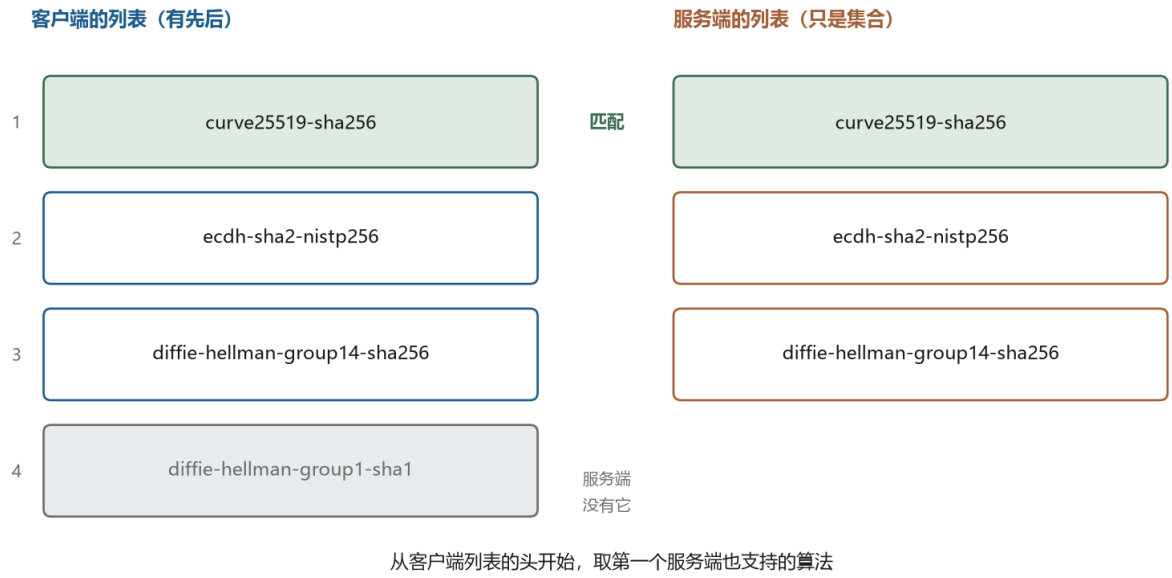



图 6-3 算法协商，取第一个双方都支持的项

客户端优先的设计有一个副作用。如果客户端被换成了老版本，只能提供弱算法，服务端又兼容它，那这条连接就会用弱算法。这让老客户端的兼容性变好了，代价是安全性由最弱的一方决定。企业里如果统一了客户端版本，可以在服务端配置里写死允许的算法，把弱的全部禁掉。

协商失败时会看到这样的报错。

```
Unable to negotiate with 10.0.0.5 port 22: no matching key exchange method found.
Their offer: diffie-hellman-group1-sha1
```

这类报错说明双方没有交集，处理方法有两个，要么升级一端，要么在客户端用 `-oKexAlgorithms=+diffie-hellman-group1-sha1` 显式允许那个老算法。后者是临时手段，不该长期使用。

## 6.5 密钥交换与密钥派生

协商完成后进入密钥交换。以 `curve25519-sha256` 为例，过程如图 6-4。




图 6-4 一次 ECDH 密钥交换的完整过程

客户端生成一对临时密钥，把临时公钥 `Q_C` 放进 `SSH_MSG_KEX_ECDH_INIT` 发出去。服务端也生成一对临时密钥，用自己的主机私钥对一段哈希值签名，然后把主机公钥、临时公钥 `Q_S` 和签名一起放进 `SSH_MSG_KEX_ECDH_REPLY` 发回。双方各自用自己的临时私钥和对方的临时公钥算出共享秘密 `K`。

那段被签名的哈希值叫交换哈希，记作 `H`。它的计算方式是把双方的版本字符串、双方的 KEXINIT 原文、主机公钥、双方的临时公钥、共享秘密按固定顺序拼接后做哈希。

```
H = SHA256(V_C || V_S || I_C || I_S || K_S || Q_C || Q_S || K)
```

这个 `H` 有两个作用。一是作为签名对象，让客户端确认服务端确实持有主机私钥。二是作为派生密钥的材料，`K` 只用来算 `H`，后面的会话密钥都从 `H` 派生。这样设计的好处是，即使 `K` 的某些比特有偏差，也不会直接影响密钥质量。

派生过程用同一个哈希函数反复计算，每次在末尾拼一个不同的字母。

```
IV_c2s   = HASH(K || H || "A" || session_id)
IV_s2c   = HASH(K || H || "B" || session_id)
Key_c2s  = HASH(K || H || "C" || session_id)
Key_s2c  = HASH(K || H || "D" || session_id)
MAC_c2s  = HASH(K || H || "E" || session_id)
MAC_s2c  = HASH(K || H || "F" || session_id)
```

六个字母对应六个方向的密钥材料，客户端到服务端和服务端到客户端各有一套，互不共用。

这叫双向独立密钥，好处是即使某一方向的密钥被攻破，另一方向仍然安全。`session_id` 是第一次密钥交换产生的 `H`，它在整条连接的生命周期里不变，用于把后续的重密钥绑定到最初的这次 交换上。

密钥长度不够时，把前面的结果再哈希一次并拼接，直到够长。这套“扩展”规则在 RFC 4253 的第 7.2 节，实现起来只有几行代码。

## 6.6 重密钥

会话密钥不能一直用。用得越久，泄露的风险越大，被分析的材料也越多。协议建议在传输了 1GB 数据或者用满 1 小时之后重新做一次密钥交换，这两个阈值在 OpenSSH 里可以用 `RekeyLimit` 调整。

```
ssh -oRekeyLimit=500M l40-server
```

重密钥的过程和第一次密钥交换一样，只是不再重新协商算法，直接复用之前的结果。新算出的密钥替换旧的，`session_id` 保持不变。这个过程对上层完全透明，你的 shell 不会有任何感觉。

顺带解释一个现象。有时候 `ssh -vv` 的输出里会突然出现一段新的 `SSH2_MSG_KEXINIT`，看起来像是重新连接了，其实只是触发了重密钥。如果连接质量不好、丢包严重，重密钥可能失败并导致断开，这时把 `RekeyLimit` 调大一点能缓解。

## 6.7 认证层

加密通道建好之后，客户端发送 `SSH_MSG_USERAUTH_REQUEST`，里面包含用户名、要使用的服务名（固定是 `ssh-connection`）和认证方法。服务器可以接受，可以拒绝并列出它还支持哪些方法，也可以要求继续。

SSH 定义了三种基本方法。`password` 最简单，明文密码放在加密通道里传过去。`publickey` 用签名证明持有私钥，是这本书的重点，细节放在第七章。`keyboard-interactive` 是一问一答的形式，服务器可以问任意多个问题，适合一次性口令、短信验证码、多因素认证。此外还有 `hostbased` 和 `gssapi-with-mic`，前者用主机密钥认证，后者走 Kerberos，日常较少见。

认证可以多轮。服务器返回失败时会给一个列表，客户端按自己的策略依次尝试。OpenSSH 客户端默认最多尝试 6 次，可以用 `NumberOfPasswordPrompts` 限制。堡垒机场景下经常配置成只允许 `publickey`，这时密码登录会被直接拒绝，报错里会写 `Permission denied (publickey)`。

有一个细节容易让人困惑。公钥认证的第一步，客户端会先发一个“试探”请求，只带公钥不带签名，问服务器“这把钥匙你能接受吗”。服务器如果接受，回一个 `SSH_MSG_USERAUTH_PK_OK`，客户端才开始签名。这个设计的目的是避免在服务器不认可这把钥匙时白做一次签名运算，同时也让客户端知道该用哪把钥匙。有些服务器为了不泄露“某个用户名是否存在”，会跳过这一步直接要求签名，两种行为都合法。

## 6.8 连接层

认证成功之后，客户端可以打开通道。每个通道有本地编号和远端编号，双方各自维护一套，互不干扰。通道类型有几种，`session` 用于 shell 和命令执行，`direct-tcpip` 用于本地转发，`forwarded-tcpip` 用于远程转发。

打开通道之后，客户端在通道里发请求。最常见的几个是 `pty-req` 请求分配伪终端并告知窗口大小，`shell` 请求启动登录 shell，`exec` 请求执行一条命令，`subsystem` 请求启动一个子系统（`sftp` 就是通过它启动的），`window-change` 在窗口大小变化时通知对方。

连接层还有流量控制。每个通道有一对窗口大小，表示对方还能接收多少字节的数据。发送方发完数据要扣减窗口，接收方处理完数据后回一个 `CHANNEL_WINDOW_ADJUST` 把窗口补回去。如果窗口耗尽，发送方必须停下来等。这套机制防止了快速的生产者把慢速的消费者撑爆。传大文件时如果速度上不去，有时候和窗口大小有关。

全局请求独立于通道，比如 `tcpip-forward` 用于请求服务端监听某个端口，这是第八章远程转发的基础。还有一类请求叫 `keepalive@openssh.com`，客户端定期发一个，相当于心跳，防止中间的 NAT 设备因为长时间没有流量而把连接表项清掉。这就是 `ServerAliveInterval` 选项背后做的事。

## 6.9 看看自己的机器支持什么

OpenSSH 提供了查询算法清单的选项，比翻文档快。

```
ssh -Q kex          # 支持的密钥交换算法
ssh -Q key          # 支持的主机密钥与用户密钥类型
ssh -Q cipher       # 支持的对称加密算法
ssh -Q mac          # 支持的 MAC 算法
ssh -Q key-sig      # 支持的签名算法
```

把这些输出和 `sshd -T` 的结果对照，就能知道协商会落在哪个算法上。

```
sudo sshd -T | grep -E 'kexalgorithms|ciphers|macs|hostkeyalgorithms'
```

`sshd -T` 会打印服务端生效的完整配置，包括默认值和配置文件里的覆盖值。这是排查“为什么用了这个算法”的最快方式，比读配置文件可靠，因为配置文件可能有 `Include`、有 `Match` 块，肉眼追不全。

## 6.10 抓包能看到什么

最后说一个常见疑问。既然 SSH 加密了，抓包还有用吗。

有用，但只能看到结构。版本交换那一行是明文，能直接读出来。之后的包，载荷是不可读的随机字节；长度字段是否可见取决于双方是否启用 ETM，启用了就能看到明文长度，没启用则

连长度也一起加密。无论哪种情况，包的大小和出现的时间都瞒不住。比如你敲一个字符，客户端可能立刻发一个很小的包，交互式命令的输出会产生一串中等大小的包，传文件时包长接近上限。通过分析包长和时序，可以推测出你正在做什么类型的事，这类研究叫流量分析。

防御流量分析需要固定长度的填充，代价是带宽浪费，SSH 没有这么做。企业里如果确实担心这个，通常是在外层再套一层隧道，或者在网络层面做流量整形。对绝大多数场景，这个风险可以忽略。

抓包在排错时仍然很有价值。`tcpdump -n -i any port 22` 能看到连接是否建立、有没有重传、RST（强制断开连接的消息）是谁发的。这些信息足以区分“网络不通”和“服务拒绝”。真要看协议内容，用 `ssh -vv` 比抓包方便得多，因为客户端本来就会把每个阶段的解析结果打印出来。

第 7 章

# 密钥、指纹与信任

## 7.1 生成一对钥匙

登录一台服务器，最稳妥的方式是公钥认证。它需要在客户端准备一对密钥，把公钥交到服务器上，私钥留在自己手里。生成密钥的命令是 `ssh-keygen`。

```
ssh-keygen -t ed25519 -C "alice@laptop" -f ~/.ssh/id_ed25519_lab
```

`-t` 指定算法类型，`-C` 是注释，会写进公钥末尾，方便你在服务器上认出这是哪台机器的钥匙。`-f` 指定文件名，不指定的话默认是 `~/.ssh/id_ed25519`。命令执行过程中会问你要不要设置口令，这个口令用来加密私钥文件本身，不是服务器的密码。

设不设口令是一个真实的取舍。设了口令，私钥文件被偷走也打不开，但每次用钥匙都要输一遍，除非配合 ssh-agent（一个替你保管解密后私钥、代为签名的小程序）。不设口令，用起来方便，但私钥文件一旦泄露就等于身份泄露。自动化脚本里的密钥通常不设口令，靠文件权限和磁盘加密来保护。人用的密钥建议设口令。

生成之后会得到两个文件。`id_ed25519_lab` 是私钥，权限自动设为 600。`id_ed25519_lab.pub` 是公钥，内容是一行文本。

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIExampleKeyMaterialHere alice@laptop
```

把这行文本追加到目标机上的 `~/.ssh/authorized_keys` 里，就能用对应的私钥登录了。这件事手工做很容易出错，`ssh-copy-id` 可以代劳。

```
ssh-copy-id -i ~/.ssh/id_ed25519_lab.pub alice@10.0.0.5
```

它会连过去，把你的公钥追加进去，还会顺手修好目录和文件的权限。Windows 上没有这个命令，可以手动把公钥内容粘贴进去。

## 7.2 私钥文件里装的是什么

打开一个 OpenSSH 格式的私钥文件，看到的是这样的文本。

```
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZW
QyNTUxOQAAACDxampleBase64MaterialGoesHereAndContinuesForSeveral
LinesUntilTheKeyBlobIsCompleteAAAAJgExampleCommentFieldHere
-----END OPENSSH PRIVATE KEY-----
```

开头和结尾的两行是标记，中间是一段 base64 编码的二进制数据。解码之后是一个按固定格式排列的结构，包含算法名、公钥、私钥、注释，可能还有用于校验的字段。如果私钥设了口令，私钥那部分会用口令派生的密钥加密，派生函数是 bcrypt，迭代次数很高，暴力破解的代价很大。

这里要澄清一个常见的混乱。`.pem`、`id_rsa`、`id_ed25519` 这些名字看起来像是三种不同的东西，其实只是两种格式和两种算法的组合。`id_rsa` 里的 `rsa` 指算法，文件格式可能是老的 PEM 也可能是新的 OpenSSH 格式。`.pem` 是文件扩展名，指 PEM 编码，它本身不说明里面装的是 RSA 还是 ECC。从 JumpServer 下载的那种 `ssh.jumpserver.pem`，打开看内容，多数是标准的 OpenSSH 或 PEM 私钥，只是名字沿用了云厂商的习惯。

格式可以用 `ssh-keygen` 互相转换。老的 PEM 格式兼容性更好，某些第三方工具只认它。

```
ssh-keygen -p -m PEM -f ./ssh.jumpserver.pem     # 转成 PEM 格式
ssh-keygen -p -m RFC4716 -f ./key                # 转成 RFC 4716 格式
ssh-keygen -y -f ./ssh.jumpserver.pem            # 从私钥导出公钥
```

最后那条命令很实用。当你只有私钥，想知道对应的公钥是什么样，用它就能打印出来，不用去翻服务器上的 `authorized_keys`。

## 7.3 公钥认证到底怎么验证

很多人以为公钥登录的过程是“服务器用公钥加密一段数据，客户端用私钥解开”，这个理解是错的。真正的过程是签名，如图 7-1。




私钥始终留在客户端：发出的只有签名，私钥本身从不传输

图 7-1 公钥认证的四步，私钥始终不出本机

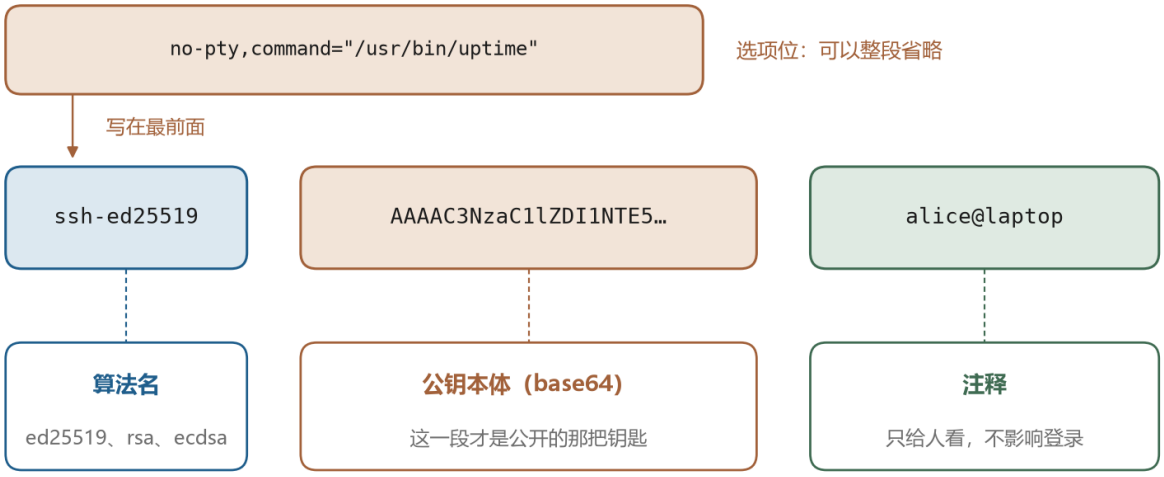
客户端先发一个试探请求，告诉服务器“我想用这把公钥登录”，里面只有公钥本身。服务器去 `authorized_keys` 里查，找到了就回一个确认。然后客户端构造一段数据，包含会话标识（第一次密钥交换时算出的那个哈希）和这次的认证请求原文，用自己的私钥对它签名，把签名发过去。服务器用客户端提供的公钥验证签名，验过了就认为客户端确实持有对应的私钥。

签名的对象里有会话标识，这一点很关键。会话标识是密钥交换时算出的那个哈希 `H`，每条连接都不一样。把会话标识放进签名，意味着这个签名只对当前这条连接有效，攻击者录下来也没法重放到另一条连接上。这是第六章提到的交换哈希的又一个用途。

整个过程中私钥从未离开客户端，连加密后的形式都没有传输过。这比“私钥加密、公钥解密”的方案安全得多，因为后者的私钥要参与运算并可能暴露时序信息。也正因为如此，服务器上只需要存公钥，即使服务器被攻破，攻击者拿到的也只是公钥，没法冒充你登录别的机器。

## 7.4 authorized_keys 的写法

这个文件的每一行是一把被授权的公钥，格式如图 7-2 所示，三个字段用空格分开。




三段只用空格隔开，一行就是一条公钥；匹配到哪一行，那一行的选项跟着生效。

图 7-2 authorized_keys 一行内容的字段构成

第一个字段是算法名，第二个是公钥本体的 base64，第三个是注释。注释不参与验证，可以随便写，很多人用它记录这把钥匙属于谁、在哪台机器上。用 `ssh-keygen -l -f` 可以查看文件里每把钥匙的指纹和注释。

```
ssh-keygen -l -f ~/.ssh/authorized_keys
```

这一行前面还可以加选项，用来限制这把钥匙能做什么。这是堡垒机和大规模部署里非常有用的功能。

```
from="203.0.113.0/24",no-pty,no-port-forwarding ssh-ed25519 AAAAC3... alice@laptop
command="/usr/local/bin/backup.sh",restrict ssh-ed25519 AAAAC3... backup@nas
```

第一行限制来源网段，禁止分配伪终端，禁止端口转发。第二行强制这条钥匙只能执行指定命令，其他一切都禁用。`restrict` 是一个总开关，等价于关掉所有可选功能，再按需打开。这类选项在需要给第三方临时权限时特别有用，你给的是一把只能做一件事的钥匙，而不是一个完整的 shell。

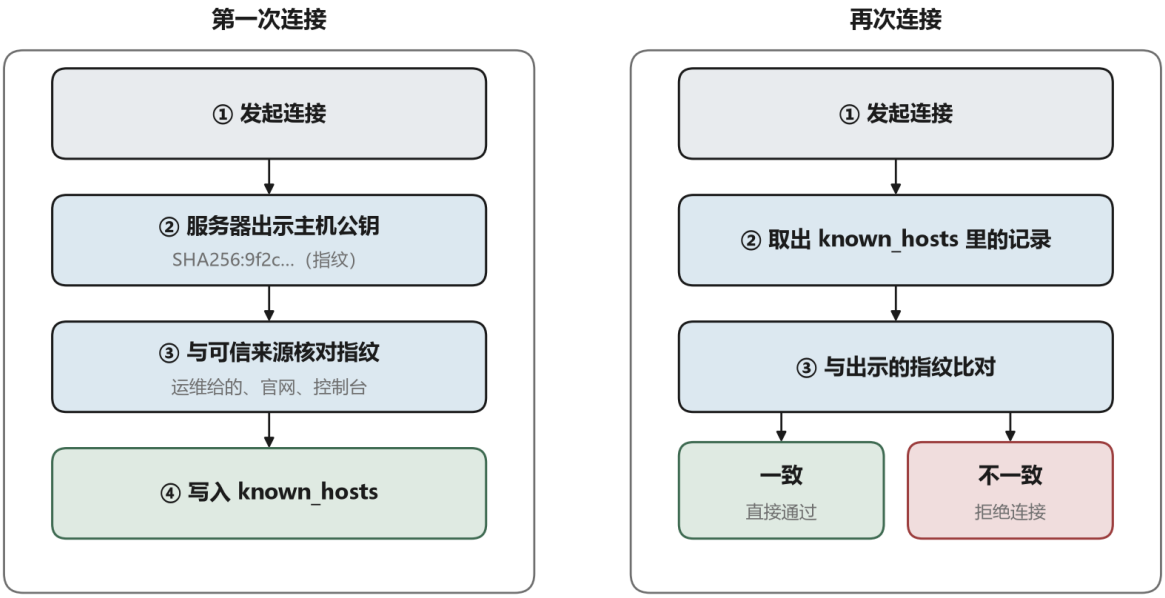
## 7.5 主机密钥与首次信任

前面讲的是服务器验证你，反过来你也需要验证服务器。如果攻击者在中间，他可以假装成服务器，骗你输入密码或者骗取你的公钥签名。SSH 用主机密钥解决这个问题。

每台运行 sshd 的机器在安装时会生成一对主机密钥，公钥放在 `/etc/ssh/ssh_host_ed25519_key.pub` 之类的文件里。客户端第一次连接时，服务器把主机公钥发过来，客户端算一个指纹显示给你。

```
The authenticity of host '10.0.0.5 (10.0.0.5)' can't be established.
ED25519 key fingerprint is SHA256:ExampleFingerprintStringHere.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

你回答 yes，这把公钥就被写进 `~/.ssh/known_hosts`。以后每次连接，客户端都拿服务器发来的公钥和记录比对，一致就通过，不一致就报警并拒绝连接。这套策略叫首次使用即信任，简称 TOFU，如图 7-3。




指纹对不上：可能是中间人，也可能是服务器重装了系统。先核实，再决定是否删掉旧记录。

图 7-3 首次连接记录指纹，之后每次比对

指纹变了意味着什么。有两种可能，一是服务器重装了系统或者换了主机密钥，二是真的有人在中间。客户端无法区分这两种情况，所以它只能停下来问你。

```
WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
```

看到这段警告不要条件反射地删掉 `known_hosts` 里那一行。先确认服务器是不是最近重装过，或者找管理员核对指纹。确认没问题之后，用下面的命令删掉旧记录。

```
ssh-keygen -F 10.0.0.5              # 查这条记录在文件里的位置
ssh-keygen -R 10.0.0.5              # 删除这条记录
ssh-keyscan -t ed25519 10.0.0.5     # 重新抓取主机公钥
```

顺带说一句 `known_hosts` 的一个坑。如果你用 IP 连过一次，又用域名连过一次，文件里会

有两条记录，因为它们被当作两台不同的主机。云端机器换 IP 之后重新分配，也容易出现指纹不匹配的告警。企业里机器多的时候，这套机制会变得难以维护，所以有了下面的证书方案。

## 7.6 ssh-agent 与密钥转发

设了口令的私钥每次用都要输一遍，很烦。`ssh-agent` 是一个后台进程，你把私钥解密后交给它保管，之后所有 SSH 连接都向它请求签名，不用再输口令。

```
eval "$(ssh-agent -s)"       # start the agent
ssh-add ~/.ssh/id_ed25519    # add the key, passphrase asked once
ssh-add -l                   # list loaded keys
```

Windows 上 OpenSSH 客户端自带的 agent 是一个系统服务，在“服务”里叫 `OpenSSH Authentication Agent`，启动之后用 `ssh-add` 添加即可，需要管理员权限才能设为自动启动。

密钥转发是另一个功能，叫 Agent Forwarding。开启之后，你在跳板机上执行 `ssh` 连第三台机器时，跳板机会把认证请求转发回你的本机，由本机的 agent 完成签名。这样私钥始终留在自己电脑上，跳板机上不需要放任何密钥。

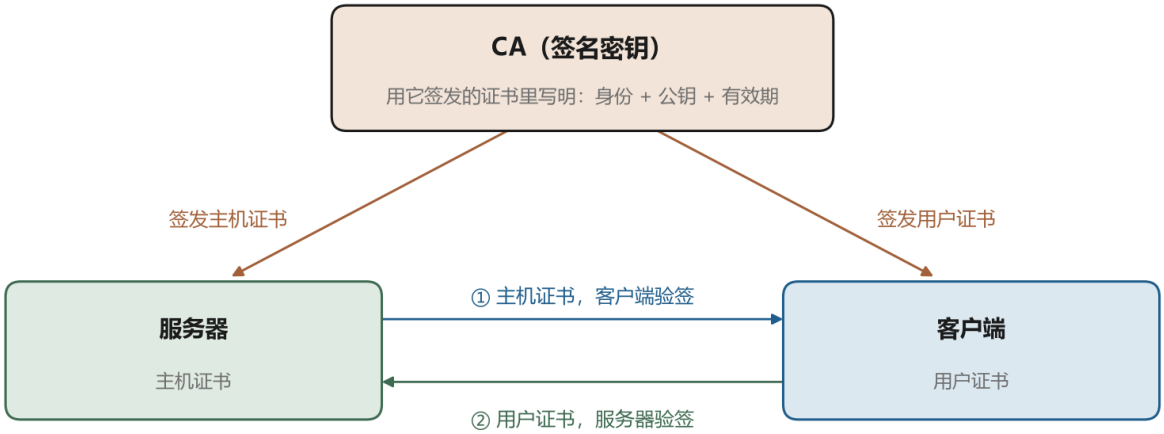
```
ssh -A jump-host
```

这个功能很方便，风险也不小。跳板机上的 root 用户可以通过 agent 的 socket 让你的 agent 签名任意数据，相当于在你不注意的时候用你的身份登录其他机器。Agent Forwarding 的原则是只在你信任的跳板机上开启，用完立刻退出。更好的做法是用第八章的 `ProxyJump`，它不需要转发 agent，认证在本机完成。

## 7.7 证书

机器一多，`known_hosts` 和 `authorized_keys` 就变成噩梦。每加一台机器都要分发公钥，每换一次密钥都要通知所有人。SSH 证书是为了解决这个问题而设计的。

思路和 HTTPS 一样，建立一个 CA，用它的私钥签发证书。主机证书里写明“这台机器的主机公钥属于这个主机名”，用户证书里写明“这把公钥属于这个用户，有效期到某天”。客户端只需要信任 CA 的公钥，就能验证所有由它签发的证书，如图 7-4。




双方只在本地预置同一个 CA 公钥：不再逐个核对主机指纹，也不再逐台分发 authorized_keys。

证书里还写着有效期与用途，过期即失效，可以按时换发。

图 7-4 由 CA 签发主机证书与用户证书

```
ssh-keygen -t ed25519 -f ca_key -C "ssh-ca"          # CA key pair
ssh-keygen -s ca_key -I alice -n alice -V +52w alice.pub   # user cert
ssh-keygen -s ca_key -h -I web01 -n web01.example.com -V +52w host.pub  # host cert
```

`-I` 是证书标识，`-n` 是允许的主体名，`-V` 是有效期，`-h` 表示这是主机证书。签发出来的文件以 `-cert.pub` 结尾，客户端拿着它就能登录，服务器端只要在 `sshd_config` 里配置 `TrustedUserCAKeys` 指向 CA 公钥即可。

JumpServer 这类系统在托管资产时，本质上做的也是这件事，只不过它把签发和管理都封装进了界面。第十一章会讲它的具体做法。

## 7.8 一份能用的客户端配置

把这一章的内容落到配置上，`~/.ssh/config` 大致长这样。

```
Host *
    ServerAliveInterval 30
    ServerAliveCountMax 3
    AddKeysToAgent yes

Host lab
    HostName 10.0.0.5
    User alice
    Port 22
    IdentityFile ~/.ssh/id_ed25519_lab
    IdentitiesOnly yes
```

`ServerAliveInterval` 每 30 秒发一个心跳，防止 NAT 设备把闲置连接清掉。`AddKeysToAgent` 让第一次输入的口令被记住。`IdentitiesOnly yes` 只使用配置里指定的私钥，避

免客户端把 agent 里的每把钥匙都试一遍，服务器限制尝试次数时会因此被拒绝。

这个文件从上往下读，第一个匹配的 `Host` 块生效，后面的同名选项覆盖前面的。`Host *` 放在最前面作为默认值，后面的具体主机块覆盖它，这是社区里通行的写法。第十二章会详细讲匹配规则，包括通配符和 `Include`。

第 8 章

# 端口转发，把远方的服务搬到本地

## 8.1 只开一个端口的世界

很多内网只允许外部访问一个端口，通常就是 SSH 的 22。数据库、内部网页、监控面板全在里面，从外面碰不到。这种设计出于安全考虑，但也带来不便。你需要临时看一眼内网的某个网页，或者用本地的图形化工具连一下内网的数据库，怎么办。

办法是把那些服务“搬”到你本地来。SSH 的连接层本来就支持多路复用，一条加密连接里可以开很多个通道，每个通道承载一路数据。端口转发就是利用这个能力，把某个 TCP 端口上的流量塞进 SSH 通道，从另一端送出去。对使用者来说，本地凭空多了一个端口，连它就等于连到了远端的某个服务。

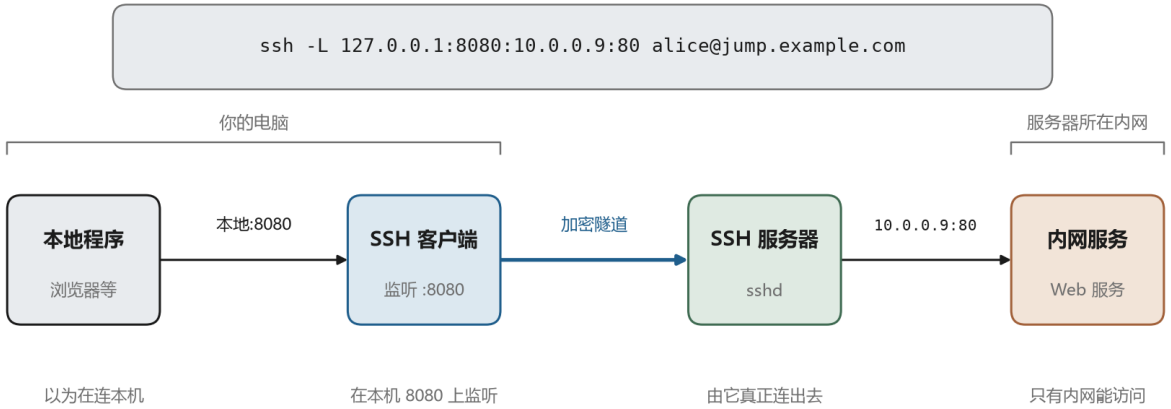
SSH 定义了三种转发方式，本地转发、远程转发、动态转发。它们的区别只在于谁监听、谁发起连接。

## 8.2 本地转发

本地转发的语法是 `-L`，格式为 `本地地址:本地端口:目标地址:目标端口`。

```
ssh -L 127.0.0.1:8080:10.0.0.9:80 alice@jump.example.com
```

这条命令的意思是，在本机监听 8080 端口，任何连到这个端口的流量，都通过 SSH 连接送到 `jump.example.com`，再由那台机器向 `10.0.0.9` 的 80 端口发起连接。图 8-1 是这个结构。




监听的那一端在本地，真正去连服务的那一端在远端。

图 8-1 本地转发，监听在本地，连接由远端发起

这里有一个特别容易搞错的地方。目标地址 是由 SSH 服务器去解析和连接的，不是由你的本机。所以 `10.0.0.9` 这个地址只需要在服务器那一侧可达，你的本机根本不需要认识它。很多人以为要先把 `10.0.0.9` 写进本机的 hosts，其实不必。

另一个容易搞错的地方是监听地址。写 `127.0.0.1:8080` 表示只监听本机回环，其他机器连不进来。写 `0.0.0.0:8080` 表示监听所有网卡，同一个局域网里的人也能用你的机器做跳板。默认建议用前者，除非你明确知道自己在做什么。

```
ssh -L 8080:10.0.0.9:80 alice@jump.example.com    # bind 127.0.0.1 by default
ssh -L 0.0.0.0:8080:10.0.0.9:80 alice@jump.example.com
```

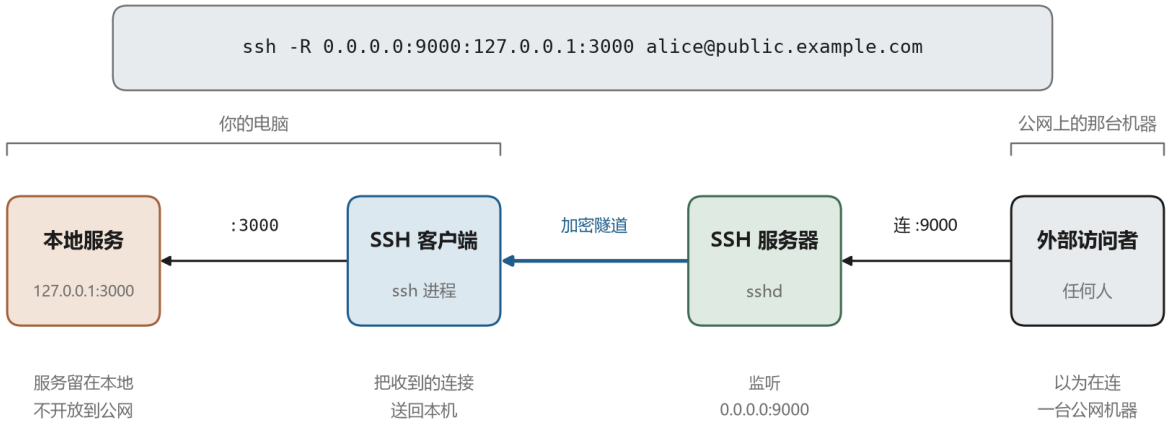
本地转发最典型的用法有三个。看内网的网页面板，把远端的 80 或 443 映射到本地。连内网的数据库，把 3306 映射出来，用本地的客户端连 `127.0.0.1:3306`。访问只允许从特定机器访问的服务，比如某个只对跳板机开放的管理接口。

## 8.3 远程转发

远程转发把方向倒过来，语法是 `-R`，格式为 远程地址:远程端口:目标地址:目标端口。

```
ssh -R 0.0.0.0:9000:127.0.0.1:3000 alice@public.example.com
```

这条命令的意思是，在 `public.example.com` 上监听 9000 端口，任何连到它的流量，通过 SSH 连接送回你的本机，再由你的本机连到 `127.0.0.1:3000`。图 8-2 是这个结构。




同样的隧道，反了个方向：监听放在远端，服务留在本地。

图 8-2 远程转发，监听在远端，连接回到本地

这个方向适合的场景是，你的开发机在内网，没有公网地址，但你想让外面的人访问你本机跑的服务。或者反过来，你在家里，想把公司内网某台机器的服务暴露到一台有公网地址的跳板机上，临时给同事看。

远程转发有一个常见的坑，就是监听地址。默认情况下，即使你写了 `0.0.0.0:9000`，服务端

也可能只在回环地址上监听，外面仍然连不进来。原因是 sshd 的配置项 `GatewayPorts` 默认是 `no`，它禁止把转发的端口绑定到非回环地址（回环地址指 127.0.0.1，永远指向本机）。要让外部能连，服务端必须设置 `GatewayPorts yes` 或者 `GatewayPorts clientspecified`，然后重启 sshd。这件事你自己改不了，需要服务器管理员配合。

## 8.4 动态转发

前面两种转发都要事先指定目标地址和端口。如果你想让浏览器把所有网站都走 SSH 出去，总不能每个网站配一条 `-L`。这种需求用动态转发，语法是 `-D`。

```
ssh -D 127.0.0.1:1080 alice@jump.example.com
```

这条命令在本机 1080 端口起了一个 SOCKS5 代理（SOCKS 是一种代理协议，客户端把目标地址交给代理，由代理去连）。浏览器配置成使用这个代理之后，每次访问网站，浏览器先把“我要访问 example.com 的 443 端口”告诉代理，代理通过 SSH 通道让服务器去连，再把数据转发回来。目标地址是运行时由客户端指定的，所以一条命令能覆盖所有目标。图 8-3 是这个结构。

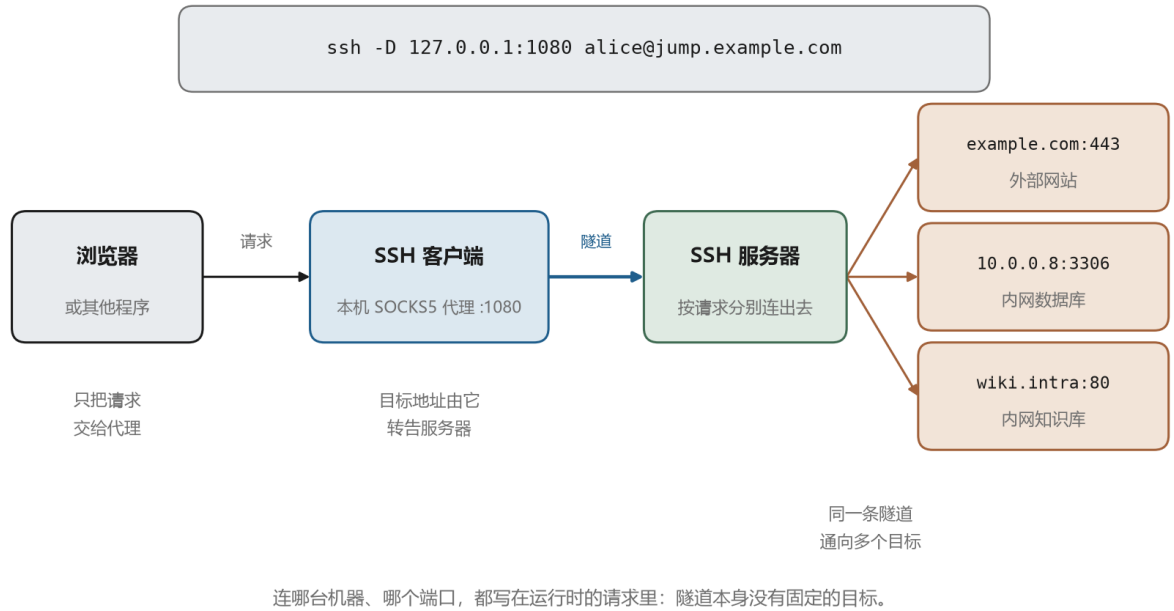



图 8-3 动态转发，目标地址在运行时决定

动态转发的用途主要是两类。一类是临时改变出口位置，让流量从服务器那一侧出去。另一类是访问内网资源，浏览器通过 SOCKS 代理访问内网的多个站点，不用一个个配 `-L`。

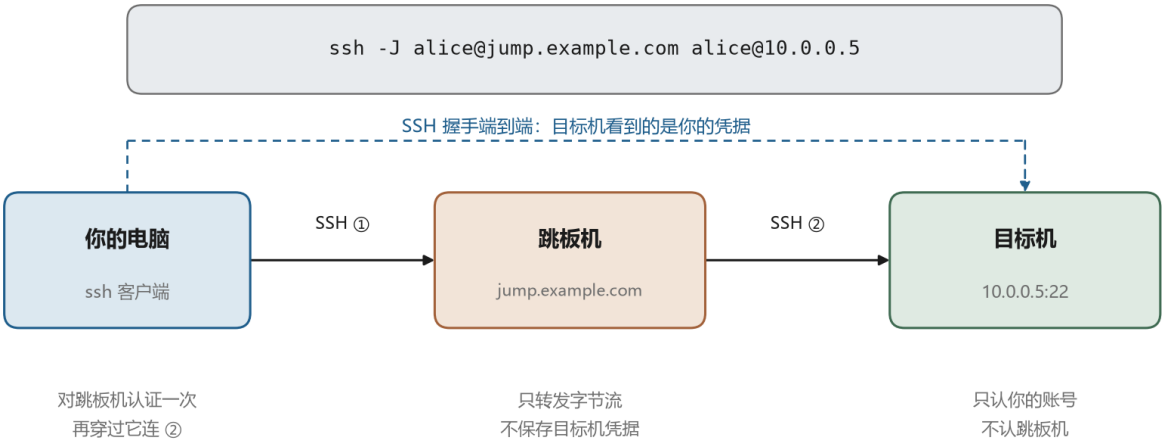
需要提醒的是，`-D` 只处理 TCP。有些应用用 UDP（第三章讲过，它不保证送达，但开销小），DNS 查询和视频通话都属于这一类，它们不经过 SOCKS 代理。另外，浏览器自己会做 DNS 解析，如果配置不当，目标域名可能在本地就被解析了，泄露你在访问什么。Firefox 可以在 `about:config` 里把 `network.proxy.socks_remote_dns` 设为 `true`，让 DNS 也走代理。

## 8.5 跳板与 ProxyJump

前面讲的转发是在一条已经建立的连接上做文章。还有一类需求是在建立连接的过程中绕路，也就是先连 A，再从 A 连 B。这件事由 `ProxyJump` 完成。

```
ssh -J alice@jump.example.com alice@10.0.0.5
```

这条命令先连跳板机，认证通过之后，在跳板机上再发起一条到 `10.0.0.5` 的连接，然后把两条连接拼接起来。你的客户端看到的是一条到 `10.0.0.5` 的 SSH 连接，实际上中间经过了跳板机。图 8-4 是这个结构。




第二段 TCP 由跳板机代开，两段都是 22 端口；对目标机来说，来源是跳板机的地址。

图 8-4 两跳连接，第二条连接由本机驱动，经跳板机转发

这里有个关键细节，第二条连接的认证是由你的本机完成的，只是数据包经过跳板机转发。跳板机上不需要放任何私钥，也不需要开启 Agent Forwarding。这是它比 `ssh -A jump` 再手动 `ssh` 更安全的地方。

`ProxyJump` 可以串联多级，用逗号分隔。

```
ssh -J alice@jump1,alice@jump2 alice@target
```

配置文件里的写法更清爽。

```
Host jump
    HostName jump.example.com
    User alice

Host lab
    HostName 10.0.0.5
    User alice
    ProxyJump jump
```

老一些的环境可能还在用 `ProxyCommand`，效果类似，写法更底层。

```
Host lab
    HostName 10.0.0.5
    ProxyCommand ssh -W %h:%p jump
```

`-W` 表示把标准输入输出接到目标地址的指定端口上，`%h` 和 `%p` 会被替换成主机名和端口。`ProxyJump` 本质上就是这个命令的封装，从 OpenSSH 7.3 起可用，现在应该优先用它。

## 8.6 转发与堡垒机的关系

第八章讲的这些技巧，在堡垒机环境里会遇到额外的限制，这不是偶然。

堡垒机的核心价值是审计。它需要知道你连了哪台机器、敲了什么命令、传了什么文件。如果允许端口转发，你就可以把一条数据库连接塞进 SSH 通道，堡垒机只能看到一串加密的字节流，看不到你执行了什么 SQL。同理，动态转发可以让浏览器访问内网任意网站，堡垒机完全无法审计。所以多数堡垒机的默认策略是禁止端口转发，JumpServer 在资产授权里就有“允许开启 SSH 端口转发”这样的开关，默认关闭。

这也解释了为什么用堡垒机时，一些平时很顺手的做法会失效。VS Code 的 Remote-SSH 需要建立端口转发才能工作，如果堡垒机禁用转发，连接会在某个阶段卡住或者报错。遇到这种情况，正确做法是让管理员在授权里打开对应的权限，或者在堡垒机上使用专门为图形化与 IDE 场景设计的接入方式。绕过限制自己搭隧道，既违反策略，也会让审计记录出现空洞。

## 8.7 让隧道稳定下来

隧道最容易出的问题是莫名其妙地断开。原因通常是中间的网络设备认为连接闲置太久，把 NAT 表项清掉了。表现是终端卡住，过一会儿报 `Connection reset by peer` 或者 `Broken pipe`。

解决办法是定期发心跳。客户端侧用 `ServerAliveInterval`，服务端侧用 `ClientAliveInterval`。

```
Host *
    ServerAliveInterval 30
    ServerAliveCountMax 3
    TCPKeepAlive yes
```

`ServerAliveInterval 30` 表示每 30 秒发一个加密的空包，`ServerAliveCountMax 3` 表示连续 3 次没有回应才断开。加了这两个选项，闲置的隧道能撑很久。

另一个常见问题是转发端口被占用。本地 8080 已经被别的程序用了，`-L` 会报 `bind: Address already in use`。换一个端口，或者用 `-L 127.0.0.1:0:...` 让系统随机分配。后者在脚本里很有用，配合 `-v` 能看到实际分配的端口号。

还有 `ExitOnForwardFailure` 这个选项值得打开。默认情况下，如果转发建立失败，SSH 仍然会登录成功，只是隧道没起来，你会以为连上了。打开之后，转发失败就整体失败，脚本里更容易发现。

```
ssh -o ExitOnForwardFailure=yes -L 8080:10.0.0.9:80 jump
```

## 8.8 一条通道能装多少东西

回到连接层。一条 SSH 连接里可以开多个通道，每个通道独立流控。本地转发的每个 `-L` 是一个通道，远程转发是另一种通道，交互式 shell 也是一个通道。它们的数量受限于服务端的 `MaxSessions`，默认是 10。

```
ssh -L 8080:10.0.0.9:80 -L 3306:10.0.0.8:3306 -L 6379:10.0.0.7:6379 jump
```

这一条命令同时开了三个转发，加上登录 shell 本身用掉的通道，还没到上限。VS Code 的远程开发会开更多通道，文件服务、终端、扩展通信各占一路，所以用 IDE 连接时偶尔会碰到 `open failed: administratively prohibited: open failed` 这类报错，含义是通道数超了，调大 `MaxSessions` 即可。

理解了通道复用，也就能理解为什么一条 SSH 连接可以很“重”。它不只是一个终端，而是一条可以承载多种用途的管道。下一章换一个角度。不再一条一条地修通道，而是换掉整台机器的路由，把一整张内网搬到身边。

第 9 章

# VPN：把一整张内网搬到身边

## 9.1 一条走不通的地址

公司内网的代码托管平台在 `10.0.0.9` 上。白天在工位，浏览器里敲这个地址，页面立刻出来。晚上回家，同一台笔记本、同一个浏览器、同一个地址，转圈转到超时。

这时候最容易得出的结论是“服务器挂了”。可第二天到公司，它好端端的。

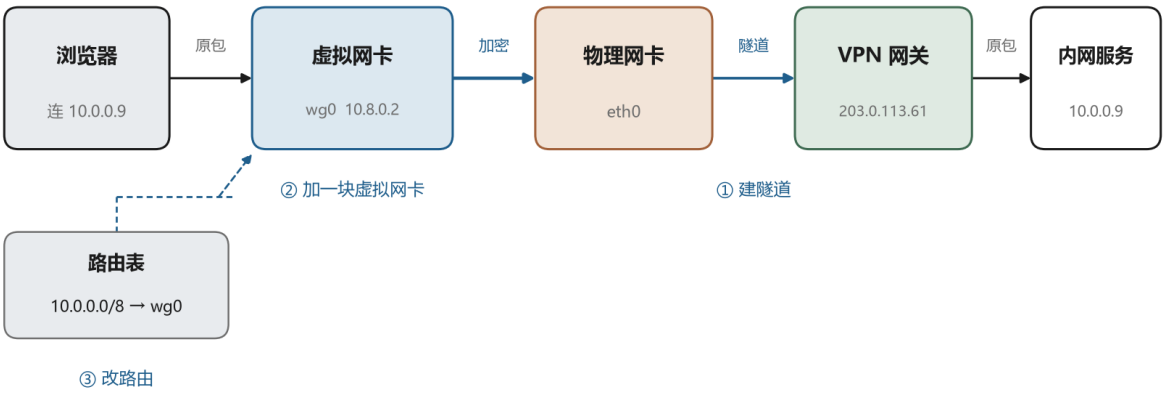
问题不在服务器，在你和 `10.0.0.9` 之间没有路。第三章讲过，以 10 开头的地址是私有地址，只在一个内网里有意义。你家路由器分给你的是 `192.168.1.23`，它知道怎么去公网，也知道怎么去自己那几台设备，但它不知道 `10.0.0.9` 在哪，也没有理由替你把包送过去。运营商那边更不会，因为全世界有几百万个内网都在用这个地址，它不属于任何一个人。

于是有了一个朴素的想法。能不能在回家之前，先在公司的内网里占一个位置，让家里的机器看起来就坐在那个网段里？地址不用改，程序不用改，连 DNS 都不用动。

VPN 做的就是这件事。全称是虚拟专用网（Virtual Private Network），关键词是“虚拟”两个字。物理上你在家，逻辑上你在这张内网里。第八章的端口转发也解决过类似的问题，但它一次只开一个端口，一个服务一条规则。VPN 想做的是一步到位，把整张网搬过来。

## 9.2 三个动作：隧道、虚拟网卡、路由

配好一个 VPN，机器上实际发生了三件事。图 9-1 画的是这三件事合起来的样子。




程序什么都没改：它仍然在连 10.0.0.9，包是操作系统和客户端替它改道的。

图 9-1 VPN 客户端在本机做的三件事

第一件是建隧道。客户端先连到公司出口的 VPN 网关，双方认证，协商出一套加密参数，然后在这条连接上封装数据。这一段的握手过程和 SSH 有相似之处，都是先证明身份，再谈密钥，只不过参与者和协议不同。

第二件是加一块虚拟网卡。系统里多出一个网络接口，名字可能叫 `wg0`、`tun0`，也可能是“以太网适配器 VPN”。它没有物理形态，驱动是软件模拟的。网关会给它分配一个内网地址，比如 `10.8.0.2`，从此你的机器在这张内网里有了合法身份。

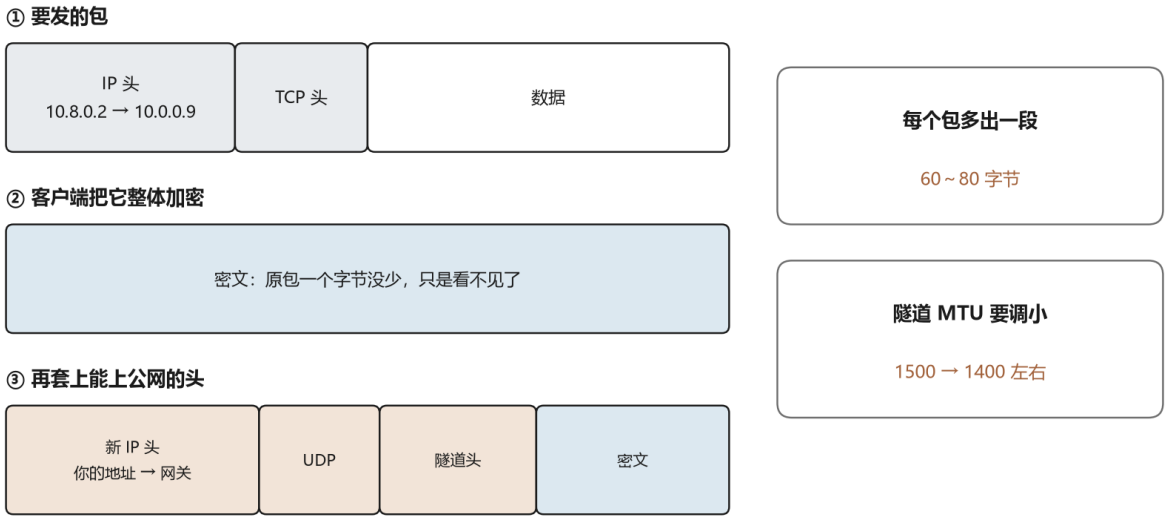
第三件是改路由。这一步最容易被忽略，却是决定“流量走不走 VPN”的唯一开关。操作系统的路由表里会多出一条记录，大意是“凡是发往 `10.0.0.0/8` 的包，从 `wg0` 出去”。以前这类包交给家里的路由器，现在交给虚拟网卡；虚拟网卡收下之后，交给客户端进程加密，再通过物理网卡发往网关。

这三件事合起来，效果就是“程序完全不知情”。浏览器仍然连 `10.0.0.9`，它不知道这个包先被包了一层、绕了一圈、最后在公司的网关上被拆开。这一点是 VPN 和端口转发最根本的区别：端口转发是程序配合，把目标地址改成 `127.0.0.1:8080`；VPN 是操作系统配合，程序什么都不用改。

也正因为改的是路由，VPN 装好之后出问题时，第一件要看的就是路由表，而不是客户端界面上的那个绿灯。

## 9.3 封装：包外面再套一个包

隧道这个词听起来玄，机制其实很直白：把一个完整的包，塞进另一个包里。图 9-2 是这个过程的拆解。




同一份数据，越往外走头越多；中间设备只看得见最外面那一层。

图 9-2 原包被整体加密，再套上一层新的头部

你本来要发的是这样一个包：源地址 `10.8.0.2`，目的地址 `10.0.0.9`，里面装着一个 TCP 段。这个包没法直接扔到公网上，因为公网不认识 `10.8.0.2`，也不认识 `10.0.0.9`。客户端于是把它整体加密，当成一段数据，再加上隧道协议自己的头部，最后加上一个公网能路由的新 IP 头：源地址是你家的公网地址，目的地址是公司 VPN 网关。

到了网关，这个过程反过来走一遍。剥掉外层，解密，得到原来那个包，再按内网的规则送

出去。对 `10.0.0.9` 来说，这个包就是从 `10.8.0.2` 发来的，和你坐在工位上没有区别。

封装带来三个后果，都得知道。

一是开销。每个包外面要多套几十个字节，IP 头 20 字节，UDP 头 8 字节，隧道协议自己的头部和认证标签加起来又是几十字节。千兆链路上这点开销无所谓，但在本来就拥堵的链路上，它会让有效带宽再降一点。

二是 MTU 变小。以太网的默认 MTU 是 1500 字节，套上一层之后装不下这么多，常见的做法是把隧道内的 MTU 降到 1400 到 1420。这一步如果没配好，就会出现一种很迷惑的现象：`ping` 通，`ssh` 也能登录，但一传大文件就卡住，网页加载到一半停住。原因是大包被打了“不许分片”的标记，中途被丢掉，而小包一路畅通。第三章讲过这个机制，隧道是它最常见的触发场景。

三是可识别性。中间设备看不到你在传什么，但能看到你在和同一个地址、同一个端口持续通信，流量特征很明显。所以有些网络会直接封掉常见的 VPN 端口，这也是为什么不少 VPN 要伪装成 443 端口的 HTTPS 流量。

## 9.4 隧道两端的互相信任

隧道建起来之前，双方要先解决信任问题：网关凭什么相信连上来的是公司员工，客户端凭什么相信对面是公司的网关而不是冒充者。这一步的思路和 SSH 完全一致，先认证，再协商密钥，只是具体协议不同。

企业里常见的有三类做法。

第一类是 IPsec，全称是 IP 安全（IP Security）。它工作在网络层，由操作系统或者专门的网关设备实现，认证与密钥协商交给 IKE 协议，数据用 ESP（一种把整个 IP 包重新打包加密的格式）封装。它的长处是站点到站点：两个办公室之间拉一条常驻隧道，两边内网互相可见，员工什么都不用装。缺点是配置复杂，而且要处理各种中间设备的干扰，比如 NAT 穿越。

第二类是 SSL VPN，名字里的 SSL 是 TLS 的旧称，所以它就是跑在 HTTPS 那套协议之上的 VPN。端口通常是 443，和你访问网站用的是同一层协议。因为 443 几乎不会被封，它的穿透性最好；早期形态是网页门户，登录后在页面上点开内部系统的链接，后来演化出轻客户端，可以把指定网段也塞进隧道。

第三类是 OpenVPN 和 WireGuard，两个具体的 VPN 软件。它们不依赖上面两套标准，各自定义了协议。OpenVPN 用 TLS 做握手，跑在 UDP 1194 上，跨平台成熟；WireGuard 更年轻，握手基于 Noise 框架，算法组合固定，代码量小到可以通读，配置只有几十行，近几年被大量采用。

无论哪一类，都要记住一件事：隧道只保护“你家到公司网关”这一段。出了网关，流量就在内网里跑，安全等级取决于内网自己的策略。所以“连了 VPN 就安全了”是个误解，它做的事情是把信任边界从公网挪到了公司出口，边界里面的世界并没有因此变成保险箱。

## 9.5 全隧道与拆分隧道

路由表怎么写，决定了哪些流量走隧道。这是 VPN 配置里最有存在感的一个选择，图 9-3 画的是两种极端。

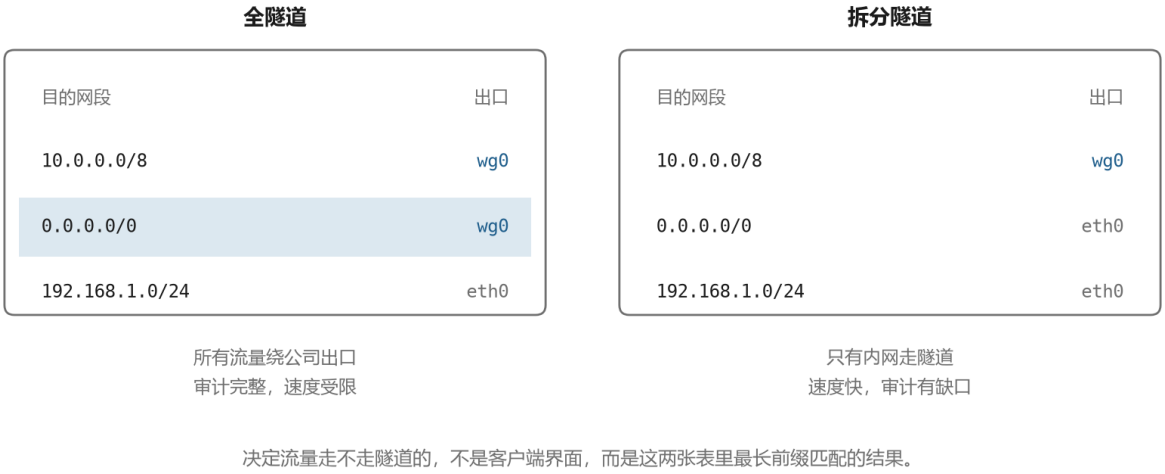



图 9-3 全隧道与拆分隧道的路由表差别

第一种叫全隧道。路由表里加一条默认路由，`0.0.0.0/0` 全部指向虚拟网卡，意思是“除了本地网段，所有流量都进隧道”。效果是你的出口 IP 变成公司的出口 IP，访问任何网站都经过公司的防火墙和审计设备，公司能看到的和你坐在工位上时一样多。

第二种叫拆分隧道。路由表里只加几条具体的：`10.0.0.0/8` 走隧道，`0.0.0.0/0` 仍然走家里的路由器。访问内网系统走隧道，看视频、下载更新走本地宽带，互不干扰。

两种做法各有代价。全隧道的好处是行为统一、审计完整、策略简单，坏处是所有流量绕行，公司出口的带宽成了瓶颈，你在家开个视频会议都要经过公司。拆分隧道快，但审计上出现缺口：员工可以一边连着 VPN，一边用本地网络访问外部服务，公司看不见；更隐蔽的是 DNS，如果查询仍然发给家里运营商的 DNS，访问了哪些域名就暴露在外面，业内管这个叫 DNS 泄漏。

判断标准不在技术，在合规。要求“所有访问都留痕”的环境，只能用全隧道；只是要访问几个内部系统的场景，拆分隧道更实用。这也解释了一个常见现象：同一家公司的 VPN，在不同部门发的配置不一样，因为约束不同。

## 9.6 连不上时按这个顺序查

VPN 的故障排查比 SSH 简单，因为它的环节少，而且每一环都能单独验证。按下面的顺序走，多数问题在第三步之前就能定位。

先看隧道有没有起来。客户端界面上的状态只是参考，真正可靠的是命令。WireGuard 用 `wg show`，会列出对端、最近一次握手时间、收发字节数。最近握手是几分钟前，说明隧道活着；显示“从未握手”，说明连网关这一步就没成，问题在地址、端口、防火墙或者认证。

再看有没有拿到虚拟地址。`ip addr` 或者 Windows 的 `ipconfig`，看那块虚拟网卡上有没有

`10.8.0.2` 这样的地址。没有地址，说明认证过了但地址分配失败，通常和网关的地址池或者策略有关。

第三步看路由指向谁，这一步最容易出问题。

```
ip route get 10.0.0.9
```

这条命令会告诉你，发往 `10.0.0.9` 的包实际会从哪块网卡出去。如果显示的是 `eth0` 而不是 `wg0`，说明路由没生效，或者被一条更精确的路由抢走了。最常见的两种抢法：一是你家路由器的网段恰好也是 `10.0.0.0/8` 里的，本地直连路由的优先级更高；二是你同时连着公司的有线网和 VPN，两条路都通内网，系统选了先来的那条。

第四步才轮到内网。能 `ping` 通网关但 `ping` 不通目标机，说明隧道没问题，是内网的 ACL（访问控制列表，一串按顺序匹配的放行与拒绝规则）、防火墙或者目标机自己的规则拦住了。这时候要看的是公司那边的策略，不是你的客户端。

最后一种情况是“小包通、大包不通”。`ping` 目标机通，`ssh` 登录卡在认证之后，`scp` 一传就断。这是 MTU 问题，用带“不许分片”标记的包试探：

```
ping -M do -s 1400 10.0.0.9
```

从 1400 往下试，找到刚好能通的那个值，把隧道 MTU 设成它。Windows 上的写法是 `ping -f -l 1400 10.0.0.9`，`-f` 表示不许分片，`-l` 是包大小。这类问题不解决，其他一切看起来都正常，只有大文件传不动，很容易被误判成网络不稳定。

## 9.7 校园网里的那个客户端

说一个几乎每个上过大学的人都遇到过的场景。宿舍插上网线，浏览器打不开任何网页，装了学校发的那个客户端，输入学号密码，网才通。客户端图标常驻在托盘里，关掉就掉线，有时还弹出一句“检测到共享上网”。

很多人把这个客户端叫 VPN。它确实常常带着 VPN 功能，但它为主干的那件事，严格说不是 VPN。

校园网要解决的是另一类问题：谁能接入这张网络。学校里几万个网口，每个网口后面是谁，出了事能不能找到人，费用怎么算，这些都要有答案。技术上有两条常见的路。

第一条是 802.1X，作用在链路层。你插上网线之后，交换机的那个端口处于未授权状态，只放行一种报文，就是认证报文。客户端在这条链路上和交换机对话，交换机把内容转给背后的 RADIUS 服务器（远程认证拨号用户服务，一套集中认证的协议），服务器查账号、查密码、查策略，返回一个结论。通过了，交换机把端口打开，必要时还会给你分配一个特定的 VLAN。图 9-4 是这个过程的时序。

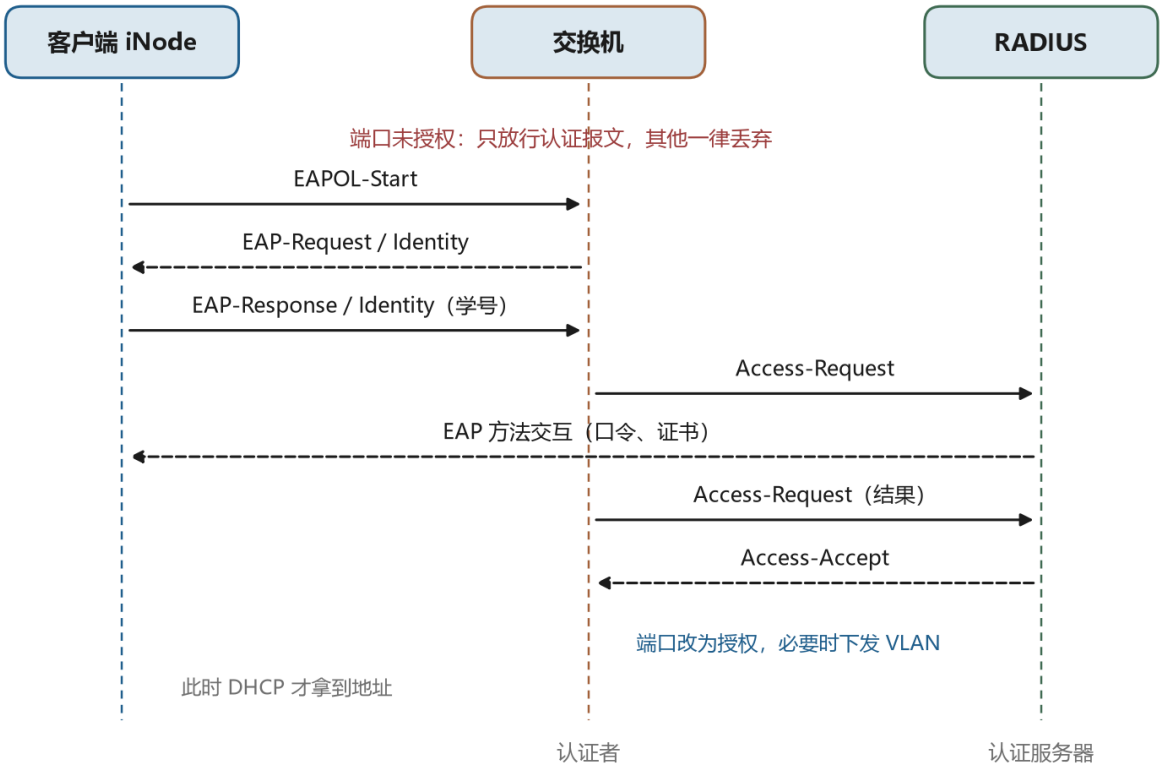



图 9-4 802.1X 认证：交换机在中间转话，RADIUS 做判断

第二条是 Portal 认证，也就是用网页代替客户端的那种登录，作用在网络层。你先正常拿到 IP 地址，但网关只放行 DNS 和认证服务器，你打开任何网页都会被重定向到一个登录页。输入学号密码，网关记住你的地址，把你加入放行名单，有效期到了再认证一次。这条路的优点是浏览器就能用，不必装客户端；缺点是掉线后你未必知道，得重新打开网页。

iNode 这类客户端（H3C 出的一套接入软件，国内高校里最常见）把它们装在了一起，还顺手带上了 SSL VPN 和 IPsec VPN 的拨号功能。所以一个图标背后其实是四套东西：接入认证、门户认证、远程拨号、策略检查。学生把它笼统地叫作“校园网客户端”，厂商管它叫智能客户端，名字里没有一个词提到它到底在做什么，混淆就是这么来的。

学校为什么要设这么多限制，答案在管理上。一人一号，才能计费；账号绑定身份，才能在你访问了不该访问的东西之后找到人；地址和账号绑定，才能防止一个账号拖一整个宿舍的路由器。于是客户端要绑 MAC 地址、绑 IP、定期发心跳、检测多网卡和代理软件，一旦发现共享就强制下线。这些要求和你家里的网络体验是冲突的，所以这类客户端总显得难用，而它难用的每一点，背后都对应一条管理规定。

把两件事分开看，就清楚了。VPN 回答的是“我不在这张网里，怎么进去”；802.1X 和 Portal 回答的是“我就插在这张网的网口上，凭什么让你进来”。前者是在网络之上再架一层，后者是网络本身的闸门。它们经常叠在一起用：在校外先拨 VPN 回到学校，进来之后仍然要过接入认证。同一个客户端把两件事都做了，人就容易以为它们是一件事。

## 9.8 它和端口转发、堡垒机各管什么

到这里，书里已经出现了三种“把远方的资源搬到手边”的办法。它们经常被拿来互相替代，其实回答的是三个不同的问题。表 9-1 是一个对照。

表 9-1 三种做法的分工

| 做法 | 它管的问题 | 典型场景 | 主要代价 |
| --- | --- | --- | --- |
| SSH 转发 | 一个端口怎么过去 | 临时开一个数据库端口 | 逐条加规则，换机重配 |
| VPN | 一整张网怎么过去 | 出差接入内网 | 全隧道时全部绕行 |
| 堡垒机 | 谁连哪台、做了什么 | 多人运维、需要审计 | 单点故障、凭据集中 |

这三者可以层层叠加，而且企业里通常就是这么用的。常见的一种是：员工在家先拨 VPN 进内网，再从内网连堡垒机，最后由堡垒机连到目标机。这样网络上多了一层边界，权限上又收拢了一次。另一种是把堡垒机直接暴露在公网，不要求先拨 VPN，适合需要让外部供应商临时接入的场景，代价是那台堡垒机本身必须做得非常结实。

这三种做法共同的思路是“把边界画在某个地方，然后守住它”。近几年流行起来的零信任接入，思路是把这个边界打散：不再一次性把整张网交给你，而是每访问一个应用就验证一次，只开一条到那个应用的通道。听起来很新，做起来其实就是把第八章的端口转发产品化、策略化，再配上一套身份系统。理解了隧道和转发，零信任的文档读起来就不难了。

第二篇到此结束。这一篇讲的是信任和通道：怎么确认对面是谁，怎么把数据加密着送过去，怎么把一条通道、一张网搬到手边。接下来的问题不再是技术上的“怎么连”，而是组织上的“让谁连”。当几十个人、几百台机器都需要这样一条通道时，凭据放在谁手里、操作记录留给谁看，就成了另一件必须专门解决的事。下一章从一场事故讲起。

第 三 篇

# 从一个人到一群人

一个人管三台机器，密钥放在自己电脑上就够了。三十个人管三百台机器，事情就变了样。

谁有哪台机器的权限，说不清。谁在什么时候登录过，查不到。同事离职三个月，他的公钥还躺在某台机器的 `authorized_keys` 里。生产库被误删，日志里只有一条 `root` 的操作记录，没人知道是谁。

这一篇讲的是组织层面的技术。堡垒机把分散的信任收拢到一个节点，用统一的认证、授权和审计来管理。VS Code 的远程开发则是另一个方向，它把开发环境搬到远端，让代码和数据待在一起，只把界面留在手边。

这两件事看起来不相关，底下的机制却是同一套。堡垒机的 SSH 网关在代理你的连接，VS Code 的远程扩展在复用你的 SSH 连接。理解了前两篇的连接原理，这一篇更多是在看它们怎么被组合、被限制、被记录。

第 10 章

# 为什么需要堡垒机

## 10.1 一个常见的事故

设想这样一件事。测试环境的数据库忽然连不上了，查下来机器还在，端口也在监听，只是数据少了一半。日志里有一条凌晨的 `DELETE` 语句，执行者用的是 `root`。问题是团队里有五个人知道 `root` 的密码，谁也说不清那条语句是谁执行的。有人猜是定时任务，有人猜是误操作，最后不了了之。

这类事情在只有几台机器的小团队里发生概率不高，机器一多就变成常态。人是会流动的，项目是会交接的，三年前配的免密登录还留在某台机器的 `authorized_keys` 里，离职同事的笔记本上可能还存着生产库的私钥。等到出事，你甚至无法回答最基本的问题，谁在什么时候从哪台机器做了什么。

堡垒机就是为了回答这几个问题而出现的一类系统。它把“所有人直接连服务器”改成“所有人先连堡垒机，由堡垒机连服务器”，中间这个节点负责认证、授权、记录。

需要先说清楚，它并不是远程访问的必要环节。SSH 是点对点的协议，客户端直连服务器是最常见也最省事的形态，个人服务器、家庭实验室、几个人的小团队都是这样用的。堡垒机解决的是规模和约束带来的问题，第二章第四节已经给出了判断标准，这里接着讲它具体怎么解决问题。

## 10.2 网状连接的代价

没有堡垒机的时候，连接关系是网状的。图 10-1 左边画的是一个小团队的样子，六台办公终端、八台服务器，理论上每条“人”和“机器”的组合都需要一条通路。

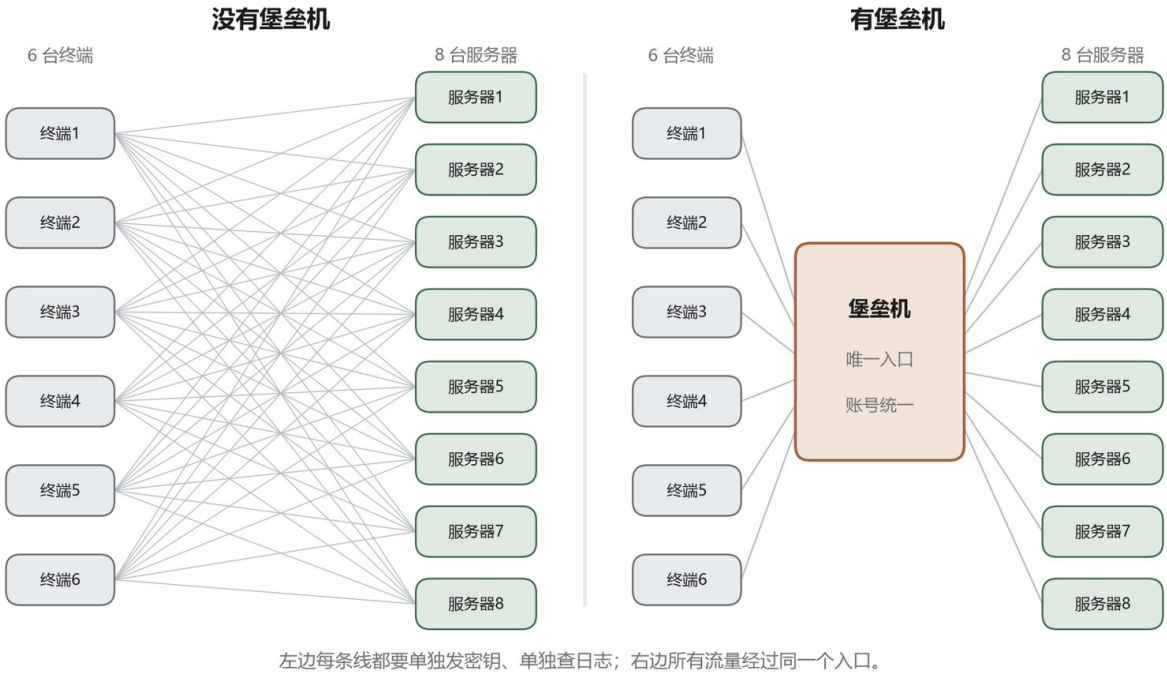



图 10-1 网状连接与星形连接的对比

网状结构带来三个具体问题。

第一个是凭据扩散。每台服务器上都要放每个人的公钥，每个人手里都要有每台服务器的地址和账号。新同事入职，要在一周内把几十台机器的访问配好。同事离职，要在一夜之间把几十台机器上的公钥删干净。漏掉一台，就是一条长期存在的后门。

第二个是权限失控。为了方便，很多人直接用 `root` 登录，或者在 `sudoers` 里给自己开全权限。时间一长，谁有什么权限没人说得清。等到要做权限审计，只能一台台机器去看 `authorized_keys` 和 `sudoers`，工作量巨大。

第三个是没有记录。Linux 的命令历史（`~/.bash_history`）是用户自己维护的，可以删、可以改、可以不写入。多个人共用同一个账号时，历史记录混在一起，无法归属到具体的人。真出了事故，只能靠猜。

改成星形之后，这三个问题都有了着力点，如图 10-1 右边。凭据只在堡垒机上有一份，人员进出只改堡垒机的授权。权限集中在授权表里，可以查询、可以回收。所有会话经过堡垒机，天然可以记录。

## 10.3 4A

堡垒机的功能通常用四个词概括，英文都以 A 开头，所以叫 4A。图 10-2 是这四个能力的示意。

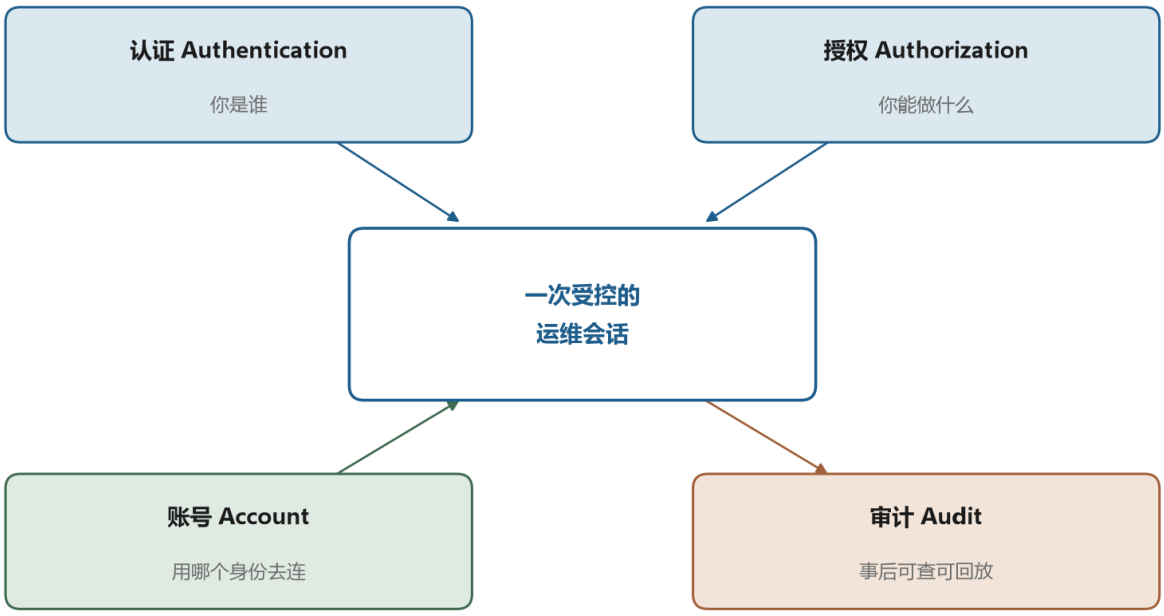


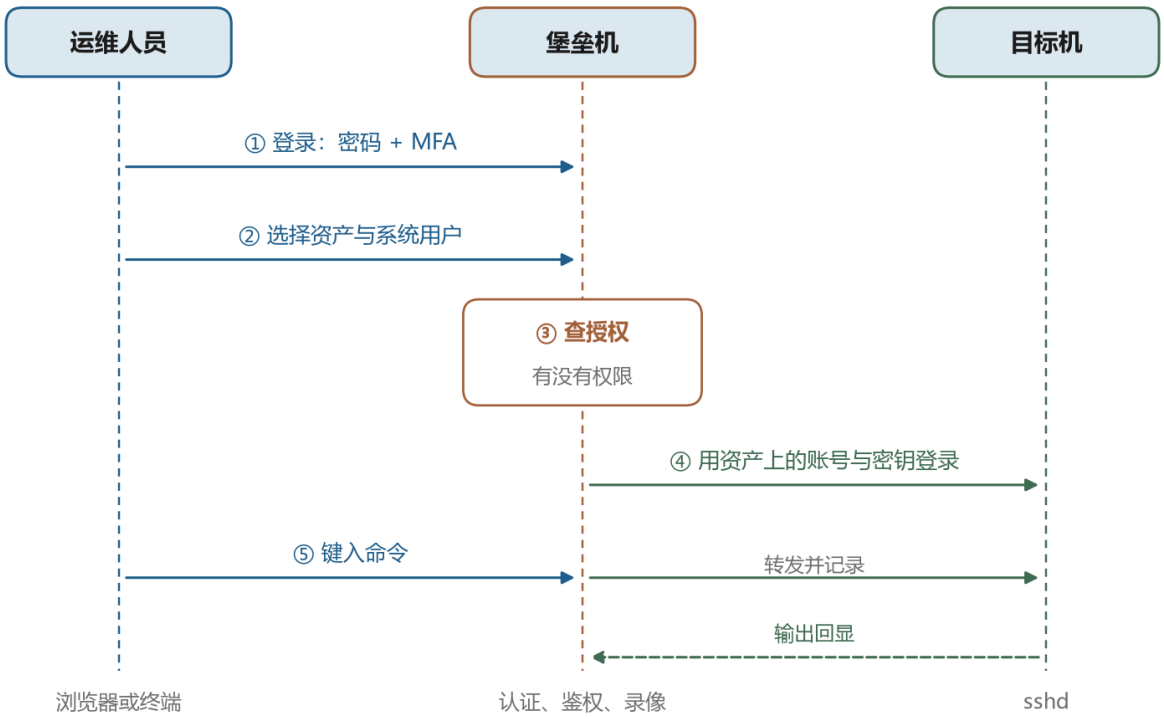

图 10-2 认证、授权、账号、审计构成一个闭环

认证解决“你是谁”。除了密码，还要支持公钥、短信验证码、动态口令、企业微信扫码，重要系统要求双因素。认证的意义在于把操作绑定到一个具体的人，而不是一个共用的账号。

授权解决“你能做什么”。给某个人分配某几台机器，允许他用某个系统账号登录，允许上传下载但不允许复制粘贴，这些都属于授权的范围。授权应该是可查询、可回收、有有效期的。

账号解决“用什么身份连机器”。这里有两层，一层是堡垒机自己的账号，一层是目标机器上的账号。用户登录堡垒机用自己的账号，堡垒机连目标机时用目标机上的账号。后者的密码或私钥由堡垒机托管，用户看不到。这就是所谓的“密码托管”，也是堡垒机最核心的价值之一。

审计解决“你做了什么”。会话录像、命令记录、文件传输记录、登录日志，都属于审计的范畴。审计数据要防篡改，要有保留期限，要能检索。




会话结束后，录像文件与命令记录一起入库，可按时间检索、回放。

图 10-3 一次受审计的登录过程

图 10-3 把一次登录的过程画了出来。用户先认证到堡垒机，选择要访问的资产，堡垒机检查他有没有对应授权，有的话用托管的凭据连到目标机，同时把整个会话录下来。用户看到的界面和直接连目标机几乎一样，区别是所有操作都留了痕。

## 10.4 流量怎么走

堡垒机接入流量的方式有两种，理解它们的区别对排查问题很重要。

第一种叫网关模式，也叫代理模式。用户直接用 SSH 客户端连堡垒机的某个端口，堡垒机在中间做协议代理。用户的连接终止在堡垒机上，堡垒机再向目标机发起一条新连接。这种方式下，堡垒机能完整解析 SSH 协议，所以能做到命令级的过滤和记录。代价是兼容性，某些客户端特性可能不支持，比如复杂的端口转发。

第二种叫直连模式，用户先通过 Web 界面拿到一张“通行证”，再直连目标机，堡垒机只做认证和记录。这种方式对网络要求低，性能好，但记录能力弱，因为流量不经过堡垒机。

多数企业用网关模式，因为它能提供完整的审计。JumpServer 的 Koko 组件就是干这个的，第十一章会详细讲。

这里有一个网络设计上的细节。网关模式下，堡垒机需要能够访问所有目标资产。它通常部署在运维区，同时能通向外网和内网。如果堡垒机到某台资产不通，用户会看到“连接超时”，而这条链路上其实有两段，用户到堡垒机、堡垒机到资产。排查时要分清是哪一段断了，方法是在堡垒机上直接 `nc -vz 资产IP 端口` 试一下。

## 10.5 合规要求

堡垒机在国内的普及和等级保护制度有直接关系。等保指的是信息安全等级保护，国内对信息系统的强制分级要求，2.0 是现行的版本，三级是其中较高的一档。它对运维审计提出了明确要求，包括对远程运维进行身份鉴别、访问控制、操作审计，审计记录要包含日期时间、用户、事件类型、结果，并且要保护记录不被删除、修改。

另一条常被引用的是网络安全法的要求，网络日志留存不少于六个月。这条规定直接决定了堡垒机的存储设计，录像文件要能存半年，容量规划必须按这个来算。

合规要求解释了一些看起来不合理的限制。比如为什么不能直接把生产库的公网端口开出去，为什么运维必须走堡垒机而不能用 VPN 直连，为什么堡垒机要限制文件上传下载。这些限制不是为了给工程师添麻烦，而是审计上的硬性要求。

## 10.6 堡垒机保护不了什么

说了这么多好处，也要说清楚边界。堡垒机不是万能的，把它当成安全问题的终极答案会带来新的风险。

它防不住有权限的人做坏事。一个有生产库删除权限的工程师，通过堡垒机删了数据，录像会记录这一切，但数据已经没了。堡垒机的作用是事后追责和事前威慑，不是事前阻止。真正减少这类风险要靠权限最小化和审批流程。

它本身是单点。所有运维流量都经过它，它挂了，所有人都进不去。所以生产环境里的堡垒机通常要做高可用，至少两台，前面挂负载均衡。这一套的成本不低，小团队往往承担不起。

它集中了所有凭据。堡垒机上存着几百台机器的密码和私钥，一旦被攻破，后果比没有堡垒机更严重。这也是为什么堡垒机的部署要求特别高，要单独划分安全域，要限制访问来源，要定期打补丁，数据库要加密。

它可能和生产环境抢带宽。网关模式下所有流量都过堡垒机，传大文件时带宽是瓶颈。有些团队因此给堡垒机单独开了一条链路，或者对文件传输做限速。

理解了这些边界，再看堡垒机的配置项就不会觉得是繁文缛节。限制复制粘贴是因为剪贴板里的内容不会被录像记录；限制文件下载是因为下载走的是另一条通道；要求双因素是因为单靠密码无法确认操作者是不是账号的主人。每一条限制背后都有一个具体的攻击场景。

第 11 章

# JumpServer 的内部构造

## 11.1 组件地图

JumpServer 是国内使用最广的开源堡垒机，用 Python 和 Go 写成，前后端分离，按协议类型拆成多个组件。图 11-1 是它的组件结构。

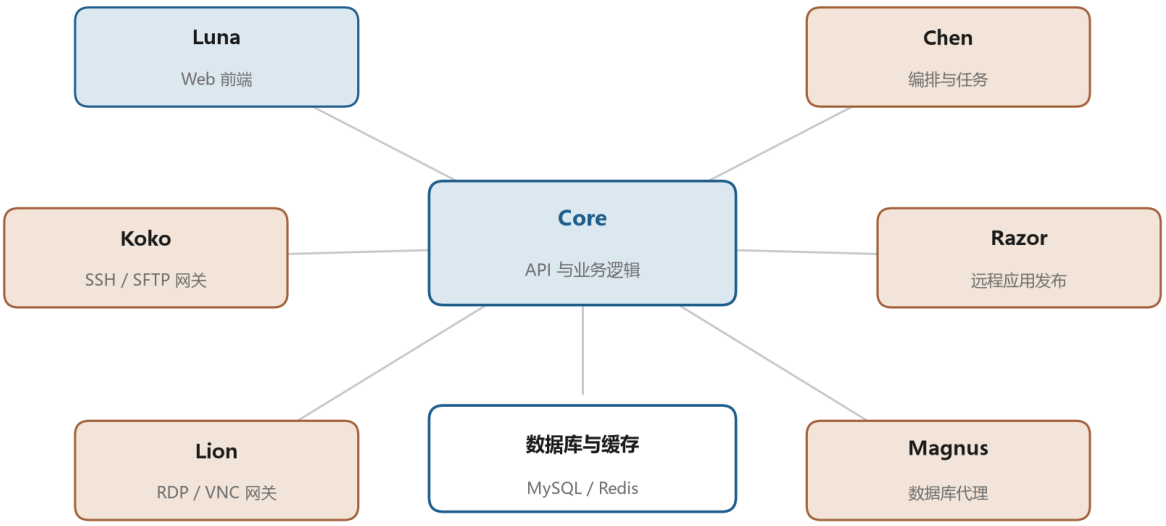



图 11-1 JumpServer 的主要组件与它们的分工

Core 是后端服务，用 Python 的 Django 框架写成，对外提供 API（程序之间调用的接口）。它管理资产、用户、授权和会话记录，所有组件的配置都从这里取，所有操作记录都往这里写。它连接一个关系型数据库存元数据（描述数据的数据，比如资产清单和授权记录），连接 Redis（一种内存里的键值存储）做缓存和消息队列。

Luna 是 Web 前端，用 Vue 写成。你在浏览器里看到的资产列表、会话窗口、录像回放都是它。它自己不碰目标机器，所有操作都通过 API 转给后端或者对应的网关组件。

Koko 是字符协议网关，用 Go 写成，处理 SSH、SFTP、Telnet。用户用 SSH 客户端连堡垒机时，连的就是它。它负责解析 SSH 协议、校验授权、用托管的凭据连目标机、把整个会话录下来。

Lion 是图形协议网关，处理 RDP、VNC。原理类似，只是它代理的是图形协议，录像是一段视频。

Magnus 是数据库代理，把 MySQL、PostgreSQL、Oracle、Redis 这些协议的连接接进来，做账号托管和 SQL 审计。

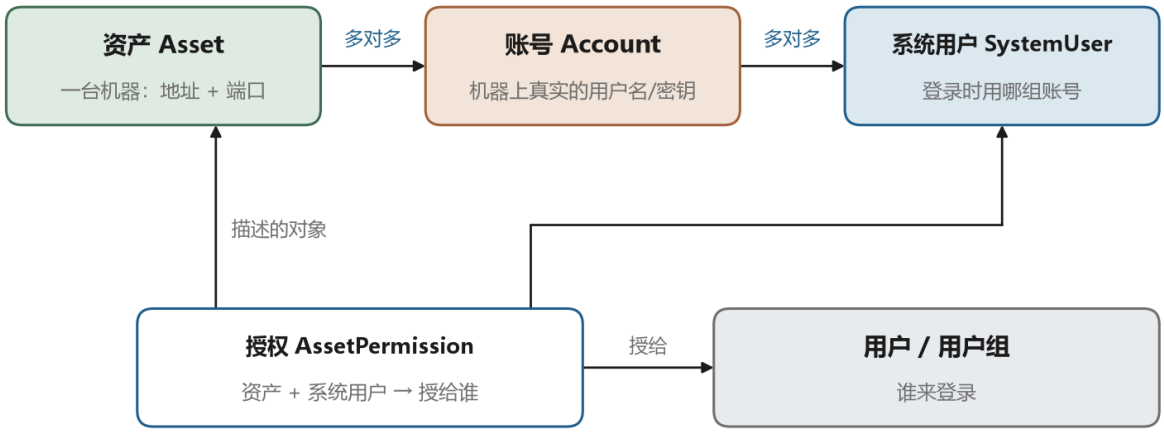
Razor 是远程应用发布，把只能跑在 Windows 上的客户端程序（比如某些管理工具）包装成网页里的一个窗口。

Chen 是任务编排，用来批量执行脚本、做定期巡检，本质上是把 Ansible 包装成了可视化界面。

这套拆分的好处是各协议互不影响，SSH 网关出问题不影响 RDP。缺点是部署复杂，组件多了之后，端口、证书、配置同步都容易出问题。官方提供了一键安装脚本和容器化部署方案，生产环境通常用后者。

## 11.2 资产模型的四层

JumpServer 的权限模型有四个核心概念，理解了它们，界面上的大部分操作就不用猜了。图 11-2 是它们的关系。




多对多：一个账号可以被多个系统用户引用，一个系统用户也可以包多个账号。

系统用户不是某台机器上的账号，而是一组账号的组合；授权把这组账号指定给谁，才决定谁能登进去。

图 11-2 资产、系统用户、账号与授权的关系

资产（Asset）是一台被管理的机器，记录它的地址、端口、平台类型（Linux、Windows、网络设备）、节点归属。节点是一棵树的节点，用来给资产分组，比如“生产环境 / 华东 / 数据库”。

系统用户（SystemUser）是“登录资产时使用的身份模板”。它记录用户名、认证方式（密码或者密钥）、密码或私钥、提权方式（sudo 或者 su）。系统用户分为普通用户和特权用户，特权用户用来做自动化采集，普通用户用来给人登录。

账号（Account）是某个资产上真实存在的账号。系统用户和资产结合，就产生一条账号记录，里面存着这台机器上这个用户的实际密码或密钥。同一个系统用户在不同资产上的密码可以不同，JumpServer 会分别保存。

授权（AssetPermission）是把“谁”和“哪些资产、用哪个系统用户、能做哪些动作”绑在一起的记录。授权的对象可以是用户，也可以是用户组。动作包括连接、上传、下载、复制、粘贴、删除等等，粒度很细。

这四层的关系可以用一句话概括，授权决定某个人能用某个系统用户登录某些资产，登录时用的具体凭据来自账号表。新人入职，管理员做的是建用户、加用户组、给用户组授权，不用碰任何一台真实机器。

## 11.3 三合一用户名

现在可以解释企业环境里常见的那种看起来别扭的配置了。

```
User alice@ubuntu@10.0.0.5
```

JumpServer 的 SSH 网关需要在一开始就知道三件事，谁在登录、用哪个系统用户、连哪台资产。SSH 协议的用户名字段只有一个字符串，于是它把三件事拼在一起，用 at 符号分隔。图 11-3 是这个解析过程。

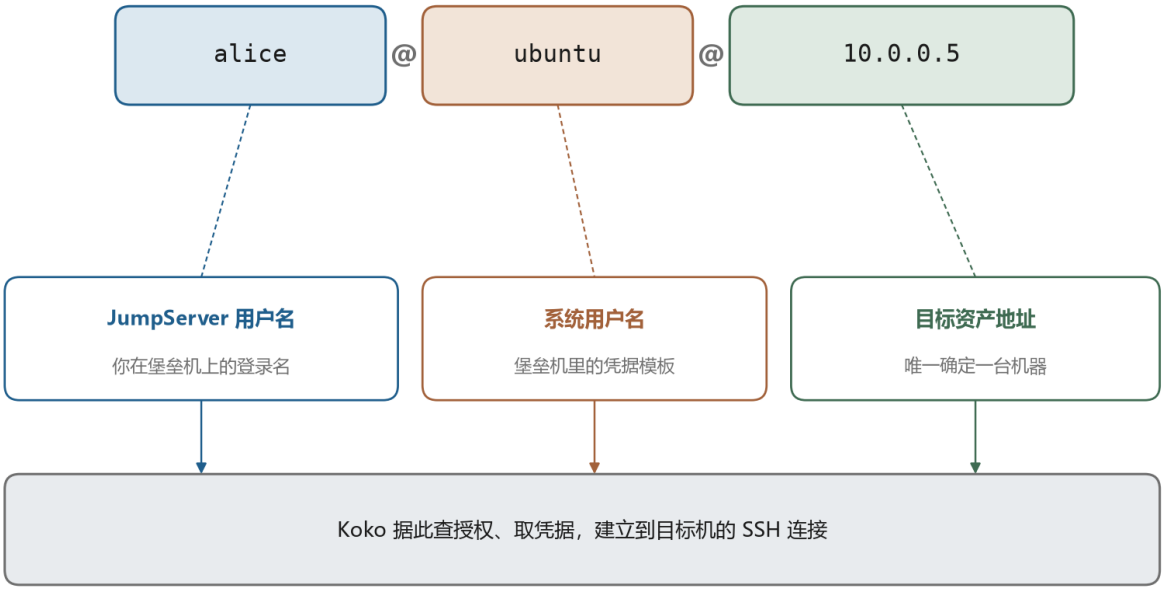



图 11-3 三合一用户名的解析

这个格式只对堡垒机有意义。把同样一串用户名发给一台普通的服务器，它只会当成一个不存在的账号，直接拒绝。协议没有为它留任何位置，是网关在收到之后自己做了解析。

第一段 `alice` 是 JumpServer 上的用户名，用来认证你本人。第二段 `ubuntu` 是系统用户名，对应 JumpServer 里配置的一个系统用户，它决定了用哪套凭据去连目标机。第三段 `10.0.0.5` 是资产地址，Koko 用它来定位具体是哪台机器。

这个格式对 SSH 客户端来说是合法的，因为用户名只是一个字符串，协议没有规定不能包含 at 符号。但它在两个地方会带来麻烦。一是有些工具会按第一个 at 符号切分用户名和主机，

导致解析错误。二是命令行里如果涉及 URL 编码，at 符号需要转义成 `%40`。VS Code 的 Remote-SSH 属于前者，所以它的配置文件里要特别小心，不能让它把用户名再拆一次。第十二章会讲它怎么处理。

还要注意一点，第二段是“系统用户名”，不是目标机上真实的 Linux 用户名。系统用户里记录的用户名可能和它同名，也可能不同。你看到 `ubuntu`，不代表目标机上的账号一定叫 `ubuntu`，那只是 JumpServer 里给这套凭据起的名字。这一点在实际排查时很容易搞混。

## 11.4 一次 SSH 网关会话

把上面这些串起来，一次通过网关的登录是这样走的。

你的客户端连到堡垒机的 58222 端口，Koko 接受连接。它先和你完成 SSH 握手，用你上传的公钥或者密码认证你的身份，这一步验证的是“你是 alice”。认证通过后，Koko 解析用户名里的三段信息，去数据库查 alice 有没有权限用 `ubuntu` 这个系统用户连接 `10.0.0.5` 这台资产。有权限的话，它从账号表里取出这台机器上这个账号的凭据，向目标机的 22 端口发起一条新的 SSH 连接。这条连接用的是托管的密码或私钥，你完全看不到。

两条连接都建立之后，Koko 在中间做双向转发，同时把整个会话录下来。你敲的每一个字符、目标机返回的每一段输出，都写进录像文件。录像不是简单的文本记录，而是带时间戳的终端会话流，回放时能看到打字的速度和节奏。

如果授权里配置了命令过滤，Koko 还会在转发之前检查你输入的命令。匹配到禁用规则的命令会被拦下来，你在终端里会看到一段提示，命令不会到达目标机。这个检查和你在目标机上配置的 `sudo` 规则是两回事，前者在堡垒机侧，后者在目标机侧，两者都可能拦住你。

## 11.5 命令过滤与会话录像

图 11-4 是一次会话的时间轴，标出了几种记录发生的位置。

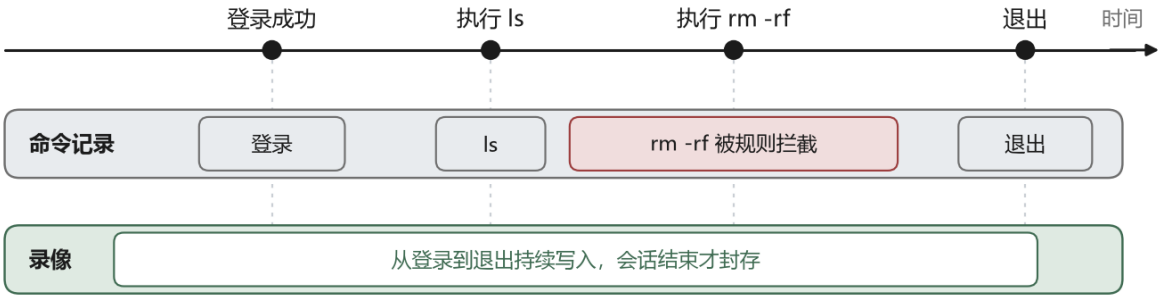



图 11-4 会话过程中命令记录与录像的写入时机

命令记录是结构化的，每执行一条命令就写一条，包含时间、用户、资产、命令内容、执行结果。它适合检索，比如查“上周谁在这台机器上执行过 rm”。但命令记录有局限，它抓的是

你敲进去的字符串，如果命令是通过脚本执行的，或者用了交互式程序（比如 `vi`、`mysql` 客户端），记录就不完整。

会话录像弥补了这个缺口。它记录的是终端上的原始字节流，回放时把这些字节按时间重新播放一遍，看起来就像当时的屏幕。对于 `vi` 里改了什么、`mysql` 里执行了什么 SQL，录像都能还原。

两者结合，审计才算完整。查问题时先看命令记录定位到时间段，再调出录像看具体操作。这也解释了为什么录像文件占空间，一个小时的交互式会话，录像可能有几 MB 到几十 MB，按六个月留存算，容量规划要认真做。

## 11.6 凭据是怎么存的

堡垒机最敏感的数据是托管的密码和私钥。JumpServer 用对称加密保存这些内容，加密密钥放在配置文件或者环境变量里，和数据库分开存放。这样即使数据库被拖走，没有密钥也解不开。

```
SECRET_KEY=...
BOOTSTRAP_TOKEN=...
```

这两个值在安装时生成，写在 `/opt/jumpserver/config/config.txt` 里。很多人图省事，把它们留在默认值或者提交到了代码仓库，这是严重的错误。拿到这两个值加上数据库，等于拿到所有资产的登录权限。

更麻烦的是密钥轮换。改了加密密钥，已经存进去的密码全部解不开，需要重新录入。所以正规做法是在部署初期就规划好密钥管理，用密钥管理服务或者外部保险箱，而不是写死在配置文件里。

堡垒机托管私钥还带来一个架构上的变化。原来每个人手里有自己的私钥，现在私钥集中在堡垒机上。这意味着堡垒机本身成了最高权限的持有者，它的安全等级必须高于它管理的所有资产。这条要求在实际部署中经常被忽略，堡垒机和被管理的机器放在同一个网段、用同一套弱密码，等于把锁和钥匙放在同一个抽屉里。

## 11.7 部署时的几个关键项

JumpServer 的配置文件里有几项值得提前想清楚。

网关端口是其中之一。默认的 SSH 网关端口是 2222，示例里用的是 58222。改端口的意义不大，但能减少扫描噪音。真正重要的是把这个端口只对运维网段开放，不要暴露在公网。

另一个是 `SECURITY_INSECURE_MODE` 之类的开关，用来允许或禁止某些不安全的行为，比如跳过主机密钥校验。这些开关在生产环境必须关闭，否则等于放弃了中间人防护。

还有会话保留策略。录像文件默认存在本地磁盘，容量有限。生产环境要配置成存到对象存储，第五章讲的对象存储在这里派上了用场。这也顺带解释了为什么这本书把对象存储放在最后

一篇，它和堡垒机是同一套运维体系的两端，一端管人怎么进来，一端管数据存在哪里。

第 12 章

# VS Code 远程开发的工作原理

## 12.1 一个编辑器，四个进程

VS Code 看起来是一个程序，实际上是一组进程。打开任务管理器，能看到好几个 Code.exe。这个结构继承自 Electron（一个用网页技术开发桌面程序的框架），也决定了远程开发可能的形态。

最外层是主进程，负责窗口、菜单、文件对话框、更新。中间是渲染进程，每个窗口一个，负责画界面。再往下是扩展宿主进程，所有扩展的代码都跑在这里，一个扩展崩溃不会把界面带崩。还有一个 pty 宿主进程，专门管理终端，让终端里的程序运行在独立进程里。

知道这个结构，就明白远程开发要解决什么问题。你希望代码在远端，编译和运行在远端，但界面和键盘在自己眼前。最直接的做法是把整个 VS Code 搬到远端，界面通过远程桌面显示，但那样延迟高、体验差。VS Code 选择的方案是把一部分进程搬到远端，本机只留界面。

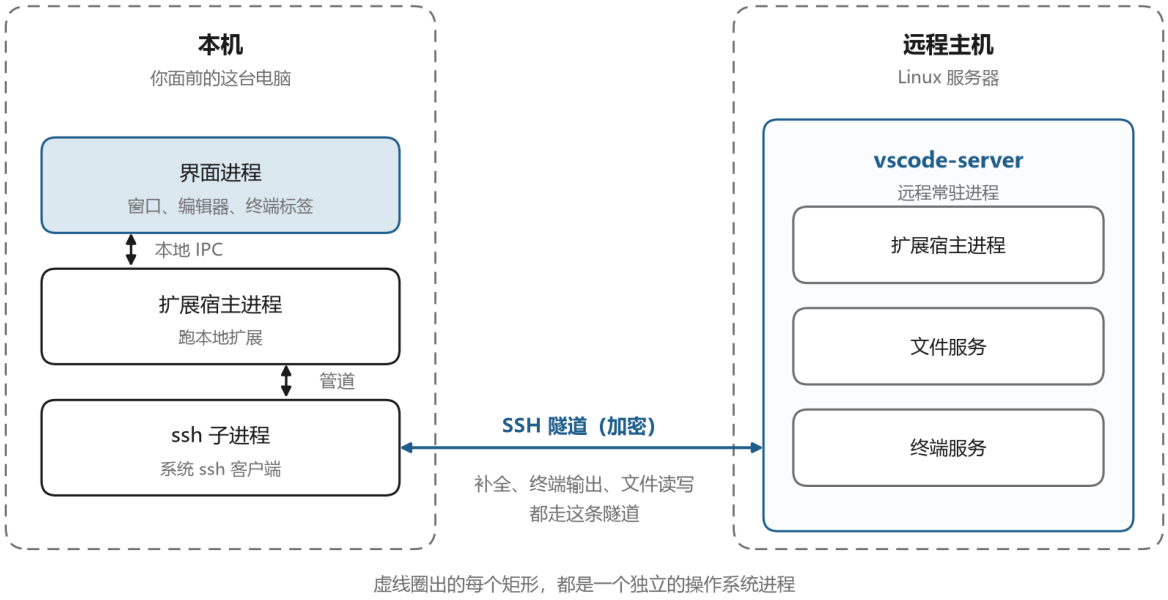


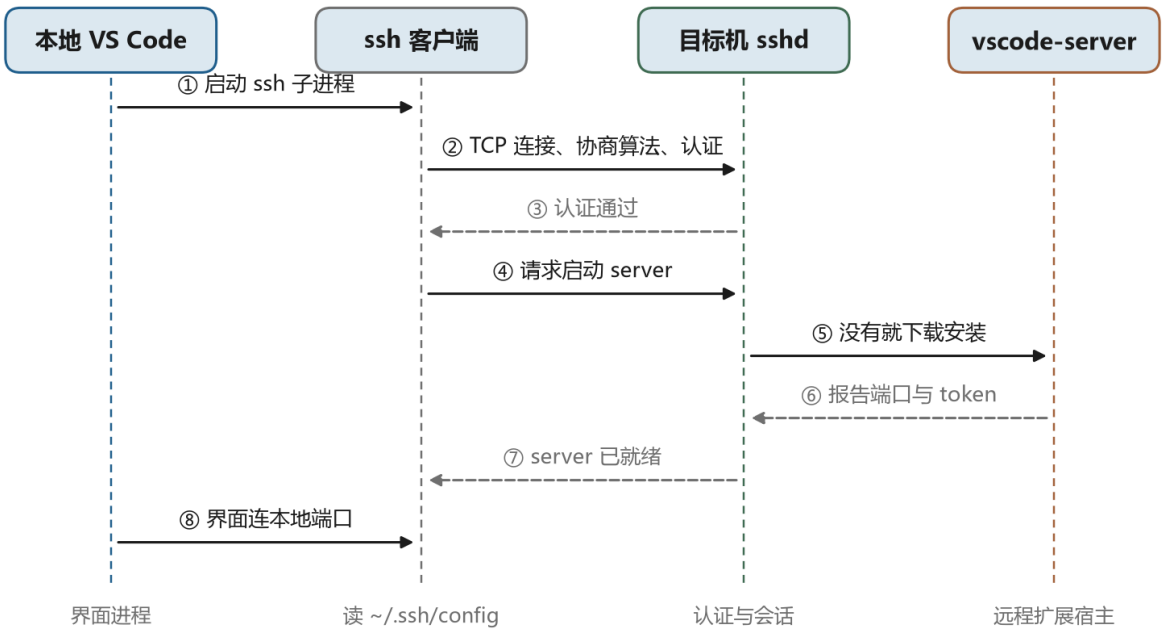

图 12-1 VS Code 远程开发时本机与远端的进程分工

图 12-1 是这个分工。本机保留界面进程和一部分扩展，远端跑一个叫 `vscode-server` 的服务，里面有远端扩展宿主、文件服务、终端服务。两者之间通过一条加密通道通信，通道上跑的是普通的 HTTP 和 WebSocket。

扩展被分成两类。只影响界面的扩展，比如主题、图标、快捷键提示，跑在本机。需要读写文件、调用编译器、访问远端的扩展，比如语言服务、调试器、Jupyter，跑在远端。VS Code 会根据扩展声明的 `extensionKind` 自动决定装在哪边，也可以手工指定。

## 12.2 连接时发生了什么

Remote-SSH 扩展做的事情，可以拆成图 12-2 里的几步。




箭头方向就是发起方；排错时先确定卡在第几步

图 12-2 Remote-SSH 建立连接的过程

你在界面上选一个主机，扩展读 `~/.ssh/config`，把别名展开成具体参数。然后它调用系统的 `ssh` 客户端，也就是你平时在终端里用的那个，建立一条连接。注意这一步用的是系统自带的 ssh，不是自己实现的协议栈，所以你在 `config` 里写的所有选项、你的 `known_hosts`、你的 `ssh-agent`，全都照常生效。

连上之后，扩展在远端执行一小段脚本来检查环境。它检查三件事，`~/.vscode-server` 目录是否存在、里面的版本是否匹配当前 VS Code 的版本、远端能不能正常执行命令。版本不匹配或者目录不存在，它就下载一个对应版本的 server 包，解压到 `~/.vscode-server/bin/<commit-id>/` 下面。

下载这一步是最容易失败的地方。远端机器需要能访问更新服务器，如果它在一个没有外网的机房里，下载就会超时。解决办法有两个，一是在本地下载好安装包，手工传上去解压；二是配置代理，让远端的下载走你的本机。后者的做法是在 `config` 里加 `Remote.SSH.RemoteServerListenOnSocket` 之类的选项，或者更简单地用第八章的本地转发把更新服务器的地址映射过去。

server 起来之后，它会监听一个本地端口或者一个 Unix socket。扩展在本机也启动一个监听，然后通过 SSH 的端口转发把两边接起来。界面通过 `127.0.0.1` 上的这个端口和远端通信，对你来说完全透明。

## 12.3 那条隧道的两端

图 12-3 画的是这条隧道两端的细节。

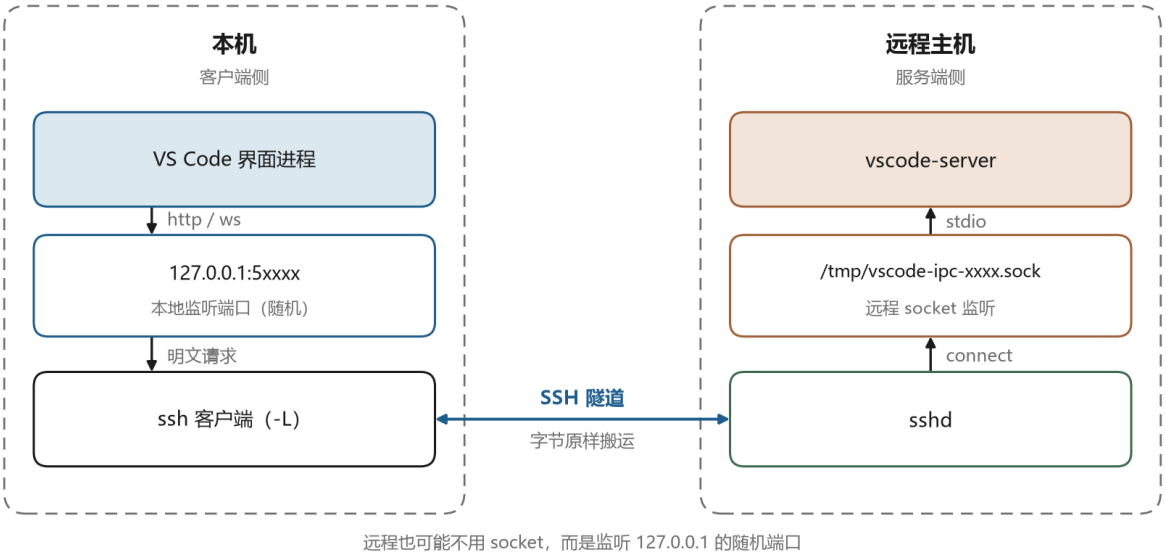



图 12-3 Remote-SSH 的端口转发，本地随机端口接到远端 socket

本机这一端，VS Code 随机选一个空闲端口，通常是 5 万多的一个数字，监听在 `127.0.0.1`。远端那一端，`vscode-server` 监听一个 Unix socket 文件，路径在 `~/.vscode-server/` 下面，或者监听一个 `127.0.0.1` 上的随机端口。中间的转发由 `ssh` 客户端完成，用的是 `-L` 或 `-R` 的等价机制。

这个设计解释了几个现象。第一，你在远端用 `ss -lntp` 会看到 `vscode-server` 监听在回环地址上，端口号是随机的，从外部连不进去。第二，如果你在堡垒机上禁用了端口转发，Remote-SSH 会在建立隧道这一步失败，报错通常出现在输出面板里，而不是终端里。第三，隧道断了之后界面会显示“正在重新连接”，因为 WebSocket 断了，VS Code 会尝试重连并重新建立端口转发。

## 12.4 ssh_config 的解析规则

Remote-SSH 完全依赖系统的 ssh 客户端，所以 `config` 文件的解析规则就是 OpenSSH 的规则，值得花点时间搞清楚。

文件从上往下读，对每一段 `Host`，检查当前连接的主机名（也可能是命令行里给的别名）是否匹配。匹配的段里的选项生效，后面的段可以覆盖前面的同名选项。`Host *` 匹配所有主机，所以它通常放在最前面作为默认值。

```
Host *
    ServerAliveInterval 30

Host lab-*
    User alice
    IdentityFile ~/.ssh/id_ed25519_lab

Host lab-gpu
    HostName 10.0.0.9
    ProxyJump jump
```

上面的例子里，连 `lab-gpu` 时，三段都匹配。`ServerAliveInterval` 来自第一段，`User` 和 `IdentityFile` 来自第二段，`HostName` 和 `ProxyJump` 来自第三段。这就是为什么默认值要写在最前面。

支持的写法还有 `Include`，可以把配置拆成多个文件。

```
Include ~/.ssh/config.d/*
```

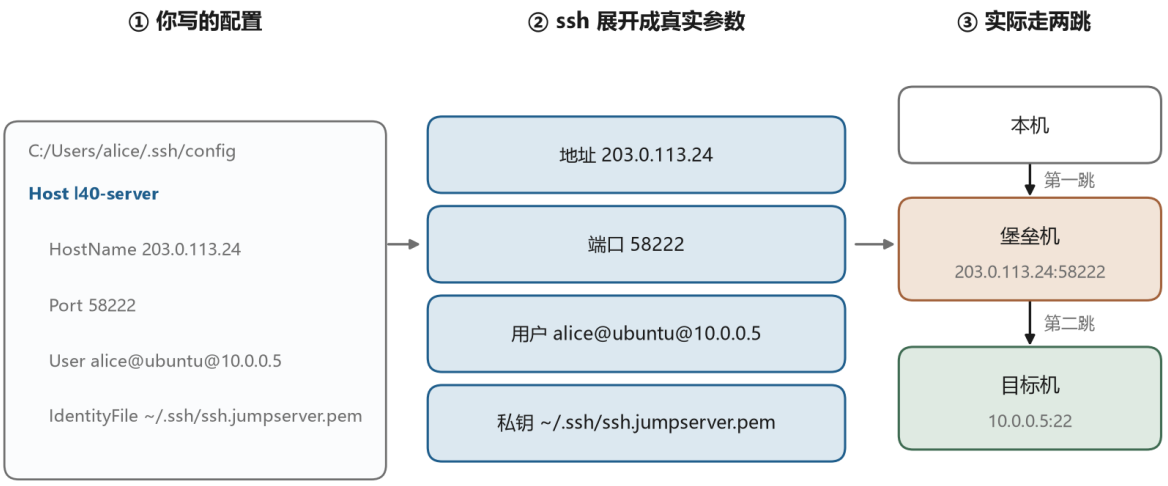
OpenSSH 7.3 起支持，通配符和 `~` 都能用。企业里常用它把不同环境的配置分开管理。

几个替换符号在写 `ProxyCommand` 时会用到。`%h` 是目标主机名，`%p` 是端口，`%r` 是用户名，`%d` 是本机的家目录。写 `ProxyCommand ssh -W %h:%p jump` 时，这些符号会被替换成实际值。

还有两个容易忽略的点。一是 `HostName` 不写时默认等于 `Host` 后面的名字，所以 `Host lab` 加 `HostName 10.0.0.5` 两者都要写清楚，别指望它能猜。二是 `Host` 后面的名字区分大小写，`Lab` 和 `lab` 是两个不同的别名。

## 12.5 逐行拆解那份配置

现在回到那种看起来别扭的配置。图 12-4 把每一行的作用标了出来。




把目标写进 User 字段是堡垒机的约定；用 ProxyJump 则拆成两行，一跳一段

图 12-4 配置文件的每一行在做什么

```
Host l40-server
    HostName 203.0.113.24
    User alice@ubuntu@10.0.0.5
    Port 58222
    IdentityFile C:/Users/alice/.ssh/ssh.jumpserver.pem
```

`Host l40-server` 定义了一个别名，你在 VS Code 里选的就是这个名字，命令行里 `ssh l40-server` 用的也是它。名字本身可以随便起，只要自己记得住。

`HostName 203.0.113.24` 是真正要连接的地址。这里是堡垒机的公网地址，不是你目标机的地址。这一点是整份配置里最容易误解的地方。

`User alice@ubuntu@10.0.0.5` 是三合一用户名，第十一章讲过它的三段含义。SSH 客户端不解析这个字符串，它原样发给服务器，由堡垒机去解析。

`Port 58222` 是堡垒机 SSH 网关的端口。不是 22，因为堡垒机的 22 端口通常留给系统自身的运维，网关用另一个端口。

`IdentityFile` 指向你在堡垒机上注册的公钥对应的私钥。这个私钥认证的是你本人，也就是 `alice` 这个堡垒机账号，不是目标机上的账号。

把五行合起来读，意思是“连接 203.0.113.24 的 58222 端口，用这把私钥证明我是 alice，然后告诉它我要用 ubuntu 这个系统用户去连 10.0.0.5”。一行配置里塞了两次认证、三段信息，难怪第一次看会懵。

这里有一个实际会遇到的问题。VS Code 的 Remote-SSH 在内部会解析用户名，某些版本会把 `User` 按第一个 at 符号切开，导致只把 `alice` 发过去。如果遇到这种版本，解决办法是在 `config` 里用 `User alice@ubuntu@10.0.0.5` 的同时，加上 `Remote.SSH.UseLocalServer` 或者改用 `ProxyCommand` 手工拼参数。具体做法随版本变化，判断依据是输出面板里的日志，看它实际发出的用户名是什么。

## 12.6 出问题时看哪里

Remote-SSH 的错误提示往往很笼统，只说“无法连接”。真正的细节在输出面板里。

打开方式是菜单里的“视图 → 输出”，然后在右侧下拉里选 “Remote - SSH”。这里会打印完整的 ssh 命令行、连接过程、每一步的输出。它其实就是在替你执行一条 `ssh` 命令，把参数都拼好了。

```
Running script with connection command: ssh -T -D 5xxx "l40-server" sh
```

看到这一行，就可以把它复制到终端里手工执行，加上 `-v` 看更详细的输出。这是排查最快的路径，因为你在终端里能拿到完整的调试信息，而输出面板有时候会截断。

几个常见故障和对应的原因。报“Could not establish connection”，先看是不是网络不通或者端口写错。报“Permission denied”，检查私钥路径、私钥权限、堡垒机上的公钥是否配对。卡在“Downloading VS Code Server”，是远端下载失败，检查外网连通性或者手工安装。连上了但界面一直转圈，多半是端口转发被禁用，或者远端的 `vscode-server` 进程僵死。

清理远端残留是另一个常用操作。有时候 `vscode-server` 的进程卡住，删掉目录重来最快。

```
ls ~/.vscode-server/bin/
ps -ef | grep vscode-server
rm -rf ~/.vscode-server
```

VS Code 里还有一个命令叫 “Kill VS Code Server on Host”，效果一样，不用记路径。执行之后重新连接，它会重新下载并启动。

最后提醒一句资源问题。`vscode-server` 会占内存，语言服务和索引在大项目上能吃掉几个 GB。如果在共享的 GPU 机器上开发，注意别让它把内存吃满，导致训练任务被 OOM 杀掉。可以在远端限制扩展数量，只装必需的语言支持。

第 四 篇

# 数据的另一端

终端这一头是人，另一头是机器。机器跑起来之后会产生数据，数据要存下来，要能被下一台机器读到。

几百 GB 的训练集，几十万个日志文件，每天新增的检查点，这些东西放在本地磁盘上不现实。它们最终都去了对象存储。对象存储的接口简单到只有几个 HTTP 方法，可用起来却处处是坑，签名错一个字节就是 403，权限少一条策略就是拒绝访问。

这一篇把对象存储拆开看。先看它的数据模型为什么和文件系统不一样，再看分片上传、元数据、生命周期这些日常会用到的机制，最后把签名算法完整推导一遍。推导的过程有点长，但值得跟一遍，因为它是所有 403 的根源。

最后一章会把四篇的内容串成一条链路，从你按下连接按钮，到训练脚本把模型写回桶里。那条链路上的每一段，你都应该能说清楚它在做什么。

第 13 章

# 对象存储的模型

## 13.1 为什么不用文件夹

服务器上的数据通常放在文件系统里，一个目录套一个目录，文件有名字、有路径、有属性。这套模型用了几十年，很成熟。但当数据量到了几十 TB、文件数到了几千万，文件系统开始吃力。目录树的元数据要放在内存里，几千万个 inode（文件系统里记录单个文件属性的数据结构）会把内存吃光；单机的容量有上限，扩容要停机；跨机器共享要靠 NFS（一种让多台机器挂载同一个目录的协议），性能和稳定性都不理想。

对象存储是为这个场景设计的。它放弃了目录树，放弃了随机写，放弃了文件锁，换来的是近乎无限的容量、很高的持久性和简单的 HTTP 接口。代价是你不能像本地文件那样 `append` 一段内容，也不能 `rename` 一个目录，每次操作都是整体替换一个对象。

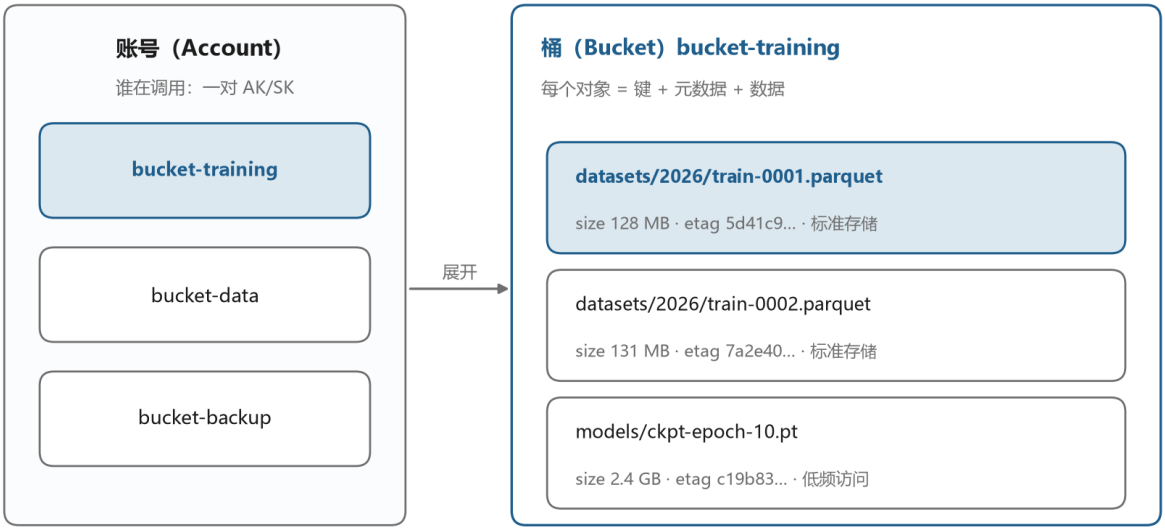



图 13-1 账号、存储桶与对象的层次

图 13-1 是它的数据模型。最上层是账号，账号下面可以建多个存储桶（Bucket），桶里放对象（Object）。对象由三部分组成，一个键（Key）、一段数据、一组元数据。键是一个字符串，可以包含斜杠，看起来像路径，但它不是路径。

```
logs/2026/11/04/app.log
```

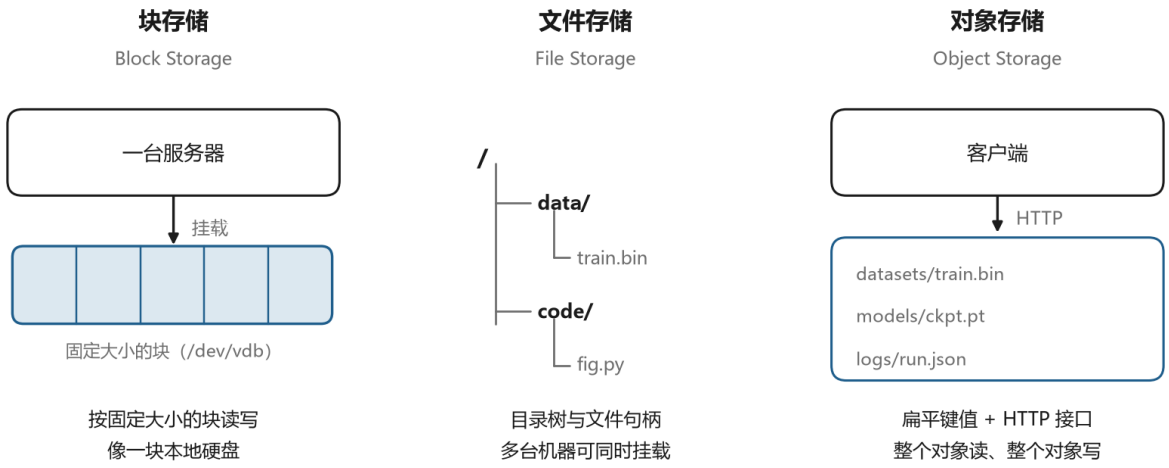
这个键在对象存储里就是一个完整的名字，没有 `logs` 这个目录。控制台上把它显示成目录树，是为了照顾人的习惯，实际存储时它就是一串字符。这个设计带来一个后果，按前缀列举很快，按“目录”操作不存在。想删除一个“目录”，得先列出所有以这个前缀开头的对象，逐个

删。

桶的名字全局唯一，跨账号也不能重复，因为访问时要用它拼域名。腾讯云 COS 的域名形如 `bucket-1250000000.cos.ap-guangzhou.myqcloud.com`，其中前半段是桶名加 APPID，后半段是地域。AWS S3 的域名形如 `bucket-name.s3.region.amazonaws.com`。这个域名规则在排查访问问题时很重要，因为签名计算和域名绑定，写错地域就会报签名不匹配。

## 13.2 三种存储形态

块存储、文件存储、对象存储经常被放在一起比较，它们解决的是不同层次的问题。图 13-2 是这三者的示意。




三种形态对应三种用法：系统盘、共享协作目录、海量非结构化数据

图 13-2 块存储、文件存储与对象存储的区别

块存储提供的是一块裸设备，操作系统在上面建文件系统，用起来和本地磁盘一样。云厂商的云硬盘属于这一类。它的优点是延迟低、支持随机读写，适合数据库。缺点是同一时刻只能挂给一台机器，共享要靠集群文件系统。

文件存储提供的是共享目录，多台机器同时挂载（把远端目录接到本地的一个路径下，用起来像本地目录），看到同一份数据。NFS、SMB（两种共享目录的协议）属于这一类，云上的 NAS（专门提供共享目录的设备）也是。它适合多台机器共享配置、共享代码，缺点是元数据操作多的时候性能下降明显。

对象存储提供的是 HTTP 接口，通过 PUT 和 GET 存取整个对象。它不适合放数据库文件，也不适合放需要频繁修改的文件。它适合放图片、视频、备份、日志、训练数据、模型权重，这些内容写进去之后很少改，但量很大。

判断用哪种存储，可以问三个问题。需要随机读写吗，需要多机共享吗，数据量大到单机放不下吗。数据库用块存储，共享配置用文件存储，海量冷数据用对象存储。

## 13.3 接口就是 HTTP

对象存储的接口是 REST 风格的 HTTP（REST 指用几个固定动作操作资源，比如 GET 取、PUT 放、DELETE 删）。上传一个对象用 PUT，下载用 GET，删除用 DELETE，查元数据用 HEAD。这几个方法都是幂等的，同一个 PUT 执行两次，结果一样，这对重试很友好。

```
curl -X PUT "https://bucket-1250000000.cos.ap-guangzhou.myqcloud.com/test.txt" \
     -H "Authorization: ..." \
     --data-binary "hello"
```

返回的状态码能说明大部分问题。200 表示成功，204 表示成功但没有内容，403 表示没有权限或者签名不对，404 表示对象不存在，409 表示冲突（比如桶名已被占用），412 表示前置条件失败，429 表示请求过于频繁，5xx 表示服务端问题。排错时先看状态码，再看返回的 XML 里的错误码，后者更具体。

```
<?xml version="1.0" encoding="UTF-8"?>
<Error>
  <Code>SignatureDoesNotMatch</Code>
  <Message>The request signature we calculated does not match...</Message>
  <Resource>/test.txt</Resource>
  <RequestId>...</RequestId>
</Error>
```

`Code` 字段是最有价值的信息，第十四章会把常见的几个错误码和它们的成因列出来。

还有一个容易忽略的方法 HEAD。它只返回头部，不返回内容，可以用来快速判断对象是否存在、有多大、是什么类型，不用把整个对象下载下来。

```
curl -I "https://bucket-1250000000.cos.ap-guangzhou.myqcloud.com/big-model.bin"
```

## 13.4 大文件怎么传

对象存储的单个 PUT 有大小限制，通常是 5 GB。超过这个大小必须用分片上传。即使没超，大文件用分片上传也更可靠，因为某一片失败只需要重传那一片。

分片上传分三步，如图 13-3。先发起一个分片上传请求，服务端返回一个 UploadId。然后按顺序或者并发上传各个分片，每片带上序号，服务端返回每片的 ETag（服务端为这段内容算出的校验值，用来确认分片完整）。最后提交完成请求，把所有分片的序号和 ETag 列出来，服务端按顺序拼接成一个对象。




图 13-3 分片上传的三个阶段

这套机制有两个细节值得知道。第一，分片有大小限制，除了最后一片，每片不能小于 1 MB（不同厂商的数字略有差异，S3 是 5 MB），最大不能超过 5 GB。第二，未完成的分片上传会占用存储空间，因为已经上传的分片还在服务端。所以要配置生命周期规则，自动清理超过一定天数还没完成的分片，否则存储费用会莫名其妙地上涨。

分片上传还支持断点续传。客户端把 UploadId 和已完成的分片列表存下来，下次从断点继续。`coscli`、`aws s3 cp` 这些工具内部都实现了这套逻辑，命令行看起来是一条命令，背后可能是几十个并发请求。

## 13.5 元数据与存储类型

每个对象除了数据，还有一组元数据。系统元数据由服务端维护，包括大小、修改时间、ETag、内容类型。用户元数据是自定义的键值对，以 `x-cos-meta-` 或者 `x-amz-meta-` 开头，可以用来记录业务信息。

```
x-cos-meta-project: video-2dto3d
x-cos-meta-owner: alice
```

ETag 常被误认为是文件的 MD5。对普通上传的单文件，它确实是内容的 MD5；对分片上传的对象，它是各分片 MD5 拼接后再算 MD5，和整个文件的 MD5 不同。用它做完整性校验时要小心，比较靠谱的做法是自己在本地算 MD5 或者 SHA-256，存进用户元数据。

存储类型决定了费用和取回时间。标准存储适合频繁访问的数据，低频存储便宜一些但取回要等，归档存储更便宜但要解冻几个小时。这些类型可以按规则自动转换，比如“创建 30 天后转低频，90 天后转归档”，这就是生命周期规则。

## 13.6 一致性与持久性

对象存储宣传的持久性通常是“11 个 9”，意思是每年丢失数据的概率是千亿分之一。这个数字来自冗余设计，同一份数据存多个副本或者用纠删码编码后分散到多台机器、多个机架，坏掉几个不影响整体。

一致性是另一个维度。早期的 S3 只保证最终一致，上传之后立刻读可能读到旧版本。2020 年底 AWS 宣布 S3 对新建对象的读写提供了强一致性，覆盖列表操作。国内厂商的文档里也有类似说明，但具体行为要看版本。这一点对训练任务有实际影响，上传完数据立刻开始训练，如果读到的是旧版本或者读不到，会很困惑。稳妥的做法是上传后校验，或者等几秒再开始。

## 13.7 工具与挂载

日常使用对象存储有三类工具。

图形界面的客户端，用来浏览桶、上传下载、生成分享链接。这类工具适合偶尔用，不适合批量操作。

命令行工具，比如 `coscli`、`aws s3`、`rclone`。它们支持并发、断点续传、同步目录、按规则过滤，是脚本里的主力。

```
coscli cp ./data/ cos://bucket-1250000000/dataset/ --recursive
coscli sync ./checkpoints/ cos://bucket-1250000000/ckpt/ --delete
```

挂载工具，比如 `cosfs`、`s3fs`、`goofys`。它们把桶挂成一个本地目录，应用程序不用改代码就能读写。这看起来很美好，坑却不少。

第一个坑是延迟。每次读文件都是一次 HTTP 请求，随机读小文件的性能比本地磁盘差两个数量级。第二个坑是随机写。对象存储不支持在文件中间插入数据，很多程序（比如数据库、某些编译工具）会先写临时文件再改名，这种模式在挂载的文件系统上可能失败或者产生额外开销。第三个坑是文件锁。对象存储没有锁的概念，多台机器挂载同一个桶，同时写同一个文件会互相覆盖。

结论是，挂载适合“读多写少、顺序访问”的场景，比如让训练脚本直接读数据集。不适合放数据库、不合适放代码仓库、不合适放需要频繁 rename 的编译输出。

## 13.8 和堡垒机的关系

这两件事看起来无关，其实在企业里经常连在一起。

堡垒机的会话录像要存起来，本地磁盘存不下六个月的数据，于是存到对象存储。训练任务

的日志和模型权重也要存，于是也放对象存储。数据的入口和人的入口，构成了运维体系的两端。

这也带来一个安全上的共同点。两者都依赖凭据，堡垒机托管服务器的密码和私钥，对象存储依赖访问密钥。凭据泄露的后果一样严重。第十四章要讲的签名机制，本质上就是让凭据在使用时不出现在网络上，这一点和 SSH 的公钥认证思路相同。

第 14 章

# 签名、密钥与 403

## 14.1 每个请求都要证明自己

对象存储的接口是 HTTP，服务器不记得你上次来过。每个请求都得自己带上身份证明，否则任何人都能读写你的数据。这种证明就是签名。

签名的思路和 SSH 的公钥认证有相似之处。客户端持有一把密钥，服务端也存一份，客户端用密钥对请求内容算出一个值，附在请求里，服务端用同样的方法算一遍，比对是否一致。差别在于 SSH 用的是非对称签名，对象存储用的是对称的 HMAC，因为双方共享同一把密钥，不需要公钥基础设施。

这套机制有两个直接后果。第一，密钥绝不能泄露，它不像私钥那样有配对的公钥可以公开，拿到它就等于拿到全部权限。第二，请求的内容参与了签名，改一个字节签名就对不上，所以中间人无法篡改。

## 14.2 访问密钥

对象存储的凭据叫访问密钥，由两部分组成，一个公开的 ID 和一个保密的 Key。AWS 叫 Access Key ID 和 Secret Access Key，腾讯云叫 SecretId 和 SecretKey。ID 用来标识身份，Key 用来算签名。ID 可以出现在日志里，Key 一次都不能。

```
SecretId  = AKIDexampleExampleExampleExample
SecretKey = exampleSecretKeyDoNotUseThisInRealLife
```

凭据分几类。主账号的密钥权限最大，能操作账号下所有资源，绝不该写进代码或者放到客户端。子账号（主账号下另外建立的身份，权限可以单独限制）的密钥按策略授权，能限定到某个桶、某个前缀、某些操作。临时密钥由安全令牌服务（云上专门签发短期凭据的服务，常简称 STS）签发，带有效期，用完自动失效，适合分发给临时任务或者前端。

这里有一个常见的错误。有人把密钥写进前端页面的 JavaScript 里，以为别人看不到。前端代码是公开的，任何人打开开发者工具就能读出来。正确做法是后端签发一个临时密钥或者预签名 URL，让前端拿着有限权限的凭据去操作。

## 14.3 签名的计算过程

对象存储的签名算法在业界基本统一，AWS 的叫 Signature Version 4，简称 SigV4，其他厂商大多兼容或借鉴。它的计算分四步，核心思想是把请求的各个部分按固定格式拼成一个字符串，然后逐层做 HMAC。

第一步，构造规范化请求。把 HTTP 方法、路径、查询参数、头部按规则整理成固定格式，再拼上请求体的哈希。

```
CanonicalRequest =
  HTTPRequestMethod + '\n' +
  CanonicalURI + '\n' +
  CanonicalQueryString + '\n' +
  CanonicalHeaders + '\n' +
  SignedHeaders + '\n' +
  HexEncode(Hash(Payload))
```

图 14-1 是这个拼接过程的示意。

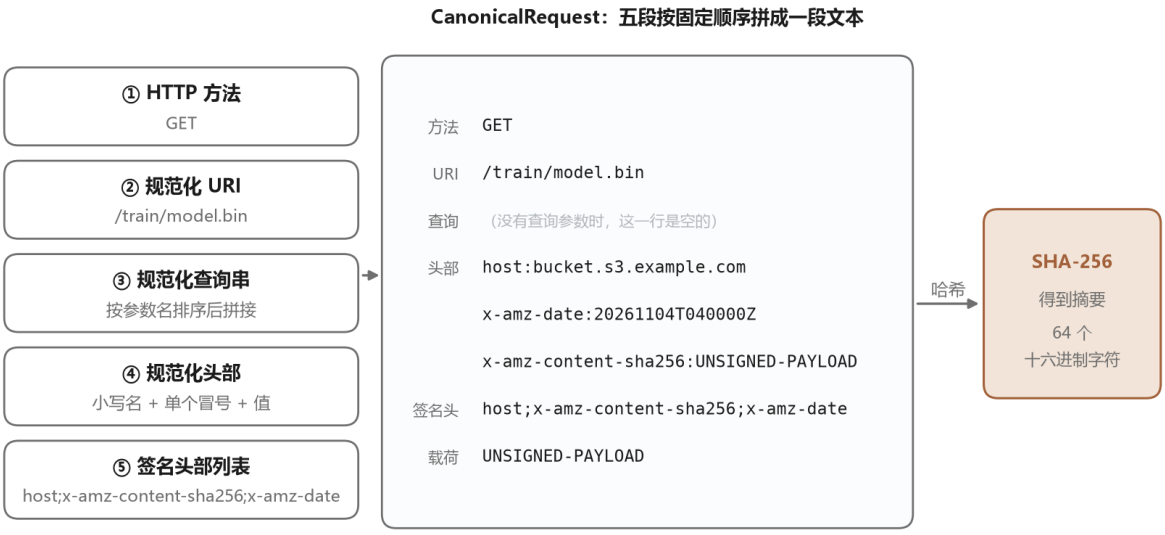



图 14-1 规范化请求的五段内容

每一步都有细节。URI 要做百分号编码，但斜杠不编码。查询参数要按参数名的字典序排列，同名的按值排序。头部的名字要转小写，值要去掉首尾空格、把连续空格压成一个，然后按名字排序。这些规则看起来繁琐，目的是让不同语言、不同库算出的结果一致。签名对不上，十次里有八次是某一步的规范化没做对。

第二步，构造待签字符串。把算法名、时间戳、凭据范围、上一步的哈希拼起来。

```
StringToSign =
  "AWS4-HMAC-SHA256" + '\n' +
  "20261104T040000Z" + '\n' +
  "20261104/ap-guangzhou/s3/aws4_request" + '\n' +
  HexEncode(Hash(CanonicalRequest))
```

第三步，派生签名密钥。这一步是四层 HMAC，每一层用上一层的输出作为密钥。

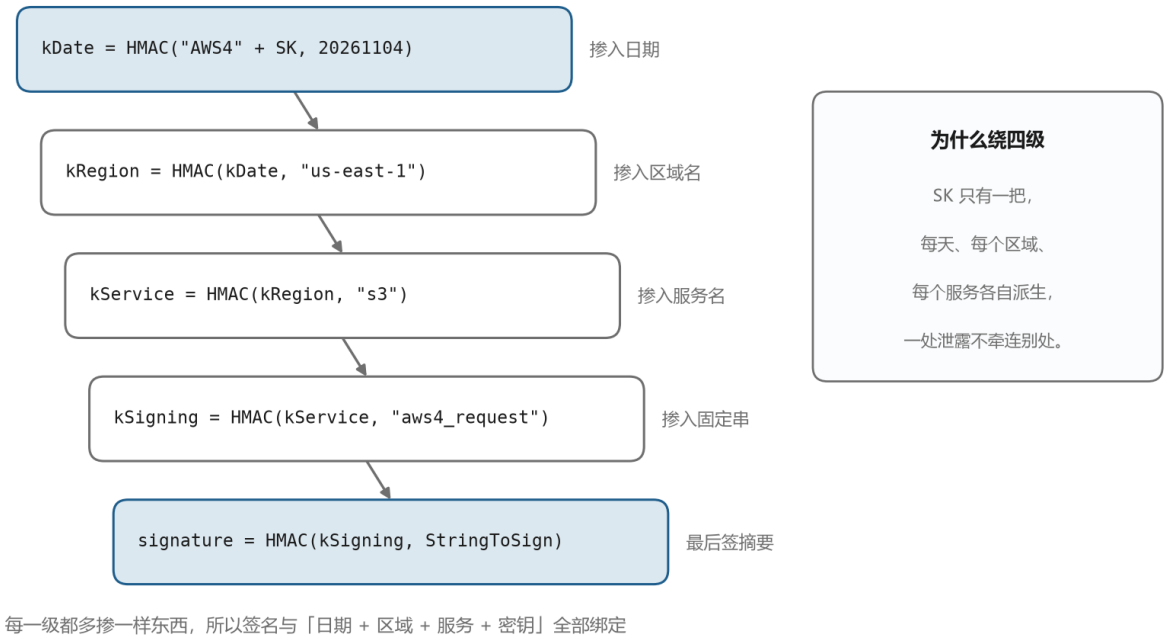



图 14-2 签名密钥的四层派生与最终签名

```
kDate    = HMAC("AWS4" + SecretKey, "20261104")
kRegion  = HMAC(kDate,    "ap-guangzhou")
kService = HMAC(kRegion,  "s3")
kSigning = HMAC(kService, "aws4_request")
```

为什么要派生而不是直接用 SecretKey，原因有两个。一是把密钥和日期、地域、服务绑定，同一个 SecretKey 在不同场景下算出的签名密钥不同，一处泄露不影响其他场景。二是让签名密钥每天变化，减少长期密钥被分析的风险。

第四步，算出签名并放进请求头。

```
Signature = HexEncode(HMAC(kSigning, StringToSign))

Authorization: AWS4-HMAC-SHA256
  Credential=AKID.../20261104/ap-guangzhou/s3/aws4_request,
  SignedHeaders=host;x-amz-content-sha256;x-amz-date,
  Signature=abcdef...
```

服务端收到请求后，用自己存的 SecretKey 重算一遍。算出来的值和你给的一致，就认为请求合法。整个过程中 SecretKey 从未出现在网络上，出现的只有它的运算结果。

这段推导看起来复杂，实际写代码时都用 SDK，不需要手工实现。但理解它有两个好处。一是看到 `SignatureDoesNotMatch` 时知道该检查什么，通常是路径编码、头部排序或者时间戳。二是能理解预签名 URL 的原理，它就是把签名结果放进查询参数。

## 14.4 预签名 URL

有时候你需要让别人临时访问一个对象，但不能把密钥给他，也不想让他走你的服务器中转。这种场景用预签名 URL。

```
https://bucket-1250000000.cos.ap-guangzhou.myqcloud.com/report.pdf
  ?X-Amz-Algorithm=AWS4-HMAC-SHA256
  &X-Amz-Credential=AKID...%2F20261104%2Fap-guangzhou%2Fs3%2Faws4_request
  &X-Amz-Date=20261104T040000Z
  &X-Amz-Expires=3600
  &X-Amz-SignedHeaders=host
  &X-Amz-Signature=abcdef...
```

图 14-3 是这段 URL 的组成。




图 14-3 预签名 URL 的字段构成

原理是把 Authorization 头里的内容挪到查询参数里，签名时把这些参数也算进去。任何拿到这个 URL 的人，在有效期内都能按指定的方法访问这个对象，过期就失效。签名的范围可以限定得很细，只允许 GET、只允许某个对象、只在一小时内有效。

预签名 URL 有几个坑。第一，它是一串明文，谁拿到谁就能用，所以要通过安全的渠道传递，不要贴在公开的群聊里。第二，有效期不能太长，几天甚至几个月的有效期等于长期后门。第三，URL 里的签名基于当时的头部和参数，如果中间有代理改写了头部（比如加了 `X-Forwarded-For`），签名可能对不上，这时要把那些头部从 `SignedHeaders` 里去掉。

## 14.5 临时密钥

比预签名 URL 更灵活的方式是临时密钥。用一个长期密钥去换一组短期的凭据，包含临时的 ID、Key 和一个会话令牌（Token）。请求时要把令牌放在 `x-amz-security-token` 头里，签名时也要把它算进去。

```
sts assume-role --role-arn ... --duration 3600   # pseudo command, syntax varies
```

临时密钥的好处是权限可控、自动过期、可以追溯。给一个训练任务发一组只在两小时内有效的密钥，任务结束后密钥自动失效，即使泄露了也没有长期风险。企业里的最佳实践是主密钥只在签发临时密钥的服务里使用，业务代码一律用临时密钥。

## 14.6 权限是怎么判的

对象存储的权限有三个层次。最外层是身份策略，绑定在子账号或者角色上，规定这个身份能做什么。中间是桶策略，挂在桶上，规定谁能对这个桶做什么，可以指定其他账号。最内层是 ACL（访问控制列表，一条条“谁可以做什么”的记录），挂在桶或者对象上，粒度最粗，通常只在需要公共读的时候用。

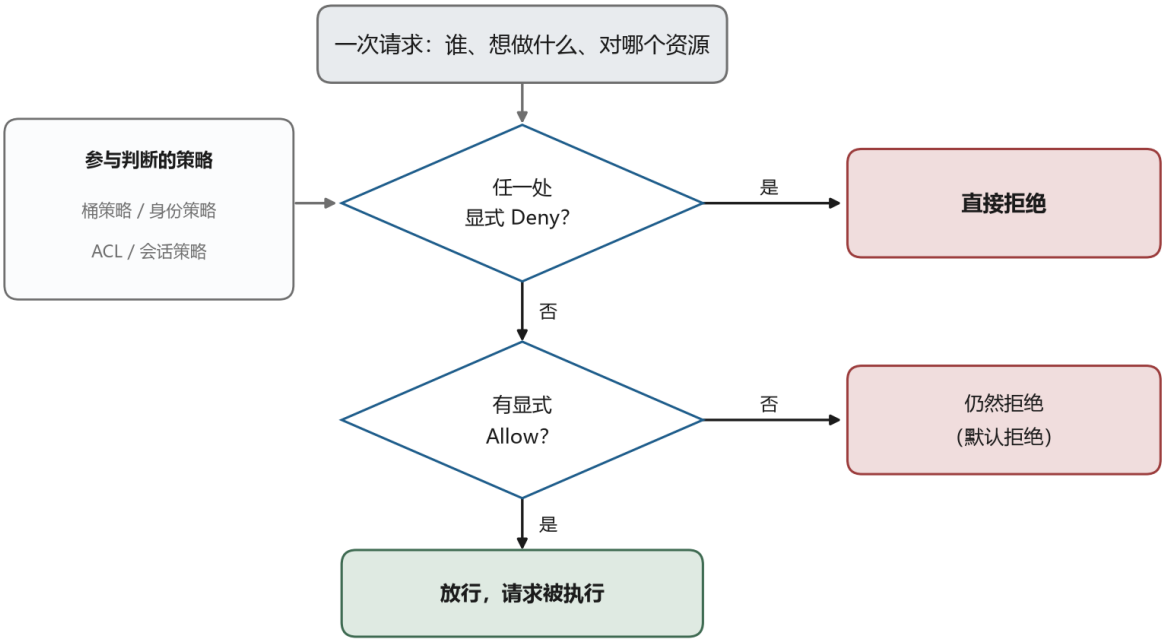



图 14-4 权限求值的顺序，显式拒绝优先

求值顺序如图 14-4。任何一个层次出现显式拒绝，请求就被拒绝，其他层次的允许都不作数。没有任何拒绝，但只要有一处显式允许，请求就通过。既没有拒绝也没有允许，默认拒绝。

这个顺序解释了一个常见困惑。你在身份策略里给了 `s3:*`，为什么还是 403。因为桶策略里有一条显式拒绝，或者桶的公共访问被组织级策略锁死了。排查时要把三个层次都看一遍，而

不是只改自己手里那个。

另一个常见困惑是权限的粒度。对象存储的权限可以精确到前缀，比如只允许访问 `dataset/` 下面的对象，不允许访问 `ckpt/`。这在多人共享一个桶时很有用，每人一个前缀，互不干扰。配置时注意前缀要以斜杠结尾，否则 `dataset` 会匹配到 `dataset-backup`。

## 14.7 403 家族

表 14-1 是几个最常遇到的错误码和它们的成因。看到错误码先查这张表，能省很多时间。

表 14-1 对象存储常见错误码

| 错误码 | 含义 | 常见原因 |
|---|---|---|
| `SignatureDoesNotMatch` | 签名不匹配 | 路径编码不对、头部没排序、地域写错、SecretKey 错 |
| `RequestTimeTooSkewed` | 请求时间偏差过大 | 本机时钟不准，与服务器相差超过 15 分钟 |
| `InvalidAccessKeyId` | 密钥 ID 不存在 | 密钥被删、复制时漏了字符、用了别的账号的密钥 |
| `AccessDenied` | 没有权限 | 策略没授权、被显式拒绝、桶策略限制来源 IP |
| `NoSuchBucket` | 桶不存在 | 桶名拼错、地域写错、桶已被删除 |
| `NoSuchKey` | 对象不存在 | 键名拼错、前缀不对、对象已过期被生命周期删除 |
| `InvalidRequest` | 请求格式错误 | 分片上传的片序号越界、Content-Length 与实际不符 |

地域写错是最隐蔽的一个。COS 的桶在某个地域，请求必须发到那个地域的域名，签名里的凭据范围也要写那个地域。如果你的 SDK 配置成了 `ap-shanghai`，桶却在 `ap-guangzhou`，请求会先被重定向，签名随之失效，报出来的错却是 `SignatureDoesNotMatch`。遇到这个错误先检查地域，再检查密钥。

时钟偏差是第二隐蔽的。虚拟机的时钟偶尔会漂移，尤其是长期挂起的实例恢复之后。签名里带了时间戳，服务端允许的正负偏差通常是 15 分钟，超出就拒绝。

```
date -u
sudo systemctl restart systemd-timesyncd
```

这两条命令能确认和修复时间问题。云端机器建议开启 NTP 同步，避免这类莫名其妙的失败。

## 14.8 几条实践建议

把这一章的内容收拢成几条可执行的建议。

密钥分级。主密钥只放在签发临时凭据的服务里，业务用子账号密钥，临时任务用临时密钥。任何密钥都不要写进代码仓库，用环境变量或者密钥管理服务注入。

权限最小化。给一个任务只授它需要的桶和前缀，只给 GET 就不给 PUT。上线前用一组只读密钥跑一遍，确认没有多余的写操作。

打开访问日志。桶的访问日志能记录每个请求的来源、操作、结果，出问题时是唯一的证据。日志本身也存到一个单独的桶里，设置较长的保留期。

关闭公共读。除非确实要做公开的资源分发（比如网站静态文件），否则桶一律私有。公共读的桶会被爬虫和扫描器盯上，数据泄露往往就是这么发生的。

定期轮换。长期密钥每季度换一次，换的时候先在代码里加新密钥，确认生效后再停用旧的。临时密钥的有效期尽量短，够用就行。

第 15 章

# 从登录到数据存取

## 15.1 一个真实的工作流

前面十四章各自讲了一个部件，这一章把它们装到一起。

设想这样的场景。你在办公室的 Windows 电脑上写代码，代码要跑在实验室的一台 GPU 服务器上，那台服务器在机房内网，外面进不去，只有一台堡垒机暴露在公网。训练数据有几百 GB，放在对象存储的一个桶里，模型权重也要定期存回桶里。你希望能在本地的 VS Code 里编辑远端代码、开终端、跑训练，同时让训练脚本自己从桶里读数据、往桶里写检查点。

这个场景里用到了四样东西，SSH 客户端、堡垒机、VS Code 的远程开发、对象存储的 API。它们分别对应这本书的四篇。图 15-1 是整体的拓扑。

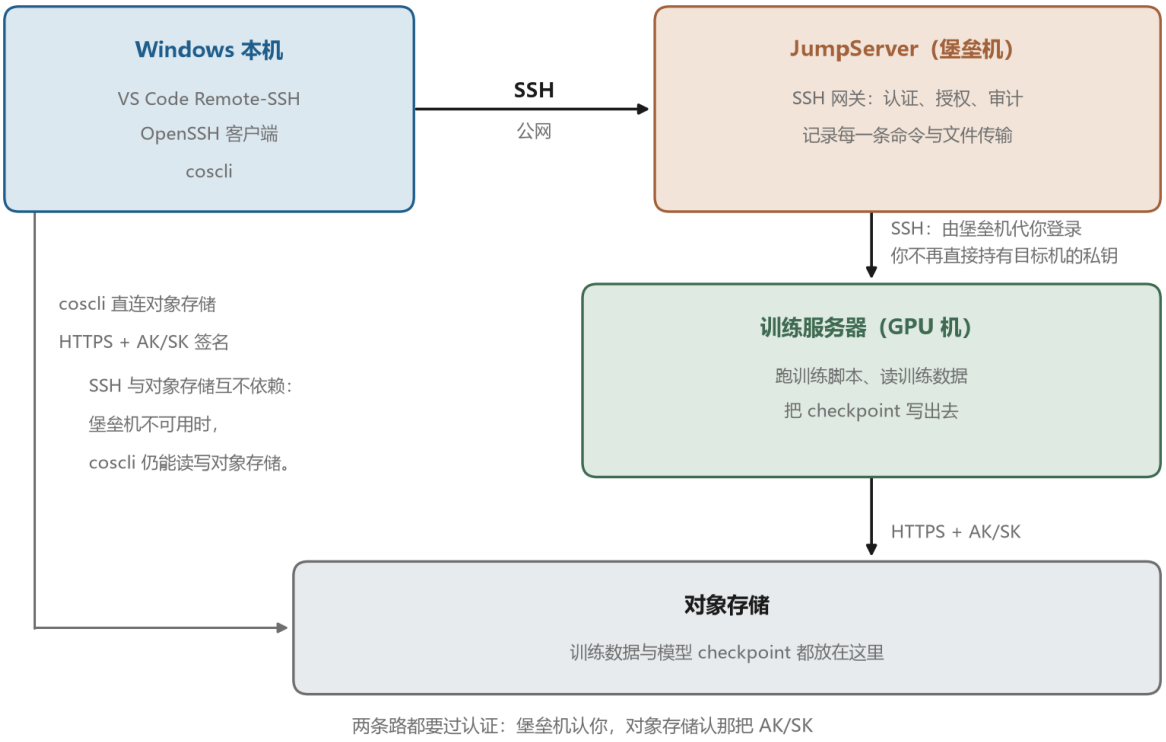
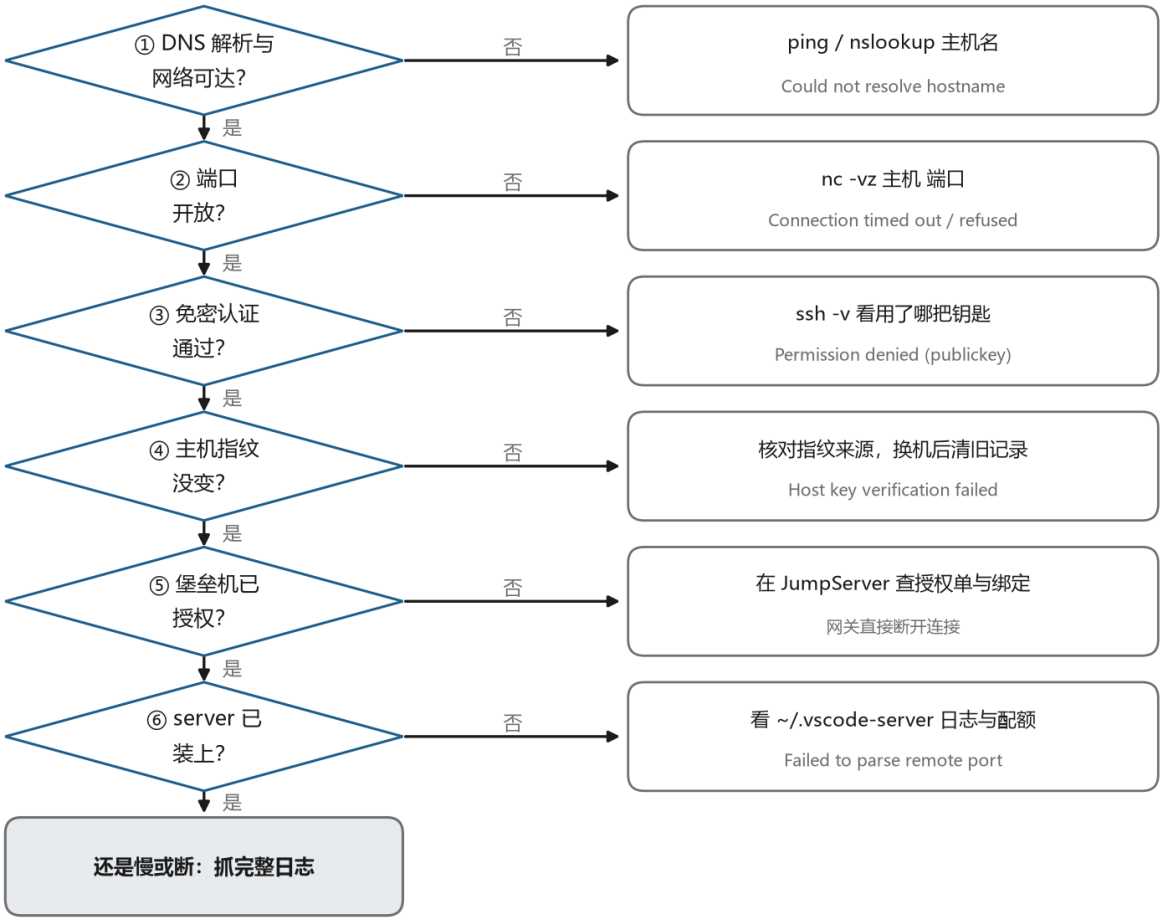



图 15-1 从本机到训练机再到对象存储的完整链路

图里有四条链路，每条都有自己的认证方式。本机到堡垒机是一次 SSH 认证，用的是你的私钥。堡垒机到训练机是另一次认证，用的是堡垒机托管的凭据，你看不到。本机到训练机之间还有一条逻辑上的 VS Code 通道，它跑在前两条连接之上。训练机到对象存储是一次 HTTPS 请求，用的是访问密钥的签名。

把这四条链路的认证对象分清楚，排错时就不会乱。图 15-2 是本机侧常见问题的判断路径，后面几节会用到。




按顺序问下去，第一个答「否」的地方就是问题所在

图 15-2 从连不上到具体原因的判断路径

## 15.2 第一步，把登录打通

一切从一条能用的 SSH 连接开始。在本机建好配置文件，内容大致如下。

```
Host lab-gpu
    HostName 203.0.113.24
    Port 58222
    User alice@ubuntu@10.0.0.9
    IdentityFile C:/Users/alice/.ssh/ssh.jumpserver.pem
    IdentitiesOnly yes
    ServerAliveInterval 30
    ServerAliveCountMax 3
```

五行核心配置，第十二章已经逐行拆过。这里补充三行可选项。`IdentitiesOnly yes` 让客户端只用指定的这把钥匙，不去试 agent 里的其他钥匙，避免因为尝试次数过多被服务器拒绝。两个 `ServerAlive` 选项负责保持连接，防止中间设备把闲置的会话清掉。

配好之后先别急着开 VS Code，在终端里试一下。

```
ssh -v lab-gpu
```

能进到远端的提示符，说明网络与认证这两关都过了。进不去的话，按图 15-2 从上往下查。先确认域名或者地址能通，再确认端口开放，再看认证。

```
Test-NetConnection 203.0.113.24 -Port 58222
```

这条命令在 Windows 上测端口连通性，比 `ping` 靠谱，因为很多服务器禁 ping 但端口是开的。如果端口不通，问题在网络或者云平台的放行规则，和密钥无关。

认证失败时，报错通常是 `Permission denied (publickey)`。这时依次检查四件事，私钥文件是否存在、权限是否过宽、公钥是否已经登记在堡垒机、用户名里的三段是否写对。第三件事最容易被忽略，很多人把公钥加到了目标机的 `authorized_keys` 里，却忘了在堡垒机上登记，结果怎么都不通。

## 15.3 第二步，让编辑器连上

终端能登录之后，VS Code 的远程连接通常一次就成。装好 Remote-SSH 扩展，按 `F1` 调出命令面板，选“连接到主机”，输入 `lab-gpu`。

第一次连接会慢一些，因为要在远端安装 `vscode-server`。如果远端的家目录所在分区空间紧张，这一步会失败。先看一下余量。

```
df -h ~
```

装好之后，界面左下角会显示远端的主机名，表示已经连上。这时打开远端的项目目录，编辑、保存、开终端都在远端进行。你的本机只负责显示。

如果卡在“正在下载 VS Code Server”，说明远端下载失败。可以在本机下载对应的安装包，用 `scp` 传上去手工解压。安装包的地址在输出面板的日志里能找到，形如 `https://update.code.visualstudio.com/commit/<commit-id>/server-linux-x64/stable`。

```
wget https://update.code.visualstudio.com/commit/<commit-id>/server-linux-x64/stable
mkdir -p ~/.vscode-server/bin/<commit-id>
tar -xzf stable -C ~/.vscode-server/bin/<commit-id> --strip-components=1
```

这条路在离线环境里很常用，值得记下来。`〈commit-id〉` 在 VS Code 的“关于”对话框里能找到。

## 15.4 第三步，让训练机读写桶

训练脚本要读写对象存储，需要在训练机上配置凭据。这一步最容易出事，因为凭据一旦落到磁盘上，就成了一条长期风险。

最稳妥的做法是用临时密钥。用一个有签发权限的身份去换一组只在几小时内有效的凭据，注入到环境变量里，不写文件。

```
export COS_SECRET_ID="<TEMP_SECRET_ID>"
export COS_SECRET_KEY="<TEMP_SECRET_KEY>"
export COS_SESSION_TOKEN="<TEMP_SESSION_TOKEN>"
```

如果没有临时密钥的签发能力，退一步用子账号密钥，把它写进只有自己能读的配置文件，权限 600。

```
mkdir -p ~/.coscli
cat > ~/.coscli/config.yaml <<'EOF'
cos:
  base:
    secretid: <YOUR_SECRET_ID>
    secretkey: <YOUR_SECRET_KEY>
    sessiontoken: ""
  buckets:
    - name: dataset-1250000000
      alias: dataset
      region: ap-guangzhou
EOF
chmod 600 ~/.coscli/config.yaml
```

配置好之后先做一次最小验证，列一下桶里的内容，确认权限没问题。

```
coscli ls cos://dataset/
coscli ls cos://dataset/train/ --limit 20
```

能列出来，说明读权限有了。再试一次写，往一个临时前缀里传一个小文件。

```
echo test > /tmp/probe.txt
coscli cp /tmp/probe.txt cos://dataset/tmp/probe.txt
coscli rm cos://dataset/tmp/probe.txt
```

这三条命令验证了列、写、删三个动作。很多 403 就是在这个阶段暴露出来的，早发现比训练跑到一半才发现好。

## 15.5 第四步，把结果取回来

训练结束，检查点和日志要取回来。小文件直接下载，大文件用分片加快，中断了能续。

```
coscli cp cos://dataset/ckpt/epoch-10.pt ./ckpt/epoch-10.pt
coscli sync cos://dataset/logs/ ./logs/ --include "*.txt"
```

`sync` 只传输有差异的文件，适合日志这种持续增长的内容。`--include` 和 `--exclude` 支持通配符，可以精确控制传哪些。传大文件时注意本机磁盘空间，一个模型检查点几个 GB 很常见。

如果要把结果分享给别人，不要直接给桶的公共链接，用预签名 URL，设一个合理的有效期。

```
coscli sign cos://dataset/ckpt/epoch-10.pt --expire 3600
```

这条命令返回一个带签名的链接，一小时后失效。分享之前确认链接里没有敏感信息，因为 URL 里的路径本身就是对象名。

## 15.6 排错的时候按顺序来

图 15-2 的决策树可以展开成一张检查清单。遇到问题，从上往下逐项确认，不要跳步。

先看能不能通。`Test-NetConnection` 测端口，`ping` 测地址，`tracert` 看路径。这一步不通，后面全是白费。

再看认证。`ssh -v lab-gpu` 的输出里找 `Authentications that can continue`，它列出了服务器接受的认证方式。如果只有 `publickey`，说明密码登录被禁用，得用密钥。如果连这一行都没有，问题在更早的阶段。

然后看授权。堡垒机的授权是独立于 SSH 认证的一层，SSH 认证通过不代表你有权限连某台资产。表现是连接建立后立刻被断开，或者提示没有权限。这时要去堡垒机界面确认授权是否包含这台资产和这个系统用户。

接着看隧道。VS Code 连上但界面转圈，多半是端口转发被禁用。在输出面板里找 `Remote-SSH` 的日志，如果看到 `open failed: administratively prohibited`，就是这个原因。

最后看对象存储。`SignatureDoesNotMatch` 查地域和密钥，`AccessDenied` 查策略，`RequestTimeTooSkewed` 查时钟。三条线索对应三个不同的层面，不要混着改。

## 15.7 一份可以照着做的检查清单

把全书的安全要点收成一份清单，部署或者接手一个新环境时可以逐条对照。

凭据方面。私钥权限 600，目录 700。密钥设口令，口令进 agent。不用共享账号，一人一把钥匙。离职当天回收授权，不只是删机器上的公钥，还要删堡垒机上的账号。对象存储的主密钥不进代码，业务用子账号，临时任务用临时密钥。

连接方面。关闭密码登录，只留公钥。关闭 root 直接登录。改默认端口能减少噪音但不是

安全保障。`known_hosts` 的警告要认真看，不要习惯性删掉。

堡垒机方面。默认禁止端口转发和复制粘贴，按需开放。开启会话录像和命令记录，保留期不少于六个月。录像存到对象存储，别放本地磁盘。堡垒机本身单独划分安全域，只对运维网段开放。

对象存储方面。桶一律私有，公共读要单独评估。开启访问日志。配置生命周期，清理未完成的分片和过期对象。跨域规则只对需要的域名开放，不要写 `*`。

这些条目看起来琐碎，每一条背后都有具体的事故。把它们做成检查表，新环境上线前过一遍，比事后复盘便宜得多。

## 15.8 最后

这本书从一台机器讲到了一群机器，从人的入口讲到了数据的出口。中间经过了网络分层、密码学、协议握手、堡垒机的权限模型、编辑器的进程结构、对象存储的签名算法。这些内容看起来分散，其实都在回答同一个问题，一个远方的资源，凭什么相信你，你又凭什么相信它。

SSH 用密钥和签名回答这个问题，堡垒机用集中管理和审计回答这个问题，对象存储用 HMAC 签名回答这个问题。手段不同，思路一致。理解了这一点，再遇到新的工具、新的协议，你也能顺着这条线索把它拆开。

剩下的就是动手。把配置文件改一改，把 `ssh -v` 的输出读一读，把桶的权限调一调，看看会发生什么。工具的价值在使用中才会显现，而这些原理的价值，在于让你在使用时知道自己在做什么。

# 附录 A 命令速查

## A.1 客户端连接

| 命令 | 作用 |
| --- | --- |
| `ssh host` | 用配置文件里的别名连接 |
| `ssh -p 2222 user@host` | 指定端口连接 |
| `ssh -l user@asset@jump jump` | 指定用户名，用户名里含 at 符号时用它 |
| `ssh -i key.pem user@host` | 指定私钥 |
| `ssh -v host` | 打印调试信息，`-vv -vvv` 更详细 |
| `ssh -N -L 8080:10.0.0.9:80 jump` | 只做转发，不登录 shell |
| `ssh -J jump1,jump2 target` | 两级跳板 |
| `ssh -D 1080 jump` | 起一个 SOCKS5 代理 |
| `ssh -T host` | 不分配伪终端，适合脚本 |
| `ssh -t host cmd` | 强制分配伪终端 |
| `ssh -o Option=Value host` | 临时指定配置项 |
| `ssh -G host` | 打印最终生效的配置，不发起连接 |

## A.2 密钥管理

| 命令 | 作用 |
| --- | --- |
| `ssh-keygen -t ed25519 -C "note"` | 生成 Ed25519 密钥 |
| `ssh-keygen -t rsa -b 4096` | 生成 4096 位 RSA 密钥 |
| `ssh-keygen -l -f key.pub` | 查看公钥指纹 |
| `ssh-keygen -y -f key` | 从私钥导出公钥 |
| `ssh-keygen -p -f key` | 修改私钥口令 |
| `ssh-keygen -p -m PEM -f key` | 转换成 PEM 格式 |
| `ssh-keygen -R host` | 从 known_hosts 删除某主机的记录 |

| 命令 | 作用 |
| --- | --- |
| `ssh-keygen -F host` | 在 known_hosts 里查找记录 |
| `ssh-copy-id -i key.pub user@host` | 把公钥装到远端 |
| `ssh-keyscan -t ed25519 host` | 抓取主机公钥 |

## A.3 agent

| 命令 | 作用 |
| --- | --- |
| `eval "$(ssh-agent -s)"` | 启动 agent |
| `ssh-add key` | 把私钥加入 agent |
| `ssh-add -l` | 列出已加载的钥匙 |
| `ssh-add -D` | 清空所有钥匙 |
| `ssh-add -t 3600 key` | 加载并在一小时后自动移除 |

## A.4 诊断

| 命令 | 作用 |
| --- | --- |
| `ss -lntp` | 列出监听的 TCP 端口和进程 |
| `ss -tnp` | 列出已建立的连接 |
| `nc -vz host port` | 测试端口连通性 |
| `curl -v telnet://host:port` | 测试端口并看握手过程 |
| `traceroute host` | 逐跳显示路径 |
| `tcpdump -n -i any port 22` | 抓 SSH 流量 |
| `journalctl -u ssh -n 50` | 看 sshd 最近日志 |
| `sudo sshd -T` | 打印服务端生效配置 |

## A.5 服务端

| 命令 | 作用 |
| --- | --- |
| `systemctl status ssh` | 查看 sshd 状态 |
| `systemctl restart ssh` | 重启 sshd |
| `sshd -t` | 检查配置文件语法 |
| `chmod 700 ~/.ssh` | 修正目录权限 |
| `chmod 600 ~/.ssh/authorized_keys` | 修正公钥文件权限 |
| `chmod 600 ~/.ssh/id_ed25519` | 修正私钥权限 |

## A.6 VPN 与接入

| 命令 | 作用 |
| --- | --- |
| `wg show` | 查看 WireGuard 隧道状态、最近握手与流量 |
| `wg-quick up wg0` | 启用一份 WireGuard 配置 |
| `ip addr` | 看虚拟网卡有没有拿到地址 |
| `ip route` | 查看路由表，Linux |
| `route print` | 查看路由表，Windows |
| `ip route get 10.0.0.9` | 查这个目标实际从哪块网卡出去 |
| `ping -M do -s 1400 10.0.0.9` | 探测隧道 MTU，Linux |
| `ping -f -l 1400 10.0.0.9` | 探测隧道 MTU，Windows |
| `tracepath 10.0.0.9` | 逐跳路径，顺带发现路径 MTU |
| `nmcli connection show` | 列出本机的网络与 VPN 连接 |

## A.7 对象存储

| 命令 | 作用 |
| --- | --- |
| `coscli ls cos://bucket/prefix/` | 列出对象 |
| `coscli cp local cos://bucket/key` | 上传单个文件 |
| `coscli cp cos://bucket/key local` | 下载单个文件 |
| `coscli cp dir/ cos://bucket/prefix/ -r` | 递归上传 |

| 命令 | 作用 |
| --- | --- |
| `coscli sync local/ cos://bucket/prefix/` | 增量同步 |
| `coscli rm cos://bucket/key` | 删除对象 |
| `coscli sign cos://bucket/key --expire 3600` | 生成预签名 URL |
| `curl -I "https://bucket.cos.region.myqcloud.com/key"` | 查看对象头部信息 |

## A.8 常用配置项

| 配置项 | 客户端或服务端 | 说明 |
| --- | --- | --- |
| `HostName` | 客户端 | 真实地址 |
| `Port` | 客户端 | 端口 |
| `User` | 客户端 | 登录名 |
| `IdentityFile` | 客户端 | 私钥路径 |
| `IdentitiesOnly` | 客户端 | 只用指定私钥 |
| `ProxyJump` | 客户端 | 跳板机 |
| `ServerAliveInterval` | 客户端 | 心跳间隔 |
| `LocalForward` | 客户端 | 等价于 `-L` |
| `RemoteForward` | 客户端 | 等价于 `-R` |
| `RequestTTY` | 客户端 | 是否分配伪终端 |
| `PasswordAuthentication` | 服务端 | 是否允许密码登录 |
| `PermitRootLogin` | 服务端 | 是否允许 root 登录 |
| `PubkeyAuthentication` | 服务端 | 是否允许公钥登录 |
| `AllowTcpForwarding` | 服务端 | 是否允许端口转发 |
| `GatewayPorts` | 服务端 | 远程转发能否绑定非回环地址 |
| `MaxSessions` | 服务端 | 单连接最大通道数 |
| `ClientAliveInterval` | 服务端 | 服务端心跳间隔 |
| `TrustedUserCAKeys` | 服务端 | 用户证书的 CA 公钥 |

# 附录 B 术语表

**SSH** Secure Shell，一套用于在不安全网络上安全访问远端主机的协议族，由传输层、用户认证层、连接层三个子协议组成。

**sshd** SSH 的服务端程序，通常在 22 端口监听，接受连接、完成认证、分配会话。

**OpenSSH** 最广泛使用的 SSH 实现，包含客户端 `ssh`、服务端 `sshd`、密钥工具 `ssh-keygen` 等。

**公钥认证** 客户端用私钥对一段包含会话标识的数据签名，服务端用对应公钥验证，从而确认客户端持有私钥。

**密钥** 能用于加密、解密或签名的一段数据，通常是一长串随机数。非对称的一对里分公钥与私钥，对称加密只有一把。

**私钥** 只能由持有者保存的密钥，用于签名或者解密，权限必须是 600。

**公钥** 可以公开分发的密钥，用于验证签名或者加密，以一行文本形式存放在 `authorized_keys` 中。

**签名** 用私钥（或共享密钥）对数据算出的值，对方用对应的公钥（或同一把密钥）能验证它出自谁手、内容有没有被改。

**指纹** 对公钥做哈希得到的短字符串，用来快速比对两把公钥是否相同。

**known_hosts** 客户端记录见过的服务器主机公钥的文件，用于检测主机身份变化。

**TOFU** Trust On First Use，首次使用即信任，第一次连接时记录主机公钥，之后比对。

**证书** 由 CA 签发的一份声明，把一把公钥和某个身份绑定起来，并写明有效期。

**主机密钥** 服务器用于证明自己身份的一对密钥，安装 sshd 时自动生成。

**authorized_keys** 服务端文件，列出允许登录的公钥，每行一把，可在行首加限制选项。

**ssh-agent** 后台进程，保管解密后的私钥，替客户端完成签名，避免反复输入口令。

**Agent Forwarding** 把 agent 的签名能力转发到远端主机，方便连续跳转，但有被滥用的风险。

**ProxyJump** 先连跳板机再连目标的连接方式，第二段认证由本机完成。

**端口转发** 把某个 TCP 端口上的流量通过 SSH 通道转送到另一端，分本地、远程、动态三种。

**SOCKS** 一种代理协议，客户端在运行时指定目标地址，常用于动态转发。

**TCP** 传输层的可靠字节流协议，靠序号、确认、重传保证不丢不乱，代价是握手、状态与拥塞控制。

**UDP** 传输层的无连接协议，头部只有 8 字节，不保证送达与顺序，可靠性由应用自己补，

适合 DNS、音视频与 VPN。

**QUIC** 跑在 UDP 之上的可靠传输协议，HTTP/3 的底座，把每条流分开重传，避免 TCP 的队头阻塞。

**VPN** 虚拟专用网（Virtual Private Network），在本机与远端网络之间建一条加密隧道，并把整台机器发往内网网段的流量都改从隧道走，使本机在逻辑上成为那个内网的一台机器。与 SSH 端口转发的区别在于它改的是全局路由，程序无感。

**隧道** 把一种协议的包整体装进另一种协议的包里传输，两端各拆一次，是 VPN 与端口转发共用的基本手法。

**封装** 在原始数据外面再加一层头部的过程。封装会占用字节，使可用的 MTU 变小。

**虚拟网卡** 由软件模拟出来的网络接口，没有物理形态。VPN 用它给本机在内网里安一个身份。

**路由表** 记录“发往哪个网段、从哪块网卡出去”的表，决定一个包走本地还是走隧道。

**全隧道** 把默认路由指向隧道的配置方式，所有流量都绕行远端出口，审计完整但速度受限。

**拆分隧道** 只把内网网段放进隧道、其余流量走本地网络的配置方式，速度快但审计有缺口。

**IPsec** 网络层的一组安全协议，用 IKE 协商密钥，用 ESP 封装与加密数据，常见于站点到站点互联。

**IKE** IPsec 的密钥协商协议，负责在两端之间建立安全关联。

**SSL VPN** 跑在 TLS 之上的 VPN，常用 443 端口，因为和 HTTPS 同端口，穿透性最好。

**OpenVPN** 基于 TLS 的开源 VPN 实现，默认使用 UDP 1194。

**WireGuard** 新一代 VPN 实现，算法组合固定，配置只有几十行，2019 年后进入 Linux 内核。

**802.1X** 链路层的端口准入认证。客户端是申请者，交换机是认证者，RADIUS 服务器做判断，通过后才放行端口。

**EAP** 可扩展认证协议，802.1X 中客户端与认证服务器之间跑的认证框架，密码、证书、令牌都能装进去。

**EAPoL** EAP over LAN，客户端与交换机之间承载 EAP 的链路层帧，端口未授权时只放行它。

**RADIUS** 集中式认证、授权、计费协议，接收交换机的查询并返回放行或拒绝。

**Portal 认证** 先拿到 IP、再通过网页登录换取放行的认证方式，常见于校园网与酒店网络。

**准入控制** 决定“谁可以接入这张网”的一类机制的总称，802.1X 与 Portal 都属于它。

**iNode** H3C 的接入客户端，把 802.1X、Portal 与 VPN 拨号装在一个程序里，常见于高校校园网。

**shell** 夹在用户与内核之间的命令解释器，负责拆词、展开、连接与执行。`bash`、`zsh`、`sh` 都是它的实现，登录用哪一个写在 `/etc/passwd` 里。

**内核** 操作系统中直接管理进程、内存、文件系统与网络的部分，只接受系统调用，不接受命令行。

**内建命令** 由 shell 自己执行的命令，如 `cd`、`export`、`exit`，不需要另起进程，因为要改的是 shell 自己的状态。

**伪终端** 用一对虚拟设备模拟终端，master 端由 sshd 持有，slave 端交给 shell。

**会话密钥** 一次连接临时协商出来的对称密钥，用完即弃，泄露了也影响不到别的会话。

**对称加密** 加密与解密使用同一把密钥，速度快，用于加密实际数据。

**非对称加密** 使用公钥与私钥一对密钥，速度慢，用于密钥交换与签名。

**AES** 最常见的对称加密算法，分组长度 128 位，密钥长度 128、192 或 256 位。

**ChaCha20** 流密码，在没有 AES 硬件加速的设备上性能更好。

**GCM** 一种认证加密模式，同时提供加密与完整性校验。

**哈希** 把任意长度输入压成固定长度输出的函数，不可逆，用于比对与指纹。

**SHA-256** 输出 256 位的哈希函数，SSH 与对象存储签名中广泛使用。

**MAC** 消息认证码（Message Authentication Code），用密钥和消息一起算出的校验值，用于判断消息有没有被改、是不是持有密钥的人发的。

**MAC 地址** 网卡的硬件编号，出厂时写死，链路层用它寻址。和上面的消息认证码只是缩写相同，两者无关。

**HMAC** 带密钥的哈希，用于验证消息完整性，只有持有密钥的人能算出正确值。

**Diffie-Hellman** 一种密钥交换协议，双方各自算出相同的共享密钥，而共享密钥从不在网络上传输。

**ECDH** 基于椭圆曲线的 Diffie-Hellman，密钥更短，计算更快。

**Curve25519** 一条适合密钥交换的椭圆曲线，对应 SSH 算法 `curve25519-sha256`。

**Ed25519** 基于 Curve25519 的签名算法，密钥 32 字节，签名 64 字节。

**RSA** 基于大整数分解困难性的算法，可用于加密与签名，密钥通常 2048 位以上。

**前向保密** 长期密钥泄露后，过去的会话仍然无法解密，前提是使用了临时密钥交换。

**重密钥** 在连接使用一段时间或传输一定数据后重新协商密钥，不影响上层会话。

**会话标识** 第一次密钥交换产生的哈希，整条连接内不变，用于绑定签名与重密钥。

**KEXINIT** 算法协商消息，双方各发一次，列出各自支持的算法清单。

**ETM** Encrypt-then-MAC，先认证再解密的数据包处理顺序，OpenSSH 默认使用。

**通道** SSH 连接内的逻辑数据流，一条连接可同时承载多个通道。

**全局请求** 独立于通道的请求，如请求服务端监听端口（远程转发）。

**堡垒机** 集中管控运维访问的设备，提供统一认证、授权、账号托管与操作审计。

**4A** 认证、授权、账号、审计四个能力的合称。

**JumpServer** 开源堡垒机，组件包括 Core、Luna、Koko、Lion、Magnus、Razor、Chen。

**Koko** JumpServer 的字符协议网关，处理 SSH、SFTP、Telnet。

**系统用户** JumpServer 中用于连接资产的凭据模板，包含用户名、认证方式、密码或私钥。

**资产授权** 把用户、资产、系统用户、动作绑定在一起的记录。

**三合一用户名** 网关模式下 `用户@系统用户@资产地址` 形式的登录名。

**会话录像** 记录终端原始字节流与时间戳的文件，可回放当时的屏幕内容。

**Remote-SSH** VS Code 扩展，通过系统 ssh 客户端连接远端并在远端运行 `vscode-server`。

**vscode-server** VS Code 在远端运行的服务，提供扩展宿主、文件服务、终端服务。

**对象存储** 以 HTTP 接口存取整个对象的存储服务，无目录树，容量近乎无限。

**桶** 对象的容器，名字全局唯一，访问时用于拼域名。

**键** 对象的名字，可含斜杠，但不表示目录层次。

**ETag** 对象的标识，单文件上传时等于内容 MD5，分片上传时是分片 MD5 的组合。

**分片上传** 把大文件切成多片分别上传，最后合并，支持并发与断点续传。

**预签名 URL** 把签名放进查询参数的临时访问链接，在有效期内可用。

**SigV4** AWS 的签名算法第四版，被多数对象存储兼容，核心是规范化请求加四层 HMAC。

**规范化请求** 把方法、路径、查询串、头部按固定规则整理成标准形式，是签名的输入。

**访问密钥** 对象存储的凭据，由公开的 ID 与保密的 Key 组成。

**临时密钥** 由安全令牌服务签发的短期凭据，带有效期与会话令牌。

**生命周期规则** 按时间自动转换存储类型或删除对象的规则。

**纠删码** 把数据编码成多份分片分散存放，坏掉若干份仍可恢复，用于提升持久性。

# 后 记

写这本书的想法，来自一个很具体的观察。远程操作这套东西在工程实践里用得极广，可它的知识分布得很散。协议细节在 RFC 里，配置项在手册页里，堡垒机的概念在厂商文档里，对象存储的签名规则在开发者指南里，而把这些串起来的那条线索，通常只存在于带过你的人的口头传授中。没有这条线索，每个知识点都能查到，却拼不成一个整体。

写作过程中最难处理的是分寸。讲浅了，读者还是不知道为什么；讲深了，一半人会在第四章放弃。最后的做法是，每个概念都从它要解决的问题讲起，公式只保留理解原理必需的那几个，比如密钥派生的六个字母、签名密钥的四层派生。这些地方含糊过去，后面就全是黑箱。

书里刻意把“直连”和“经过堡垒机”分开讲。远程访问并不需要一个中间设备，SSH 本来就是点对点的协议，堡垒机是组织规模变大之后的产物。很多教材把企业环境里的形态当成默认形态，读者就会以为没有堡垒机就没法远程登录。这个误解会让人在面对一台普通的云主机时不知所措。

同样被刻意分开的还有协议与实现。SSH 是协议，OpenSSH 是实现，JumpServer 是建立在这套协议之上的系统，对象存储是另一条线上的技术。把这三者混着讲，读者会记住一堆产品名，却不知道遇到新工具时该往哪儿看。

技术书总有过时的一天。协议会演进，界面会改版，参数会增删。如果哪天书里的某条命令跑不通了，先去看那个工具的手册，再用书里的原理去判断差异。原理的寿命比参数长，这也是这本书把大量篇幅花在原理上的原因。

书中的地址、用户名、密钥文件名都是示例，不对应任何真实资产。读者在自己的环境里遇到的具体问题，未必能在书里找到一模一样的答案，但应该能找到一条可以顺着往下查的线索。这也是这本书真正想留下的东西。

Dr. Dezeming

2026 年秋

# 这本书讲什么

一条 ssh 命令背后有多少层协议，为什么要先加密再认证，
堡垒机的“用户@系统用户@资产”是怎么被解析的，
VS Code 在你和目标机之间架了什么，
对象存储的签名为什么总是对不上。

---

知道为什么这么做，配置错了能自己看出来，
报错信息能自己读懂，别人给的指南也能自己判断。

**Dr. Dezeming**

远程之门 · 2026